

University of Nebraska - Lincoln

DigitalCommons@University of Nebraska - Lincoln

Computer Science, Computer Engineering, and
Bioinformatics: Dissertations, Theses, and
Student Research

Computer Science and Engineering, Department
of

7-23-2025

Investigating the Impact of Agent Openness on Planning in Multi-agent Systems

Bala Subramanyam Duggirala

University of Nebraska-Lincoln, bduggirala2@huskers.unl.edu

Follow this and additional works at: <https://digitalcommons.unl.edu/computerscidiss>



Part of the [Computer Engineering Commons](#), and the [Computer Sciences Commons](#)

Duggirala, Bala Subramanyam, "Investigating the Impact of Agent Openness on Planning in Multi-agent Systems" (2025). *Computer Science, Computer Engineering, and Bioinformatics: Dissertations, Theses, and Student Research*. 249.

<https://digitalcommons.unl.edu/computerscidiss/249>

This Thesis is brought to you for free and open access by the Computer Science and Engineering, Department of at DigitalCommons@University of Nebraska - Lincoln. It has been accepted for inclusion in Computer Science, Computer Engineering, and Bioinformatics: Dissertations, Theses, and Student Research by an authorized administrator of DigitalCommons@University of Nebraska - Lincoln.

INVESTIGATING THE IMPACT OF AGENT OPENNESS ON PLANNING IN MULTI-AGENT SYSTEMS

by

Bala Subramanyam Duggirala

A THESIS

Presented to the Faculty of

The Graduate College at the University of Nebraska

In Partial Fulfillment of Requirements

For the Degree of Master of Science

Major: Computer Science

Under the Supervision of Professor Leen-Kiat Soh

Lincoln, Nebraska

July 23, 2025

INVESTIGATING THE IMPACT OF AGENT OPENNESS ON PLANNING IN MULTI-AGENT SYSTEMS

Bala Subramanyam Duggirala, M.S.

University of Nebraska, 2025

Advisor: Leen-Kiat Soh

Multi-agent systems (MAS) possess significant potential for modeling real-world scenarios requiring coordinated actions (like wildfire fighting or ridesharing) among autonomous entities or agents (e.g., wildfire fighting agents) in complex, dynamic environments. Effective decision-theoretic planning (where each agent must carefully consider both the immediate and the future situations or states, and coordinate with the other agents (neighbors) to evaluate what needs to be done at present) within MAS, especially multiagent planning, where the planning agent directly models its neighbors in order to estimate their optimal actions, is critical, yet challenged by factors like partial observability, openness, and diverse agent types with different goals, capabilities, or both.

Current state-of-the-art frameworks for this planning come with certain drawbacks. The standard Interactive Partially Observable Markov Decision Process (I-POMDP) suffers from scalability as it becomes intractable for the planning agent to maintain an interactive *belief over beliefs* for each of its neighbors. The relatively faster I-POMDP-Lite suffers from incorrectly assuming full observability to approximately solve a nested MDP for each neighbor. Although solvers like Partially Observable Monte Carlo Planning (POMCP) enhance I-POMDP by accounting for partial observability and

approximating the resulting belief updates—planning ad hoc for each neighbor still renders real-time computation impractical for large domains. Offline solvers such as Point Based Value Iteration (PBVI) improve scalability as well as account for partial observability but have not been previously analyzed regarding agent openness.

Addressing these gaps, this thesis investigates PBVI as a scalable offline solver for partial observability settings and integrates PBVI-generated neighbor action policies into the I-POMDP-based planning, while simultaneously adopting a model-aware belief update algorithm for efficient belief updates. Our contributions include notation-level refinements and a reimplementaion of the PBVI algorithm to better handle environments with both fully and partially observable components, along with efficient belief updates, adaptable implementations, and comprehensive empirical analysis highlighting agent behavior under varying agent openness levels in environments with different agent types (cooperating and competing). These modifications make effective MAS planning computationally feasible, fostering practical applicability across complex, real-world domains.

ACKNOWLEDGEMENTS

I would like to express my deepest gratitude to Dr. Leen-Kiat Soh, whose mentorship in critical thinking has been a gift that will guide me for life. His insights and intellectual rigor have profoundly shaped the way I approach problems.

I am also sincerely thankful to Dr. Adam Eck, who taught me how to debug effectively and write efficient, maintainable code. His practical advice and clarity of thought have been invaluable throughout this research.

I would like to thank Dr. Nirnimesh Ghose and Dr. Hau Chan for their support and for serving on my committee. Their feedback and presence have been encouraging during every stage of this thesis.

Finally, I extend heartfelt thanks to my friends, family, and God for their unwavering support, love, and blessings throughout this journey.

TABLE OF CONTENTS

<u>Chapter 1 – Introduction</u>	1
1.1 The Problem	1
1.2 Motivation	5
1.3 The State of the Art and What is Missing	7
1.3.1 Planning	7
1.3.2 Agent Openness	10
1.4 Our Investigation	11
1.5 Contributions	11
1.6 Overview of the Thesis	13
<u>Chapter 2 – Background</u>	14
2.1 Multi-Agent Systems	14
2.1.1 Introduction	14
2.1.2 Cooperative and Competitive MAS	15
2.2 Partial Observability	16
2.3 Agent Openness	18
2.4 Reasoning Models	20
2.4.1 Planning	20
2.4.2 POMDP	21
2.4.3 Finitely Nested Interactive POMDP	23
2.4.4 PBVI	27
<u>Chapter 3 – Methodology</u>	33
3.1 I-POMDP with PBVI to Facilitate Planning in Large Domains	34
3.1.1 MOMDP PBVI Algorithm	38
3.1.2 Level-1 Planning with PBVI for Level-0	47
<u>Chapter 4 – Implementation Overview</u>	62
4.1 Introduction	62
4.2 Agents	64
4.3 Domains	66
4.4 Reasoning Module	68
4.5 Simulation	71
4.6 Technology Details	74
4.7 Cybersecurity Defense Domain	76
4.7.1 States	77
4.7.2 Actions	77
4.7.3 State Transition Function	79
4.7.3.1 Internal State Transition Probabilities	79
4.7.3.2 External (Security) State Transition Probabilities	80
4.7.4 Observation Function	83
4.7.5 Reward Function	84
<u>Chapter 5 – Investigations and Results</u>	85
5.1 Introduction	85
5.2 Investigations	85
5.2.1 Baselines	86
5.2.2 Agent Openness	89
5.2.3 High Observation Error	92
5.3 Cybersecurity Defense Setups for the Investigations	94
5.3.1 Setups	94

5.3.2 Domain Defaults	96
5.3.3 Reasoning Algorithms	97
5.4 Results	99
5.4.1 Baselines (No Agent Openness + Low Observation Error) Research Question 1.....	99
5.4.2 Baselines (No Agent Openness + Low Observation Error) Research Question 2.....	101
5.4.3 Baselines (No Agent Openness + Low Observation Error) Research Question 3.....	108
5.4.4 (Agent Openness + Low Observation Error) Research Question 4	110
5.4.5 (Agent Openness + Low Observation Error) Research Question 5	119
5.4.6 (Agent Openness + High Observation Error) Research Question 6	122
5.4.6.1 Under No Agent Openness	122
5.4.6.2 Under Low Agent Openness	125
5.4.6.3 Under High Agent Openness.....	126
5.4.7 (Agent Openness + High Observation Error) Research Question 7	128
5.4.8 Summary.....	130
5.4.8.1 No Agent Openness + Low Observation Error	130
5.4.8.2 Agent Openness + Low Observation Error	132
5.4.8.3 Agent Openness + High Observation Error	133
5.4.8.4 Why Level-1 Reasoning Shines Overall	135
Chapter 6 – Conclusions and Future Work.....	137
6.1 Summary	137
6.2 Future Work	138
6.2.1 Investigating Defender Performance Against Intelligent Attackers	138
6.2.2 Investigate Effectiveness of Using Information-Based Exploration in the IPOMCP Framework	139
References	141

Chapter 1 – Introduction

This chapter introduces the thesis with some background wherever necessary. We first introduce the problem dealt within this thesis, followed by what motivates us to deal with this problem. Next, we discuss what is the state of the art in this area, and what is missing from the state of the art. Given the state of the art, we then discuss the contributions of this thesis towards developing a solution. We conclude this chapter with an overview of the rest of the thesis.

1.1. The Problem

Multi-agent systems (MAS) have a great potential to help us in modeling several real-world situations where multiple autonomous entities (e.g., wildfire fighting agents) must work together – whether in environments that demand coordinated actions, computation-intensive tasks, or both. For example, the wildfire fighting agents must coordinate with each other to carefully pick a fire to fight. And modeling the other firefighting agents (neighbors) to estimate such a coordinated action could be computationally intensive. Critical to the success of such a multi-agent system to model such real-world problems is its ability to reason well about a complex, ever-changing environment to formulate optimal strategies that progress each agent towards its goals despite the difficulty of synchronizing multiple agents' behaviors within such setting.

Any MAS is made up of two or more autonomous agents who take input from their environment, decide what to do, and then carry out actions that in-turn affect the environment. To accomplish this, each agent must plan, i.e., dig deep into the future and calculate all possible outcomes (e.g., immediate next states after an action is taken, long-term ultimate outcome after a sequence of actions) of a given situation, before deciding what is presently best to do. For example, a self-driving car must start increasing its speed after looking at a hill-climb sign, to successfully make the climb, simultaneously considering the other cars on the road. Or a surgery-performing robot must predict what other robots assisting the same surgery could be doing and avoid a sequence of procedures that can lead to a fatal cardiac arrest. Such situations, complicated further by practical factors, e.g., broken camera of a surgery robot that could result in it operating on a wrong body part of a critical patient; limited operating resources, e.g., a wildfire fighting agent might run out of fire suppressant that forces it to temporarily leave the environment—resulting in fires becoming bigger; and other similar things make this planning process critical for any MAS to reach its goal, thereby making it a challenging area of research with broad applications.

Effective planning is challenged by the complex properties of the MAS made even more challenging by (a) the environment's partial observability, (b) the openness in the environment, and (c) the types of the agents. Complexities introduced by each of these factors can make the planning process computationally intensive and time-consuming as an agent is required to consider numerous possible scenarios to identify the optimal plan (or policy)—making it impractical to arrive at solutions within in a timely manner—for example, an agent cannot take days, or even hours to decide a single action during a surgery.

First, not being able to observe the environment properly or completely makes it harder for the agent to decide an action. Based on this lack of accurate and complete observability, the

environment of a MAS can be classified into either partially observable or fully observable [Kaelbling, Littman, and Cassandra 1998]. The environment can also sometimes have mixed observability where only a part of it can be fully observable—such a mixed observability setting is a simpler, more structured version of partial observability. While the agent is aware of the complete state information in a fully observable setting, it can only make ‘observations’ of the actual environment in the other two settings. These partially observable settings are seen in many real-life applications like Wildfire Fighting, CarFlag, Block-Picking, Autonomous Underwater Vehicle (AUV) Navigation [Nguyen, Hai et al. 2022] [Ong et al. 2010]. In such partially observable settings, since the planning agent is uncertain, it needs to factor in a higher number of possible states than in a fully observable system, both for estimating their current state, as well as planning into the future. This becomes a challenge, especially in domains with large state spaces.

Second, openness in a MAS environment amplifies uncertainty in an agent’s planning. It is a challenging property of a MAS and refers to the agents’ ability to deal with entities freely leaving and joining the MAS [Hattab and Chaari 2021]. Openness in a MAS environment can be divided into agent openness, task openness, and type openness [Kakarlapudi 2021]. With agent openness, the agents of the MAS are free to leave or join the system at any time (e.g., a wildfire fighting agent must leave the environment temporarily in order to refill its fire suppressant). Similarly, task openness means that the existing tasks can disappear, and new tasks can appear in the environment at any time (e.g., a driver in a ridesharing environment must deal with riders (tasks) that come and go. Type openness refers to agents changing their type, albeit briefly, anytime during the simulation (e.g., a ground-based wildfire fighting fighter could become airborne when air-support, i.e., when helicopters arrive to help the situation) [Chandrasekaran, et al. 2016].

While task openness and type openness deal with the properties of each agent in a MAS, agent openness [Chandrasekaran, et al. 2016], [Eck, et. al. 2020] [Kakarlapudi, et al. 2022] deals with a more fundamental, and hence, more critical aspect, i.e., the presence of each agent in the system. Considering agent openness in planning is critical since each agent of the MAS models the reasoning (and actions) of their peers before making their own decision. To plan effectively in such environments, each agent is faced with the challenge of reasoning not only about the action choices of each peer, but also to first predict if they are present in the environment to take that action (e.g., the presence of each agent is integrated into the state space of the environment in [Eck, et al. 2020]). Additionally, if the environment does not transition as expected (meaning the peer has not taken the expected action), the subject agent (or the planning agent) must now reason about not only the peer's action but also if the peer was present in the environment in the previous time steps to have taken any action, and must correct their belief [Chandrasekaran, et al. 2016].

Thirdly, predicting what the other agents are doing is difficult if there are different types of agents in the system, especially if the subject agent is not aware of their reasoning models. Depending on the goal of each agent in the MAS, there are three such types of settings, i.e., purely cooperative, purely competitive, and a hybrid where there are agents that mutually cooperate to work against the goal of another set of agents. The hybrid setting is also referred to in the literature as the *cooperative-competitive* setting [Wang, Jianrui, et al. 2022]. While not knowing the reasoning model of the peers already becomes a challenge in the first two settings, it becomes more critical in the hybrid setting requiring the agent to plan with utmost care. For example, the *Cybersecurity Defense* domain (described in-detail in Section 2.5) consists of two types of agents, i.e., the Attackers and the Defenders that work against each other to respectively attack and defend a cyber-network. In such a system where each agent is not aware

of the reasoning model of each other agent, careful planning is necessary to estimate not only what actions the peers (from a defender's perspective) could choose (for exemplifying relocating to a specific subnetwork to repair it), but also what the opponents are plotting (i.e., their possible targets of attack, etc.)—all while being unable to fully observe the environment (e.g., subnetworks in the Cybersecurity Defense domain) and not knowing exactly when the opponents will be present in the environment.

Finally, while each of the above challenges warrants the need for effective planning, it also simultaneously requires the planning to be efficient. Incorporating each of the above complexities increases computational time and hardware resources demands, making it difficult to scale or even implement them, often rendering them impractical to fit the real-world applications.

1.2. The Motivation

Planning in multi-agent systems finds potential in many real-world applications. While some of these applications tend to enhance the quality of our lives, some can even deal with urgent and critical situations.

Wildfires are one example of such critical applications. Approximately 70,000 wildfires per year have been documented since 1983 (as per United States Environmental Protection Agency), burning millions of acres of land area. But, even today, it remains a challenge for the human firefighters to contain such fires proactively and completely. While the inherent danger involved in fighting such fires is one reason, another is the added pressure on the firefighters to coordinate and plan under stress [Synolakis and Karagiannis 2024]. Given these factors, it is a good idea to assist or support the human firefighters with planning-based dispensable robots that are intelligent enough to plan for and contain such wildfires. Such dispensable robots,

which can work in dangerous, high-risk environments, are already being implemented in the form of multi-agent Unmanned Aerial Vehicle (UAV) teams that assist in containing wildfires through coordinated planning under uncertainty [Haksar and Schwager 2018] [Seraj and Gombolay 2020].

Examples of other critical situations include military defense [Aberdeen et al. 2004], cybersecurity defense [Eck, Unpublished], disaster management [Omidshafiei, Shayegan, et al. 2017], and so on. Like firefighting, even these situations tend to be solved to only some extent using human intellect and physical capabilities. A more complex and thorough solution often requires us to plan deeper into the future. For example, it can be difficult for a team of search-and-rescue operators to come up with the most likely locations of the stranded victims after an earthquake, when provided with limited information, such as the event’s epicenter and magnitude. Furthermore, it is another challenge to narrow-down a singular target to rescue, since they might not know if any of their peers are planning, or already efficiently dealing with the same target. Even though these rescue agents could communicate, such communication might not be immediate, reliable, or rich enough to support coordinated reasoning under time pressure and uncertainty. Such are the tasks that could be more feasible when we use artificial intelligence-based approaches, like planning in a multi-agent system.

Given the above, it is important not only to design such multi-agent systems to model the real-world challenges but also to integrate planning-based methods and make them feasible to test and implement. To fit several problems of a similar category and plan more efficiently, a generalized multi-agent system-based planning framework that can adapt to varying scenarios, coordinate multiple agents, and generalize across different environments can require a larger number of agents, states, actions, or all of them. For example, consider the above-mentioned search-and-rescue team operating in a large metropolitan city. Here, numerous operators could

be required if the impacted area is large. Or each operator must choose from multiple important locations, tasks, or both. While this added flexibility can make the system more effective, it also necessitates more calculation for the decision-making process. Given the exponential nature of these calculations where a small change in the number of agents, states, or actions can drastically increase the joint action space and the dimensionality of the belief space, the problem can easily become prohibitively large to solve. Therefore, we must make sure that our solutions are feasible to implement, can be viably used for empirical investigations, and eventually tailored for deployment and further scaling.

Planning in multi-agent systems is an open research area due to the various problems mentioned in Section 1.1. All these problems are motivated by the real-world situations. Hence, solving or making progress with each of these problems brings us closer to adapting these systems for more practical real-world applications.

1.3. The State of the Art and What is Missing

1.3.1. Planning

For planning in partially observable multi-agent systems, the I-POMDP [Gmytrasiewicz and Doshi 2004], which extends the well-known POMDP framework [Kaelbling, Littman, and Cassandra 1998] to multi-agent systems, is considered a robust [Schwartz, et al. 2022] framework in the literature; each agent models each other agent in the system to make coordinated decisions. The I-POMDP-Lite [Hoang and Low 2013], which approximates I-POMDP by replacing its recursive belief modeling and intractable real-time belief updates with Nested-MDP based reasoning that treats lower level modeled neighbors (i.e., the internal models maintained by the planning agent, not the actual agents themselves) to be following a fixed, fully observable policy, is considered one of the fastest solvers for this framework [Schwartz, et al. 2022].

However, these state-of-the-art approaches come with a few drawbacks that are discussed below.

Although I-POMDP-Lite provides a more tractable solution method for the I-POMDP framework in terms of solving-speed, it comes with a limitation in terms of assuming that each of the planning agent’s modeled neighbors can fully observe the environment to predict their mixed strategy using a nested-MDP. Here, “nested” refers to the fact that each modeled neighbor’s reasoning is embedded within the planning agent’s own decision-making process, and the MDP refers to a standard fully observable Markov Decision Process used to approximate the neighbor’s behavior. This modeling choice makes it harder for us to deeply analyze the impacts of agent openness. Since each modeled neighbor computes their mixed strategy assuming that it can fully observe the environment, including its own openness model, this reasoning does not fit well into the real-time simulation where each agent can only partially observe if it is inside or outside the environment. For the same reason, while the I-POMDP-Lite solver can readily use these pre-computed nested-MDP policies and make the planning faster, we have a chance for inefficiency due to its assumed full-observability.

Even with a faster online level-0 solver called the POMCP [Silver, and Veness, 2010] that can solve the neighbors’ models assuming *partial* observability, feasibility with larger domains remains a challenge. Here, online solver refers to one that computes the best action at runtime, using the current belief of the agent and a given horizon for the lookahead, instead of precomputing the full policy or value function. Note that the nested MDPs used in I-POMDP-Lite are examples of offline solvers. Such an offline solver precomputes the policy before execution, enabling faster action selection during runtime. The online POMCP, however, solves the POMDP framework and uses Monte-Carlo sampling for the look-ahead tree search (MCTS) and the belief updates of the agent. This can be incorporated into the online level-1 planning to solve the

neighbors' models. But even with MCTS facilitating a faster approach, it is still time-consuming to plan online for each neighbor of the level-1 subject agent because of the inherent nested reasoning. Since each level-1 agent maintains a belief over beliefs of each of its modeled level-0 neighbors, this quickly becomes intractable with increasing number of states where the level-1 agent must solve an online POMDP in real-time for each such nested belief of the neighbor. Furthermore, the existing techniques used to aid in the lower-level neighbor model's belief propagation in such solvers, for example, *rejection sampling*, have drawbacks in the form of slowness and particle deprivation [Silver, and Veness, 2010].

Given the above, a natural solution approach to solve the level-1 I-POMDP framework would be to use neighbor policies that 1) can be computed offline for speeding up the planning, and 2) assume partial observability for better accuracy. While the classic POMDP value iteration algorithms [Pineau, Gordon, and Thrun 2003] generate such partially observable offline policies, they can still become intractable in large partially observable domains due to the two well-known factors, the curse of dimensionality, and the curse of history.

Point-Based Value Iteration (PBVI) [Pineau, Gordon, and Thrun 2003], on the other hand, which not only assumes partial observability but also implements selective belief exploration, works as a much tractable offline solver that can be integrated into the higher levels of the *I-POMDP* planning, making it a viable solution in this area. However, there has been no previous research that analyzes agent openness in an I-POMDP-based algorithm where a partially observable solver like PBVI drives the lower-level neighbor reasoning in a cooperative and competitive setting.

1.3.2. Agent Openness

[Chandrasekaran, et al. 2016] present the first approach to demonstrate robust decision-theoretic planning in domains that consider agent openness. Subsequently, [Eck, et. al. 2020] present a scalable solution of the same that can work with many agents. Communication (not our focus for this thesis) has also been introduced into such solvers [Kakralapudi, et. al. 2022] to enable the agents to reason better. However, given the completely online-based planning nature of these approaches, they are not readily scalable and hence have not been tested in large domains with partial observability, particularly in the models where even the lower-level neighbor reasoning is also based on partial observability.

Although offline solvers like PBVI can be incorporated into the planning to alleviate this problem, such approach has not been tested so far in the literature to model complex MAS like the ones in our scope. This is important because we are interested to see if PBVI, as a level-0 solver that does not explicitly model its neighbors, will still be able to come up with a reasonable policy when there is agent openness involved in a partially observable environment. Additionally, belief updates in a partially observable environment, especially for the lower-level neighbor reasoning, become much harder due to agent openness. This is because an agent, when being outside the environment, cannot observe the state of the environment and can only receive a “no observation” (or NOOB) for that time-step, making it harder to transition to the subsequent belief.

Additionally, the challenges related to planning, openness, and other problems mentioned in Section 1.1., like partial observability, cooperative-competitive agents, and so on, have previously not been applied to multi-agent environments, all at the same time.

1.4. Our Investigation

The goal of this thesis is to analyze the planning algorithms that help agents plan in a complex multi-agent setting designed to emulate the problems mentioned in Section 1.1, with an emphasis on analyzing agent openness.

Firstly, we investigate PBVI as an alternative to the intractable POMCP-based online planning that doesn't scale well for large domains with partial observability. We then apply the PBVI algorithm to generate offline policies in an open-agent, partially observable environment. These offline policies also serve as standalone approaches to solve the level-0 POMDP problem that can be compared against the more complex I-POMDP approach.

Secondly, we incorporate the pre-computed offline PBVI policies into the higher-level I-POMDP-based planning to make it faster and check if this in fact is feasible for a relatively large domain like Cybersecurity Defense [Eck, Unpublished]. In the process, to handle the problem of lower-level belief propagation in partially observable environments, we describe and implement a belief-update algorithm (called the *Model-Aware Monte Carlo Belief Update Algorithm*) that uses cached transition probabilities for a faster belief update.

Finally, we analyze and present our findings of the cooperative-competitive multi-agent system's behavior, with a focus on how each agent behaves under different levels of agent openness.

1.5. Contributions

The contributions of this thesis can be divided into three aspects: (1) re-implementing algorithms, (2) code implementations, and (3) empirical findings.

Regarding the algorithms, we adapt and re-implement the PBVI algorithm to handle mixed observability in domains like Cybersecurity Defense [Eck, Unpublished] that have mixed observability. While this does not alter PBVI’s core functionality, it enables more efficient and faster planning by separating the fully observable components of the state space from the partially observable ones. Second, we modify the existing I-POMCP algorithms to accommodate PBVI and efficiently handle the lower-level belief updates.

Next, we implement the above algorithms such that the codebase is generic and hence easily adaptable to any relevant domain. By using Cython (composed of Python + C) as the programming language, we make sure that the code is both easily implementable because of Python, and efficient because of C, so that it works well even with large domains. Furthermore, we enable the codebase to use GPU-based hardware acceleration to make the I-POMCP-PF algorithm feasible to run thousands of planning rollouts or trajectories—that is, simulated action-observation sequences used to estimate the value of various possible actions for a given belief (specifically where the level-1 I-POMCP-PF agent must choose an argmax action for its level-0 PBVI neighbor model by computing a dot product between the neighbor’s belief vector and the entire solution set of the pre-computed PBVI alpha-vectors) .

Finally, since evaluating the behavior of each agent of a multi-agent system, especially in such a complex environment, is non-trivial (given numerous existing naïve, and complex baselines that can be applied to such settings), our empirical analysis of the same can be considered a contribution to such open-agent environments where all the discussed challenges, together, have not been evaluated previously. In this part of our contribution, we also put an additional emphasis on analyzing the impact of agent openness by comparing and interpreting results from different experiment setups with varied levels of agent openness involved. In a nutshell, we conduct several cases studies to model MAS handling various real-world challenges.

1.6. Overview of the Thesis

Chapter 2 discusses the background and related work. Chapter 3 presents our methodology.

Chapter 4 describes details of our implementation. Chapter 5 discusses the results. Chapter 6 concludes and posits future work.

Chapter 2 – Background

In this chapter, we discuss the necessary background and related work to this thesis. We first introduce what multi-agent systems are, in Section 2.1—including different types of agents with different goals or capabilities. Then in Section 2.2, we discuss partial observability, an important characteristic of any multi-agent system. Section 2.3 gives a background on agent openness. We will then discuss multi-level planning in multi-agent systems that helps each agent take autonomous decisions, and give a background on relevant frameworks, in Section 2.4. In this section, we specifically present the Partially Observable Markov Decision Process (POMDP), the core framework for planning under uncertainty. We then describe the Finitely Nested Interactive POMDP (I-POMDP), which extends the POMDP framework by allowing agents to reason about each other’s beliefs and actions. Furthermore, we also discuss in the same section a technique called Point-Based Value Iteration (PBVI), and how can it help improve the existing planning frameworks in terms of scalability.

2.1. Multi-Agent Systems

2.1.1. Introduction

A Multi-Agent System (MAS) is a type of Distributed Artificial Intelligence (DAI) that has garnered importance in academia because of its potential to solve complex computing problems [Balaji and Srinivasan 2010] [Dorri, Kanhere, and Jurdak, 2018] [Chandrasekaran et al. 2016] that

model real-world scenarios such as wildfire fighting, disaster response, autonomous ridesharing, and cybersecurity defense. Other components of DAI include Parallel AI, which improves the efficiency of classical AI methods through task parallelism across distributed hardware, and Distributed Problem Solving (DPS), which divides a problem among cooperative computing entities with predefined communication structures [Dorri, Kanhere, and Jurdak, 2018].

Multi-Agent Systems extend DPS by introducing autonomous entities called agents that serve as the computing entities [Dorri, Kanhere, and Jurdak, 2018]. MAS are more flexible than DPS since agents are capable of learning and making autonomous decisions, while interacting with other agents (i.e., neighbors) and the environment. Then they solve their tasks by performing actions that impact the environment.

2.1.2. Cooperative and Competitive Agent Types

The nature of interactions between different types of agents of a MAS can be cooperative, competitive [Wang, Jianrui, et al. 2022], or both. This cooperative or competitive nature of interactions is influenced by several factors such as cost functions or the rewards which motivate each agent to accomplish a certain task, information regarding the neighbors, and the environment [Wang, Jianrui, et al. 2022].

Cooperative settings are where each agent cooperates with each other agent to optimize towards a global goal [Wang, Jianrui, et al. 2022]. Wildfire Fighting [Chandrasekaran, et al. 2016] is a good example to describe a cooperative MAS where each wildfire fighting agent must coordinate and cooperate with each other agent in the system to extinguish as many fires as possible.

Competitive settings, on the other hand, are where competitions exist between each agent. The Attacker or the Defender agents of the Cybersecurity Defense domain [Eck,

Unpublished] serve as a good example to demonstrate a competitive setting. In fact, this MAS as a whole is a setting where a set of agents (attackers or defenders), while cooperating within themselves, compete with another set of agents. Such settings can be termed as cooperative-competitive settings, i.e., a mix of both, also discussed in [Zhang, et al. 2021].

In either of the above settings, each planning agent may or may not be aware of the reasoning models (i.e., the decision-making models) of each neighbor. Not knowing a neighbor's reasoning model means that the planning agent must assume either its own or some other reasoning to model the actions of each neighbor – without explicitly accessing the neighbor's planning process or belief updates. In such cases, an inaccurate reasoning model of its neighbor could lead the agent into believing that its neighbor is doing an action that is different from what the neighbor actually does. This, in turn, could lead to further inaccuracy in the belief estimates which drive the subsequent choices of actions, ultimately making it challenging for each agent to either cooperate or compete effectively. While observing the environment from time to time can limit the impact of this challenge to a certain extent by helping the agents condition their beliefs, this is still a major issue in open agent settings where an agent can intermittently be out of the environment and cannot make any observations at all during that period.

2.2. Partial Observability

Partial observability in a MAS refers to a setting where each agent can *only* partially observe or sense the environment [Kaelbling, Littman, and Cassandra, 1998].

In the well-known fully observable settings, each agent of the MAS can fully observe the current state of the environment. This helps the agent to precisely plan for the future possible states which result from the agent's actions.

Such fully observable settings, however, are not the only possibility in the real world. For example, a cleaning robot might not be able to clearly see what is ahead, due to a faulty camera. Or a wildfire fighting agent might only have a rough idea of its own location in a forest due to lack of a GPS signal. Such settings can be classified as the partially observable settings [Kaelbling, Littman, and Cassandra, 1998]. In these settings, since the agent cannot fully observe the state of the environment, it can only make an observation of the state with some error probability. In other words, the agent is only able to observe the true state with a certain probability, and with the remaining probability, it observes a different state due to noise or sensor error. And since each agent is not fully aware of their current state, it can only maintain a belief of what their actual state is [Kaelbling, Littman, and Cassandra, 1998]. This belief is a probability distribution over the entire state space where the probability of each state is from the point of view of the agent.

Partial observability complicates planning in MAS because the agent cannot rely on a known state to determine the best action; instead, it must reason over an evolving belief, i.e., a distribution of many possible states, making the planning process significantly more computationally expensive and uncertain. In some cases, there can also be settings where there is a mixed observability, and each agent can fully sense or observe only a part of the environment, such as in the Cybersecurity Defense domain [Eck, Unpublished]. This is a simpler case of the partially observable settings where the environment states are factored into fully and partially observable components. Such structure can be leveraged to maintain beliefs only over the partially observable parts, thereby reducing the computational overhead during planning [Ong et al. 2010].

2.3. Agent Openness

Agent openness is a type of openness in a MAS. As the name refers to, the openness is a characteristic of the system where certain attributes are subject to change with time. Therefore, agent openness refers to agents of a MAS freely leaving or joining the environment at any time [Kakarlapudi 2021]. For simplicity, and because POMDP is natively constrained in modeling dynamic sets of agents, the total number of agents remains constant within the scope of this thesis, and any entries or exits made by each agent are temporary. The other types of openness are task openness and type openness [Kakarlapudi 2021].

Agent openness can be seen in many real-world situations. For instance, a wildfire fighter might run out of suppressant and temporarily leave the system to recharge [Kakarlapudi, et al. 2022]. Or a fighter jet might need to temporarily return to base to refuel. Or a ridesharing driver might take a break for lunch or a vacation. In such cases, such agents are out of the environment in terms of them not making any decisions or reasoning during their absence with regards to any tasks or activities that might directly or indirectly interact with the environment or with other agents in the system.

Agent openness makes the planning process of each agent more complex from the agent's individual perspective. To plan effectively, the agent must account for its own openness model. For example, a wildfire fighting agent must account for the fact that they might run out of suppressant at any moment and would need to go outside of the environment to recharge. Furthermore, when outside, it is sometimes not possible to observe the environment at all. The agent must now update its belief of the environment's state without making any observation and instead depending on just the modeled neighbors' actions.

In addition to an agent's individual planning, agent openness also has a significant effect when planning for each neighbor. Depending on if the neighbor is entering or leaving the system, it is either suddenly available, or not available to work with their peers, respectively. This is usually not a problem in a fully observable environment since each agent can fully observe each other agent entering or leaving the system. But to model each neighbor and plan efficiently in a *partially* observable environment, an agent, in addition to modelling its own openness, must now predict if each of its neighbors is available in the environment to take an action. Furthermore, an agent must also make an additional correction to its belief after the time-steps in which a neighbor unexpectedly leaves or re-enters the environment. For example, suppose an agent updates its belief based on the modeled action of a neighbor that is assumed to be inside the environment. But if this neighbor was outside the environment and therefore did not take any action, this results in the state-update in a manner that the planning agent was not predicting. Therefore, the planning agent must update or correct its belief based on the incoming observation of the actual environment in the subsequent time steps.

The degree of agent openness, especially a higher degree, further influences the aforementioned individual planning and neighbor modelling. More openness makes the planning more uncertain and can even lower the effectiveness of the planning to a point where naïve or even random action choices are equally effective as planning intensely. For example, consider agent openness in the wildfire suppression setting where each agent only has a small amount of suppressant that can run out at any time. This means that each neighbor might leave the environment more frequently, and hence the planning agent cannot rely on any of its neighbors to stay and fight the same fire as it does – and not knowing when the neighbor will return, it could therefore end up choosing a different and less important fire to fight or abandoning its attempts altogether to save the valuable suppressant. Carefully modelling the

neighbor and predicting their actions might go to waste if the neighbor is out of the environment too frequently.

2.4. Reasoning Models

2.4.1. Planning

Planning, in general, refers to finding an optimal way to act in the environment to accomplish tasks, given a complete and correct model of the dynamics of the world and the structure of rewards [Kaelbling, Littman, and Cassandra 1998]. Many real-world applications, like wildfire suppression, cybersecurity-defense, search-and-rescue operations, and so on, are essentially examples of planning problems. Here, an agent must decide what is best for them to do by not only considering the current state but also the future possible states.

Planning, unlike in deterministic modelling, is non-trivial in stochastic modeling – in MDPs due to uncertainty in state transitions, and in POMDPs due to uncertainty in both state transitions and observations [Kaelbling, Littman, and Cassandra 1998]. Stochastic modeling refers to uncertainty involved in the change of one environment state to another. This change could either be a result of an agent’s actions or can independently evolve with time. In the context of planning, it now means that we cannot naively plan by using just the sequences of actions, since the outcomes of actions are not guaranteed, or because the outcomes of the executed actions are not deterministic. Instead, we need to consider plans to be mappings from states (in MDPs) or beliefs (in POMDPs) to actions, thereby accounting for any future possible uncertainties [Kaelbling, Littman, and Cassandra, 1998].

Such planning problems where we need to choose an optimal action in stochastic and partially observable environments can be modelled as Partially Observable Markov Decision Processes (POMDPs) [Kaelbling, Littman, and Cassandra, 1998].

2.4.2. POMDP

A Partially Observable Markov Decision Process (POMDP) [Kaelbling, Littman, and Cassandra, 1998] [Kakralapudi 2021] is a generalization of the Markov Decision Process (MDP) [Puterman, M. L., 1990] for the partially observable environments. It is described using the tuple $\langle S, A, T, R, \Omega, O, \gamma \rangle$, where:

- S, A, T , and R , are the state space (can be further factored using the variables $F_1, F_2, \dots, F_k$ to represent, for example, k health levels of each subnetwork of the Cybersecurity Defense domain), the action space which is the set of actions each agent can perform, the transition function ($T : S \times A \times S' \rightarrow [0, 1]$) which defines the change from the current state to the next state of the environment after taking an action, and the reward function [$R : S \times A \rightarrow \mathbb{R}$], that defines the rewards for the state-action combinations, respectively. These parameters represent the Markov Decision Process.
- O is defined as the finite set of observations that each agent can experience from its environment.
- $\Omega: S \times A \times O \rightarrow [0, 1]$ represents the observation function. This function gives the probability distribution of possible observations, given an action and the resulting next state. The probability is given by $\Omega(s', a, o) = P(o|a, s')$.
- $\gamma \in (0, 1]$ is a discount factor that adjusts the weight of the future rewards.

In the partially observable environment, since each agent cannot exactly know their current state, it maintains a belief $b \in B$ over all possible current states and their probabilities, where $b: S \rightarrow [0,1]$ is a probability distribution over all possible states from S , and B is the belief space (i.e., the set of all such distributions). The agent chooses an action in each time-step using this belief and eventually updates this belief using the Bayes' rule as follows [Kakralapudi 2021].

$$b(s') = \Pr(s'|o, s, a) = \alpha \Omega(o, s', a) \sum_{s \in S} b(s) T(s, a, s') \quad (\text{Equation 2.1})$$

where α is a normalizing constant. In a multi-agent system, where a POMDP-based agent operates without modeling their neighbors, the actions of other agents are usually marginalized as “Configurations” into the state and observation spaces—effectively treating them as part of the environment’s stochastic dynamics. $\mathcal{C}$ is usually used to denote the set of all possible configurations C from the point of view of a level-0 agent.

The policy, or the solution, of a POMDP problem is defined as the best strategy for acting under uncertainty [Kakrapudi 2021], and therefore maps the current belief state to a distribution over actions, $\pi : B \times A \rightarrow [0, 1]$ where $\pi(b, a)$ gives the probability of taking action a in belief state b .

We note that the above formulation assumes a finite-horizon setting, where the agent plans over a limited number of future steps, denoted by the planning horizon h [Smallwood and Sondik 1973]. This is a standard assumption in many practical POMDP solvers, where the value function depends on the remaining number of steps in the planning process and is written as $V^h(b)$ to reflect this planning depth.

A value function $V^h(b)$ is then defined to help determine the policy of the POMDP. It gives the long-term expected reward as the utility of taking an action a with a belief b , and is given using the Bellman equation as follows [Kakrapudi 2021].

$$V^h(b) = \max_{a \in A} \sum_{s \in S} b(s) (R(s, a, C) + \gamma \sum_{s' \in S} T(s, a, C, s') P(C) \sum_{o \in \mathcal{O}} \Omega(o, a, C, s') V^{h-1}(b(s'))) \quad (\text{Equation 2.2})$$

where $b(s')$ is the belief update that uses Equation 2.1, and h is the planning horizon that decides how deep the agent plans into the future before deciding on an action for the present.

In other words, $V^h(b)$ is the policy, which computes the best action to take for a given belief b , while also looking into the future recursively through $V^{h-1}(b')$, where $b'(s')$ is the updated belief for each next state s' after taking an action a and receiving observation o , and h is the planning horizon that determines how deep into the future the agent plans.

2.4.3. Finitely Nested Interactive POMDP

Finitely Nested Interactive POMDP (Finitely Nested I-POMDP) [Gmytrasiewicz and Doshi, 2005] is an extension of the more fundamental yet intractable framework, i.e., Interactive POMDP (or I-POMDP) which models multi-agent interactions by infinitely nesting the planning agent's beliefs over other agents' beliefs and strategies.

An I-POMDP is an extension of the POMDPs to multi-agent settings [Gmytrasiewicz and Doshi, 2005] and is mainly characterized by the addition of each neighbor's beliefs and intended actions into the state space of the planning agent, in order to enable it to reason about its neighbors during planning, unlike standard POMDPs where other agents are treated as part of the environment, thereby allowing for more effective decision-making in interactive MAS. In other words, the planning agent now also maintains a *nested* belief over all its neighbors and propagates this nested belief by estimating the mixed strategy of all the agents, i.e., the probability distributions over actions chosen by itself and each of its neighbors, given their respective beliefs and models. However, this nesting becomes quickly intractable, as each agent recursively models its neighbors who model it in return, resulting in an infinite nesting of the beliefs that renders exact planning computationally infeasible.

To address this, the same authors propose the Finitely Nested I-POMDPs [Gmytrasiewicz and Doshi, 2005] where each agent assumes that its neighbors reason at a fixed level below its own. For example, a "level-0" agent doesn't directly model its neighbors and only solves a

POMDP. A “level-1” agent, on the other hand, models their neighbors to be level-0, and so on. This bounded nesting makes planning tractable without losing the essence of strategic multi-agent reasoning.

The formalism of a Finitely Nested I-POMDP [Gmytrasiewicz and Doshi, 2005] [Kakrappudi 2021] is described using the tuple $\langle IS_i, A_i, T_i, R_i, \Omega_i, O_i, \gamma \rangle$, where:

- Agent i is the subject (or planning) agent, and the set of agents $Ag/\{i\}$ refers to the neighbors of agent i , i.e., the other agents whose models are explicitly included in agent i 's interactive state.
- $IS_{i,l}$ is a set of interactive states (*is*) of level l for the subject agent i , and are given by $IS_{i,l} = S \times \theta_{j,l-1} \times \theta_{k,l-1} \times \dots \times \theta_{z,l-1}$, where $l > 0$. Here, S represents the state space. Each neighbor $j, k, \dots, z$ is ascribed a model, e.g., $\theta_{j,l-1}$, from the set of computable models given by $\theta_{j,l-1} = \langle b_{j,l-1}, \hat{\theta}_j \rangle$ where $b_{j,l-1}$ is the subject agent's level $l - 1$ interactive belief over agent j , and $\hat{\theta}_j$ is agent j 's frame (i.e., a representation of agent's capabilities and preferences [Kakrappudi 2021]). For $l = 0$, $IS_{i,0} = S$.
 - In simple terms, this means that agent i maintains a belief about what agent j knows and how it is likely to behave, based on its capabilities and objectives. For example, in the *Cybersecurity Defense* domain, agent i (a defender) may model agent j (an attacker) as believing certain subnetworks are more exploited, and as targeting only such subnetworks (i.e., the more exploited ones).
- $A = A_i \times A_j \times \dots \times A_z$ is the joint set of actions of the agents. For example, the individual subnetworks each defender of the *Cybersecurity Defense* domain chooses to defend. For notational simplicity, we let $a_{-i} \in A_j \times \dots \times A_z$ denote the set of actions of the neighbors of agent i .

- O_i is the set of possible observations of agent i .
- $T_i(s, a_i, a_{-i}, s') = P(s'|s, a_i, a_{-i})$ gives the probabilities of the state transitions caused by the agent i and all its neighbors $Ag/\{i\}$.
- $\Omega_i(s', a_i, a_{-i}, o_i) = P(o_i|a_i, a_{-i}, s')$ gives the probability of agent i observing o_i , after the joint action (a_i, a_{-i}) taken in state s .
- $R_i(s, a_i, a_{-i}) \in \mathbb{R}$ is the reward function of agent i that gives R_i as the reward after executing (a_i, a_{-i}) in state s .
- $\gamma \in (0, 1]$ is a discount factor that adjusts the weight of the future rewards.

The belief update for a Finitely Nested I-POMDP [Kakrapudi 2021] can be given as

$$b_{i,l}^t = \Pr (IS_{i,l} | b_{i,l}^{t-1}, a_i^{t-1}, a_{-i}^{t-1}, o_i^t) \quad (\text{Equation 2.3})$$

Equation 2.3 defines the belief update for a Finitely Nested I-POMDP at time t , denoted by $b_{i,l}^t$, as the probability distribution over interactive states $IS_{i,l}$ conditioned on the agent's previous belief $b_{i,l}^{t-1}$, its previous action a_i^{t-1} , previous modeled neighbors' actions a_{-i}^{t-1} , and the latest observation o_i^t . In simple terms, it describes how agent i , at nesting level l , revises its belief over the interactive state space by incorporating the new information received at time t . This update also accounts for the mixed strategy (i.e., joint actions of the agent and its neighbors) and the belief updates of each modeled neighbor whose beliefs are themselves nested within this belief.

$$\begin{aligned}
b_{i,l}^t(is^t) = & \alpha \sum_{is^{t-1} \in IS} b_{i,l}(is^{t-1}) \times \prod_{\substack{j \in Ag \\ \{i\}}} \left(\sum_{a_j^{t-1}} \Pr(a_j^{t-1} | \theta_{j,l-1}^{t-1}) \right) T_i(s^{t-1}, a_i^{t-1}, a_{-i}^{t-1}, s^t) \\
& \times \Omega_i(s^t, a_i^{t-1}, a_{-i}^{t-1}, o_i^t) \prod_{\substack{j \in Ag \\ \{i\}}} \left(\sum_{o_j^t} \tau_{\hat{\theta}_j}(b_{j,l-1}^{t-1}, a_j^{t-1}, o_j^t, b_{j,l-1}^t) \right) \\
& \times \Omega_j(s^t, a_j^{t-1}, a_{-j}^{t-1}, o_j^t) \quad (\text{Equation 2.4})
\end{aligned}$$

Equation 2.4 expands the belief update from Equation 2.3 by explicitly showing how the new belief over each interactive state is^t is computed. It marginalizes over all possible previous interactive states is^{t-1} , neighbors' actions a_j^{t-1} , and neighbors' observations o_j^t . The updated belief $b_{i,l}^t(is^t)$ is weighted using the previous belief $b_{i,l}^{t-1}(is^{t-1})$, the probability of the joint action of the neighbors $\Pr(a_j^{t-1} | \theta_{j,l-1}^{t-1})$, the transition function $T_i(s^{t-1}, a_i^{t-1}, a_{-i}^{t-1}, s^t)$, the observation function $\Omega_i(s^t, a_i^{t-1}, a_{-i}^{t-1}, o_i^t)$, and the belief updates of each neighbor via $\tau_{\hat{\theta}_j}(b_{j,l-1}^{t-1}, a_j^{t-1}, o_j^t, b_{j,l-1}^t)$. In short, it formalizes how agent i updates its belief by considering both the environment's dynamics and its recursive models of each neighbor.

Given this, the planning agent assigns each level l belief a value that is the sum of expected rewards discounted over a given horizon H , $r_0 + \gamma r_1 + \gamma^2 r_2 + \dots + \gamma^{H-1} r_{H-1}$. We maximize over the Bellman equation for each belief-action pair as follows [Kakralapudi 2021].

$$Q_i^t(b_{i,l}^t, a_i^t) = p_i(b_{i,l}^t, a_i^t) + \gamma \sum_{o_i^{t+1}} \Pr(o_i^{t+1} | b_{i,l}^t, a_i^t) V_i^{t+1}(b_{i,l}^{t+1}) \quad (\text{Equation 2.5})$$

$$V_i^t(b_{i,l}^t) = \max_{a_i \in A_i} Q_i^t(b_{i,l}^t, a_i^t) \quad (\text{Equation 2.6})$$

where:

$$p_i(b_{i,l}^t, a_i^t) = \sum_{is^t \in IS_l^t} b_{i,l-1}^t(is^t) \sum_{a_{-i} \in A_{-i}} \prod_{j \in Ag} \Pr(a_j^t | \theta_{j,l-1}^t) R_i(s, a_i, a_{-i}) \quad (\text{Equation 2.6}),$$

and $b_{i,l}^{t+1}$ is the updated belief after performing action a_i^t and receiving observation o_i^{t+1} .

Given the above, the policy is $\pi_{i,l}$ given as the distribution of the action values that maximize the Q-value [Kakralapudi 2021] at a given belief.

$$OPT(b_{i,l}^t) = \operatorname{argmax}_{a_i} Q_i^t(b_{i,l}^t, a_i) \quad (\text{Equation 2.7})$$

and $\pi_{i,l}(a_i^*) = \frac{1}{|OPT|}$ where OPT is the series of optimal actions from the given belief.

2.4.4. PBVI

Value iteration [Pashenkova, Rish, and Detchter 1996] *“is an iterative procedure that calculates the expected utility of each state using the utilities of the neighboring states until the utilities calculated on two steps are close enough”* and helps us towards computing an optimal policy for each state in fully observable settings like MDPs. In partially observable domains like POMDPs, however, exact value iteration is intractable, and approximate methods such as Point-Based Value Iteration are used instead. PBVI [Pineau, Gordon, and Thrun 2003] refers to an approximate offline POMDP planning algorithm. It addresses the scalability issue that plagues the value iteration algorithms. Although PBVI does not guarantee globally optimal policies, it provides bounded suboptimality that depends on how well the explored and chosen belief points cover the belief space. Additionally, since this solves a POMDP, it can be used in a higher-level planner, i.e., I-POMDP, to provide level-0 policies for each neighbor, which are then integrated into the I-POMDP agent’s reasoning and belief updates during planning.

Scalability is a major issue with the value iteration algorithms of the POMDPs for two main reasons [Pineau, Gordon, and Thrun 2003]. First is the curse of dimensionality where in a problem with n physical states, POMDP planners must consider an $(n-1)$ dimensional continuous space to reason about their beliefs. Second is the curse of history where the number of distinct action-observation histories that lead to various reachable beliefs grows exponentially as we plan more in-depth for the future (i.e., by increasing the planning horizon).

In general, both the above-mentioned curses are related [Pineau, Gordon, and Thrun 2003]. Increasing the dimensionality of the belief space also increases the room for distinct histories. However, they can also act independently in some cases [Pineau, Gordon, and Thrun 2003]. That is, the curse of dimensionality can increase exponentially with the horizon even in a problem with only a few states, and on the other hand, problems with large state spaces may not have many action-observation histories. In most domains, POMDP value iteration is primarily affected by the curse of history than the curse of dimensionality [Pineau, Gordon, and Thrun 2003]. The curse of history can hence be a better predictor of the overall run-time [Pineau, Gordon, and Thrun 2003].

Informed by the above observation, PBVI claims that many real-world POMDP problems, where, in-general, the number of states is not a problem, *can be solved easily if we can avoid the curse of history*. The curse of history refers to maintaining the action-observation histories that lead us to a certain point or time-step in the planning process. The number of such action-observation histories grows exponentially as we go further down the horizon thereby making it harder to store these histories and resulting in a computational bottleneck. PBVI addresses the curse of history by 1) maintaining a small set of reachable beliefs that summarize and replace storing the raw action-observation histories; it does so by readily employing various belief exploration strategies to efficiently reduce the number of required beliefs for a good solution

that is also guaranteed to be bounded [Pineau, Gordon, and Thrun 2003], and 2) focusing planning only on this smaller belief set which results in faster value updates; PBVI is therefore significantly more efficient (quadratic) than an update which considers the complete belief space (exponential) [Pineau, Gordon, and Thrun 2003]. Furthermore, since PBVI a level-0 offline planner which does not directly model the other agents, the policies generated by PBVI can also be integrated into the higher-level planning models of the I-POMDP solvers.

As the name suggests, the point-based value iteration (PBVI) algorithm solves a POMDP for a finite set of belief points $B = \{b_0, b_1, \dots, b_n\}$. PBVI maintains alpha-vectors to update the value function of the underlying POMDP. An alpha-vector, generally denoted by $\alpha = [V(s_1), V(s_2), \dots, V(s_{|S|})]$, is a vector of expected utility of each state $s \in S$. Such alpha-vectors are chosen with respect to each action and observation outcome and selected to maximize expected value at each belief. Therefore, at a given iteration, the set of alpha-vectors represents a piecewise-linear and convex approximation of the value function over the explored beliefs [Pineau, Gordon, and Thrun 2003]. The value function of a POMDP at the n^{th} horizon can be updated using the alpha-vectors of the previous solution V_{n-1} using the Backup operator H [Pineau, Gordon, and Thrun 2003]. In other words, PBVI applies H to each belief in B , computing a new alpha-vector (using a dot product) that maximizes the expected value over all actions and possible observations from that belief.

An exact value backup, denoted by $V = HV'$, is given by [Pineau, Gordon, and Thrun 2003]:

$$V(b) = \max_{a \in A} [\sum_{s \in S} R(s, a) b(s) + \gamma \sum_{o \in O} \max_{a' \in V'} \sum_{s' \in S} T(s, a, s') \Omega(o, s', a) \alpha'(s') b(s)] \quad (\text{Equation 2.8})$$

where A , Ω , S , R , and b , are the attributes of a POMDP, and α is the alpha-vector.

[Pineau, Gordon, and Thrun 2003] modify the above for PBVI such that only one Alpha-vector is maintained for each belief point. Given this, PBVI still preserves the piece-wise linearity and convexity of the POMDP's value function defining it over the entire belief simplex. Furthermore, PBVI chooses this alpha-vector from a set of candidates (using a dot-product) such that it is the most optimal for a given belief point and discards the rest for computational efficiency.

To implement this point-based $V = HV'$, we first create the projections for each action, and observation, as follows:

$$\Gamma^{A,*} = \alpha^{a,*}(s) = R(s, a) \quad (\text{Equation 2.9})$$

$$\Gamma^{A,O} = \alpha_k^{a,*}(s) = \gamma \sum_{s' \in S} T(s, a, s') O(o, s', a) \alpha'_i(s'), \forall \alpha'_k \in V' \quad (\text{Equation 2.10})$$

Next, for each belief in the set B , and for each action a , we construct Γ_b^A using each of:

$$\Gamma_b^a = \Gamma^{a,*} + \sum_{o \in O} \operatorname{argmax}_{\alpha \in \Gamma^{a,o}} (\alpha \cdot b) \quad (\text{Equation 2.11})$$

Finally, the best action for each b is given by:

$$\operatorname{argmax}_{\Gamma_b^a \in \Gamma_b^A} (\Gamma_b^a \cdot b), \forall b \in B \quad (\text{Equation 2.12})$$

After completing the value backup (Equation 2.8) for an initial set of beliefs, PBVI expands this belief set, performs value backups, and finds a new solution for the newly expanded set of beliefs [Pineau, Gordon, and Thrun 2003]. Each new belief is derived as a one-step look-ahead from each previous belief such that the new belief is the farthest from any belief already in B . This interleaving of the value iteration and the belief expansion make PBVI an “*anytime*” algorithm, meaning that the solution is guaranteed to be bounded whenever we decide to stop the algorithm (due to time, or performance constraints) [Pineau, Gordon, and Thrun 2003]. This bound depends on how comprehensively the belief set B covers the agent's

reachable belief space. As PBVI expands B by adding beliefs that are farthest from those already in the set, it progressively covers the under-explored areas of the belief space. As a result, the suboptimality bound tightens with each belief expansion, balancing computation and solution quality.

Additionally, different belief point selection methods have been developed for use in PBVI to make it more efficient in terms of convergence. This is because the value function of PBVI is dependent on both the number of beliefs being added as well as what beliefs are being added to B [Pineau, Gordon, and Thrun 2006]. Out of the five different belief expansion techniques mentioned in [Pineau, Gordon, and Thrun 2006], i.e., Random Belief Selection (RA), Stochastic Simulation with Random Action (SSRA), Stochastic Simulation with Greedy Action (SSGA), Stochastic Simulation with Exploratory Action (SSEA), and Greedy Error Reduction (GER), the GER technique is generally considered to be the most effective. However, GER suffers from scalability because its run-time is directly proportional to $|B|^3$, and this becomes progressively worse as $|B|$ keeps doubling after every round of belief expansion. This means that, in domains with large state, action, and observation spaces, it will not be feasible to compute the reachable beliefs that are beyond a certain step away from the initial set of beliefs. Stochastic Simulation with Exploratory Action (SSEA) is the second-best alternative to solve this problem since its run-time is only proportional to $|B|^2$. More details about the belief expansion techniques are discussed in Chapter 3.

Although PBVI is generally a very good choice to generate offline belief-action policies that can be integrated into the level-1 planning, it still has a couple of drawbacks. First, since PBVI is an offline planner, it is forced to maintain only a small subset of the estimated beliefs for the planned future states as opposed to a typical online planner that tailors its planning to the evolving current belief at execution time. In other words, PBVI might not generate an optimal

policy for the beliefs in its explored belief set that are significantly distant from the initial belief. Second, since the belief set of PBVI expands at a rate of twice the number of beliefs added after each iteration, the algorithm not only quickly becomes intractable in terms of computing the policies for the growing belief space but also in terms of the solution size that can introduce bottlenecks in the higher-level planners that adapt PBVI to handle the level-0 reasoning. Given these drawbacks, however, PBVI is still a better choice to integrate into level-1 planning than the other online planners since the above-discussed advantages that PBVI comes with far outweigh these drawbacks.

Chapter 3 – Methodology

In this chapter, we discuss the various algorithms that we build our research on, what makes them infeasible for larger domains, and our contributions to feasibly implement them.

Our overall goal is to analyze level-1 multi-agent Finitely Nested I-POMDP planning and decision-making in a partially observable environment. Although there are existing frameworks [Silver, and Veness, 2010] [Kakralapudi, et al. 2022] [Hoang and Low 2013] that bring us closer to this goal, they come with certain drawbacks: (1) while online solvers account for partial observability, they cannot readily handle large domains like Cybersecurity Defense [Eck, et al. Unpublished] with a large number of states, actions, and possible observations, due to the nested online level-0 reasoning (i.e., solving a level-0 POMDP at each node of the planning tree based on the agent’s current belief) resulting in a significant computational bottleneck, and on the other hand, (2) the offline solvers tested in the literature so far for such nested level-0 reasoning do not account for partial observability. Given this, we aim to incorporate level-0 offline policies generated using the PBVI (short for Point-Based Value Iteration) [Pineau, Gordon, and Thrun 2003] methodology into level-1 online planning to support the evaluation of modeled neighbor actions. This approach should help effectively address the scalability issues and help us analyze the resulting agent behavior.

3.1. I-POMDP with PBVI to Facilitate Planning in Large Domains

Using PBVI [Pineau, Gordon, and Thrun 2003] with level-1 I-POMDP planning, specifically using the well-known MCTS-based solver called the Interactive Partially Observable Monte Carlo Planning with Particle Filter (I-POMCP-PF) [Kakralapudi, et al. 2022] that is our focus, gives us two main advantages. First, it makes the level-1 planning computationally feasible by generating precomputed offline policies that can be integrated into this planning. Here, we note that, although PBVI only solves the underlying POMDP, which is essentially a single-agent problem, the resulting policies can still be integrated into the level-1 I-POMDP framework by assigning them to level-0 modeled neighbors (who are assumed to only solve the level-0 single agent POMDPs). This allows the level-1 agents to reason about their neighbors' actions based on these pre-computed policies without requiring online planning for them. Second, it accounts for partial observability when reasoning for the level-0 modeled neighbor so that its solution fits better into the partially observable I-POMDP framework.

The I-POMCP-PF solver, as mentioned earlier, is not readily feasible for larger domains like the Cybersecurity Defense domain with large state, action, and observation spaces (e.g., our baseline setup for the Cybersecurity Defense involves 125 states, 11 actions, and 126 observations, and comes with 2 attacker agents and 2 or 3 defender agents). This framework needs to continuously maintain and update a nested particle filter to keep track of the belief space for both the planning agent as well as the modeled neighbors (that are assumed to be one level below the planning agent's reasoning level). In other words, each nested particle of this nested particle filter *needs to maintain a separate particle filter for each modeled neighbor to update its respective modeled belief*. As the domain becomes larger and the number of possible beliefs increases exponentially, it would in turn necessitate a larger number of particles to

maintain accuracy of the belief updates and it will quickly become intractable to reason and plan for each neighbor. For example, for a particle filter with 125 nested particles (empirically reasonable), if each such nested particle is to maintain a separate particle filter for each of the 3 modeled neighbors, the algorithm must process $125 * 3 * 125 = 46,875$ particles just to evaluate one single action in the planning trajectory.

A better solution for the above would be to use an *offline pre-computed* solution to eliminate the bottleneck of having to plan ad-hoc for each level-0 neighbor of the level-1 planning agent. It would be even more advantageous for such an offline planner to handle partial observability for its solution to better fit into the partially observable level-1 planning, generate the solution quickly, and keep it fairly small for computational efficiency. For example, using an offline pre-computed solution, since the level-1 planning agent can now quickly retrieve the best possible action for each of the neighbor's particle filter (i.e., the belief) using a dot product, the planning agent can avoid the overhead of having to "plan" for each such particle filter. In other words, each of the above-mentioned 375 particle filters (125 particle filters * 3 neighbors) can be simply represented as a belief vector to carry out a dot product with the set of level-0 Alpha-vectors rather than having to be propagated a *horizon* number of times into the future to account for all possible future states and their respective rewards to build a value function and finally compute the best action. An offline planner already accounts for such future beliefs and rewards when computing its policy and all that the planning agent now needs to do is to map each neighbor's current belief to a pre-computed best action.

PBVI [Pineau, Gordon, and Thrun 2003] ticks all these boxes. It is an offline solver that solves the single agent POMDP and offers an anytime solution governed by established error bounds. It can be used to solve the feasibility issue that plagues the MCTS-based level-1 planning in large domains. Specifically, because PBVI generates an offline solution in the form of

a set of Alpha-vectors that can be efficiently integrated into the level-1 planning and facilitate a quick dot product-based solution retrieval for each of the modeled neighbors' beliefs, it eliminates the planning overhead for each neighbor. PBVI's solution is also relatively small since it implements a highly selective belief exploration, and this smaller solution makes the level-1 planning even more efficient as the retrieval of the neighbor action becomes faster.

Furthermore, computations involving such vector-based data structures can be significantly sped up using the modern hardware acceleration tools to achieve a potential $O(1)$ time-complexity. We also note that the PBVI policy is generated at a frame (i.e., type of the agent) level and the same solution can be used for all level-0 agents that belong to this respective frame, thereby eliminating the overhead to separately generate these policies for each individual agent. Another advantage of PBVI is that it effectively deals with partial observability by only maintaining a set of reachable beliefs, expanding them one time-step into the future at a time and continuously updates its value function in the process. Handling partial observability results in a more efficient planning overall as the level-1 agent's estimate of the level-0 neighbor's action in the partially observable environment is more accurate compared to the solution of a solver that assumes full observability (e.g., I-POMDP-Lite). The frequency and extent of this belief expansion is also flexible (i.e., we can control how many beliefs are being added and how frequently between each value iteration), and we can fine tune it to meet our performance requirements since this is an anytime-solution algorithm.

Note that PBVI is not without its shortcomings. These shortcomings can manifest either during the offline policy generation or during the online planning phases. First, a disadvantage PBVI faces during the offline policy generation arises due to its belief exploration methodology. PBVI does not account for the complete belief space and only maintains a set of reachable beliefs. While this results in a much faster solution, the algorithm might fail to account for a few

remote beliefs that could seriously impact the value function. Second, the solution generated by PBVI could not be completely compatible with the level-1 planning. That is, PBVI uses a fundamentally different algorithm to generate its policy and therefore could arrive at a different solution for a given scenario than what the level-1 planning agent might be expecting. Furthermore, since PBVI generates a generalized solution that can be used by any modeled neighbor belonging to the same frame, this could lead to coordination issues between the modeled neighbors where all the neighbors of the same frame choose the same redundant action even if the ideal solution for the situation would be to divide and conquer. While these drawbacks could confuse the level-1 planning agent and therefore slow the convergence of the level-1 algorithm, the advantages of using PBVI that we had discussed above far outweigh these disadvantages, reinforcing PBVI as a suitable option for our goal.

We therefore plan to generate the level-0 offline policies using the PBVI reasoning and plug them into the level-1 planning. We note that, since the domain within our scope, i.e., Cybersecurity Defense domain [Eck, Unpublished], exhibits both fully and partially observable state components), we re-implement PBVI in a factored form to exploit this structure. Specifically, we adapt PBVI into a Mixed Observability Markov Decision Process (MOMDP) formulation, which we refer to as “MOMDP-PBVI”. This improves the efficiency of the original PBVI algorithm by separating the fully observable components (e.g., the agent’s internal state such as its location) from the belief updates (which track the partially observable external states, such as the security state of each subnetwork). This re-implementation enables the level-1 agents to quickly retrieve a pre-computed action choice when evaluating the best possible action for each modeled neighbor thereby improving the computational bottleneck. This helps us towards our goal to analyze the agent behavior using the level-1 I-POMDP reasoning.

3.1.1. MOMDP-PBVI Algorithm – A Re-Implementation of PBVI

We now describe the modified PBVI algorithm below, calling it MOMDP-PBVI, which is suitable to use for the Cybersecurity Defense domain (with separate fully and partially observable components) that we are interested in. This algorithm is a structural adaptation of the original PBVI [Pineau, Gordon, and Thrun 2003] and improves efficiency by factoring the state space into fully observable (internal states) and partially observable (external states) components, which reduces the dimensionality of the belief space. For instance, in a scenario with 125 external states and 4 internal states, a belief vector of length 500 (125×4) can be reduced to 125×1 when the internal states are fully observable, resulting in a more compact and tractable representation. The functionality, however, remains the same as the original.

Given that we are factoring the state space into fully observable internal states, and partially observable external states, the following represent notational level modifications to a few terms that we discuss earlier in Chapter 2 to explain the PBVI algorithm.

- Each belief b is now represented using both the uncertainty of the external states represented with a belief b , and the fully known internal state is as (b, is) .
- The transition, $T(s, a, s')$, observation, $\Omega(o, s', a)$, and Reward, $R(s, a)$ functions from the above Equation 2.8 are re-written as $T(es, is, a, C, es', is')$, $\Omega(o, es', is', a, C)$, and $R(es, is, a, C)$, respectively, to account for the factored state space, and also to represent the action space split into the level-0 planning agent's action a and the configuration of the neighbors C .
- $\alpha^{a,*} \in \Gamma^{A,*}$ from earlier Equation 2.9 is now computed with certainty for each internal state and is represented using $\alpha^{a,*,is} \in \Gamma^{A,*,IS}$.

- $\alpha^{a,o} \in \Gamma^{A,O}$ from earlier Equation 2.10 is now computed additionally for each next internal state is' given the know internal state is , and is represented using $\alpha^{a,o,is,is'} \in \Gamma^{A,O,IS,IS'}$.
- Equation 2.12's terms remain the same since it already considers all possible current and future internal states and beliefs.

Algorithm MOMDP-PBVI

The following MOMDP-PBVI algorithm will generate the offline policies for each level-0 neighbor such that they can be plugged into the online level-1 I-POMDP planning. The solution is a set of alpha-vectors—each representing the value function for a particular belief and action, used to approximate the optimal value function over the continuous belief space in a POMDP.

Variables and Definitions for MOMDP-PBVI:

- ES is the set of external states $\{es_1, es_2, \dots, es_{|ES|}\}$, representing the partially observable aspects of the environment—e.g., the security status of subnetworks in the Cybersecurity Defense domain.
- IS is the set of internal states $\{is_1, is_2, \dots, is_{|IS|}\}$, representing the fully observable state variables that are internal to the agent—e.g., the current location of the agent.
- $\alpha^{a,is} = \langle v_1, v_2, \dots, v_{|ES|} \rangle$ is an Alpha-vector that contains the expected utility v of each partially observable external state $es \in ES$, for a given action a , and internal state is .
- $B_{curr} = \{(b, is)_1, (b, is)_2, \dots, (b, is)_n\}$ is a set of tuples of beliefs and their respective internal states. These represent only the subset of the total belief space—specifically, the

beliefs that were explored and selected during the belief expansion process that determines which belief points are used for value backups.

- $V_{curr} = \{\alpha_1^{a_1, is_1}, \alpha_2^{a_2, is_2}, \dots, \alpha_n^{a_n, is_n}\}$ is the current set of Alpha-vectors $\alpha_k^{a_k, is_k}$, where a_k, is_k are the action, and the internal state of the k^{th} Alpha-vector. Here, $n \geq |B_{curr}|$, i.e., we have at least one Alpha-vector corresponding to each belief, and more in case there are multiple optimal Alpha-vectors for the same belief – this is determined by the backup process, which adds one or more alpha-vector for each belief depending on how many action-internal state combinations give the best expected value for that belief.
- V_{new} represents the updated set of Alpha-vectors whenever the *Improve()* function is called.
- $\alpha^{a,*,is} \in \Gamma^{A,*,IS}$ is a projection vector (as described in the PBVI paper) that contains immediate rewards for all the external states ES of the environment, given the action a , and the internal state is .
 - $\Gamma^{A,*,IS}$ is the set of $\alpha^{a,*,is}$ for each action and internal state.
- $\alpha^{a,o,is,is'} \in \Gamma^{A,O,IS,IS'}$ is a projection vector representing the future discounted reward for all the external states ES a given action a , observation o , internal state is , and next internal state is' .
- $\Gamma^{A,O,IS,IS'}$ is the set of $\alpha^{a,o,is,is'}$ for each action a , observation o , internal state is , and next internal state is' .
- Γ_b^A is the set of Γ_b^a for each action a where Γ_b^a represents the set of projection vectors comprising both immediate and future rewards (see *BACKUP* Line 10).
- A is the set of all possible actions a for the agent.
- $\mathcal{C}$ is the set of all possible configurations C .
- O is the set of all possible observations o .

- $S = \{es_1, es_2, \dots, es_{|ES|}\} \times \{is_1, is_2, \dots, is_{|IS|}\}$ is the state space that is comprised of the external states ES , and the internal states IS .
- $R(es, is, a, C)$ is the reward function which returns the reward given an external state es , internal state is , action a , and configuration C .
- $\Omega(o, es', is', a, C)$ and $T(es, is, a, C, es', is')$ are the observation, and transition functions, respectively.
- γ is the discount factor for the future values.
- H = horizon

Algorithm MOMDP-PBVI:

PBVI-MOMDP is an offline level-0 planning algorithm that generates the optimal policy for the level-0 agent of a multi-agent system by assuming that this agent does not model its neighbors.

The algorithm starts with *PBVI_CREATEPLAN()* procedure using an initial set of beliefs. The alpha-vectors are initialized using respective immediate rewards cached as $(\Gamma^{A,*,IS})$

(*PBVI_CREATEPLAN()* Line 8), and the possible future rewards computed as $\Gamma^{A,O,IS,IS'}$

(*IMPROVE()*—Line 16). We update their value using the procedure *IMPROVE()*, also known

as a ‘value-iteration’ step. From these generated sets of alpha-vectors, only those alpha-vectors corresponding to the current set of explored beliefs are retained using a value-backup step using

the procedure *BACKUP()* (Line 10). Simultaneously, between the value iterations, the

algorithm also expands its current set of beliefs (usually doubling its size) using the desired

belief expansion techniques (e.g., *SSEAEXPANDBELIEFS()*, *GEREXPANDBELIEFS()*, etc.

[Pineau, Gordon, and Thrun 2008]). Each of these belief expansion techniques subsequently

employs the *UPDATEBELIEF()* procedure to compute the 1-step-ahead set of future beliefs

in each iteration. PBVI is an anytime-algorithm [Pineau, Gordon, and Thrun 2003] which ensures

that the solution at any given iteration is always bound by its error function, and we can cut off the algorithm based upon desired level of convergence, satisfactory exploration of the belief space, or even time/computation constraints.

The following *PBVICREATEPLAN()* procedure starts with a set of initial beliefs, and the corresponding alpha-vectors, and returns the optimal policy value function in the form of alpha-vectors V_{curr} .

- Line 10 is the value-iteration step where new sets of alpha-vectors are generated for the current and the future states, based on the current set of alpha-vectors. Value-backup is also performed within the same function call.
- Line 11 shows the belief expansion step, currently set to use SSEA.

```

1: procedure PBVICREATEPLAN( $V_{curr}, b_{init}, iS_{init}, \epsilon, h, \gamma$ ):
2:    $B_{curr} \leftarrow \{(b_{init}, iS_{init})\}$ 
3:    $cachedHeuristics \leftarrow \phi$  # global variable
4:    $\Gamma^{A,*,IS} \leftarrow \phi$  # initializing Gamma A * IS
5:   for  $a \in A$  do: # caching Gamma A * IS
6:     for  $is \in IS$  do:
7:        $\alpha^{a,*,is}(es) \leftarrow \sum_{C \in \mathcal{C}} R(es, is, a, C) P(C), \forall es \in ES$ 
8:        $\Gamma^{A,*,IS} \leftarrow \Gamma^{A,*,IS} \cup \alpha^{a,*,is}$ 
9:   while True: # perform value-iteration, backup, and belief expansion until converge
10:     $V_{new} \leftarrow IMPROVE(V_{curr}, B_{curr}, \Gamma^{A,*,IS}, h)$ 
11:     $B_{new} \leftarrow B_{curr} \cup SSEAEXPANDBELIEFS(B_{curr})$  # also can call GER Expand Beliefs
12:     $V_{curr} \leftarrow V_{new}$  # updating the current set of Alpha-vectors
13:     $V_{curr} \leftarrow PRUNEDOMINATEDALPHAS(V_{curr})$ 
14:     $B_{curr} \leftarrow B_{new}$  # updating the current set of belief points
15:    if  $convergenceThreshold < \epsilon$  do: # refer to PBVI paper for more information
16:      break
17:  return  $V_{curr}$  # global alpha-vectors to-be plugged into I-POMCP-PF

```

The following *PRUNEDOMINATEDALPHAS()* procedure is an efficiency helper function to check and remove any alpha-vectors that are completely dominated (i.e., the computed value function is lesser for each and every state of the alpha-vector when compared to another

existing alpha-vector from the current set – meaning such dominated alpha-vector would never be picked up for any belief and therefore removing the same will make the algorithm more efficient). To determine whether a candidate alpha-vector is completely dominated, we compare this candidate alpha-vector to all the remaining alpha-vectors from the same set (Line 3 iterates over the set of the current set of alpha-vectors, and Line 5 iterates over the candidates from the same set). For each candidate, we check if there exists at least one state that has higher value than that of any other alpha-vector from the set (Line 9) or prune it if this check fails (Line 13), ultimately compacting the size of the solution.

```

1: procedure PRUNEDOMINATEDALPHAS( $V_{curr}$ ):
2:   for  $is \in IS$  do:
3:     for  $\alpha_i^{is} \in V_{curr}$  do:
4:        $domFlag \leftarrow True$ 
5:       for  $\alpha_j^{is} \in V_{curr}$  do:
6:         if  $\alpha_i^{is}$  equal to  $\alpha_j^{is}$  do:
7:           continue
8:         for  $es \in ES$  do:
9:           if  $\alpha_j^{is}(es) > \alpha_i^{is}(es)$  do:
10:             $domFlag \leftarrow False$ 
11:            break
12:         if  $domFlag$  is True do:
13:            $V_{curr} \leftarrow V_{curr} / \alpha_j^{is}$  # removing the dominated Alpha-vector
14:   return  $V_{curr}$ 

```

The following *IMPROVE()* procedure performs value iteration for the current (already cached) and the future states corresponding to the current set of alpha-vectors, and in-turn calls the value-backup function to store only the needed set of alpha-vectors for the subsequent time-steps.

- Line 16 stores the newly generated set of alpha-vectors that correspond to the future states.

- Line 19 calls the value-backup function.

```

1: procedure IMPROVE( $V_{curr}, B_{curr}, \Gamma^{A,*,IS}, h$ ):
2:   for  $i \in (1, 2, \dots, h)$  do: # we run this process for  $h$  times
3:      $V_{curr, is'} \leftarrow \phi$ 
4:     for  $is' \in IS$  do:
5:        $V_{curr, is'} \leftarrow \{\alpha^{is'} \mid \alpha^{is'} \in V_{curr}\}$ 
6:       if  $V_{curr, is'}$  equal to  $\phi$  do: # MOMDP fix when all  $is$  are not explored yet
7:          $V_{curr, is'} \leftarrow V_{curr, is'} \cup \alpha^{is'} \mid \alpha^{is'}(es) = 0, \forall es \in ES$ 
8:        $\Gamma^{A,O,IS,IS'} \leftarrow \phi$  # initializing the Gamma projection vectors set
9:       for  $a \in A$  do:
10:        for  $o \in O$  do:
11:          for  $is' \in IS$  do:
12:            for  $is \in IS$  do:
13:              for  $\alpha^{is'} \in V_{curr, is'}$  do:
14:                for  $es \in ES$  do:
15:                   $\alpha^{a,o, is, is'}(es) \leftarrow$ 
16:                     $\gamma \sum_{C \in \mathcal{C}} \sum_{es' \in ES} \Omega(o, es', is', a, C) T(es, is, a, C, es', is') P(C) \alpha^{is'}(es')$ 
17:                   $\Gamma^{A,O,IS,IS'} \leftarrow \Gamma^{A,O,IS,IS'} \cup (\alpha^{a,o, is, is'})$ 
18:         $V_{new} \leftarrow \phi$ 
19:        for  $(b, is) \in B_{curr}$  do: #  $B_{curr}$ 
20:           $V_{backup} \leftarrow \text{BACKUP}((b, is), \Gamma^{A,*,IS}, \Gamma^{A,O,IS,IS'})$  # returns a set of A.Vs
21:          for  $\alpha^{a, is} \in V_{backup}$  do:
22:             $V_{new} \leftarrow V_{new} \cup \alpha^{a, is}$ 
23:         $V_{curr} \leftarrow V_{new}$ 
24:   return  $V_{curr}$ 

```

The following *BACKUP*() procedure performs the value-backup for the currently generated set of alpha-vectors and retains only those that correspond to the currently explored set of beliefs.

- Line 9 shows how the best alpha-vector for each given belief from the current set of beliefs is chosen computed using the dot-product between each belief and the current set of alpha-vectors.

```

1: procedure BACKUP( (b, is),  $\Gamma^{A,*,IS}$ ,  $\Gamma^{A,O,IS,IS'}$ ):
2:    $\Gamma_b^A \leftarrow \phi$ 
3:   for  $a \in A$  do:
4:      $\Gamma^{a,*,is} \leftarrow \{\alpha^{\dot{a},*,is} \mid \alpha^{\dot{a},*,is} \in \Gamma^{A,*,IS} \wedge \dot{a} = a \wedge is = is\}$ 
5:     for  $o \in O$  do:
6:       for  $is' \in IS$  do:
7:          $\Gamma^{a,is,o,is'} \leftarrow \{\alpha^{\dot{a},\dot{o},is,is'} \mid \alpha^{\dot{a},\dot{o},is,is'} \in \Gamma^{A,O,IS,IS'} \wedge \dot{a} = a \wedge is = is \wedge \dot{o} = o \wedge is' = is\}$ 
8:          $\Gamma_b^a \leftarrow \Gamma^{a,*,is} + \sum_{o \in O} \sum_{is' \in IS} \arg \max_{\alpha \in \Gamma^{a,o,is,is'}} (\alpha \cdot b)$ 
9:          $\Gamma_b^A \leftarrow \Gamma_b^A \cup \Gamma_b^a$  # stored for each action
10:        for  $\alpha^{a,is} \in \{\arg \max_{\Gamma_b^a \in \Gamma_b^A} (\Gamma_b^a \cdot b)\}$  do:
11:           $V_{backup} \leftarrow V_{backup} \cup \alpha^{a,is}$ 
12:   return  $V_{backup}$ 

```

The following *UPDATEBELIEF()* procedure performs a Bayesian update 1-time-step into the future for the passed-in belief.

```

1: procedure UPDATEBELIEF((b, is), a, o, is'):
2:   for  $es' \in ES$  do:
3:      $newProbability \leftarrow 0$ 
4:     for  $es \in ES$  do:
5:        $newProbability \leftarrow newProbability + \Omega(o, es', is', a, C)T(es, is, a, C, es', is')P(C)b(es)$ 
6:        $b'(es') \leftarrow newProbability$  # given is'
7:        $b'.Normalize()$ 
8:   return (b', is')

```

The following *SSEAEXPANDBELIEFS()* procedure uses a max-min technique (i.e., the belief that has the maximum value of the minimum Manhattan distance to any given belief from the current set of beliefs) to selectively add beliefs to the current set of explored beliefs. More details in [Pineau, Gordon, and Thrun 2008].

```

1: procedure SSEAEXPANDBELIEFS( $B_{curr}$ ):
2:   for  $(b, is) \in B_{curr}$  do:
3:      $B_{new} \leftarrow \phi$ 
4:      $B_{candidates} \leftarrow \phi$ 
5:     for  $a \in A$  do:
6:       for  $o \in O$  do:
7:         for  $is' \in IS$  do:
8:            $(\tilde{b}, \tilde{is}) \leftarrow \text{UPDATEBELIEF}((b, is), a, o, is')$ 
9:            $B_{candidates} \leftarrow B_{candidates} \cup (\tilde{b}, \tilde{is})$ 
10:     $(b', is') \leftarrow \text{argmax}_{(\tilde{b}, \tilde{is}) \in B_{candidates}} (\text{minManhattanDistance}((\tilde{b}, \tilde{is}), B_{curr}))$ 
11:     $B_{new} \leftarrow B_{new} \cup ((b', is'))$ 
12:  return  $B_{new}$ 

```

The following *GEREXPANDBELIEFS*() procedure, where GER stands for Greedy Error

Reduction, adds beliefs to the current set of beliefs such that each new belief added will maximally reduce the worst-case error bound for the value-function, thereby making the algorithm converge faster. More details can be found in [Pineau, Gordon, and Thrun 2008].

```

1: procedure GEREXPANDBELIEFS( $B_{curr}, V_{curr}$ ): #  $\epsilon_{GER}$  &  $\hat{O}$  are defined below
2:    $N \leftarrow |B_{curr}|$ 
3:    $B_{new} \leftarrow \phi$ 
4:    $max \leftarrow x | x < 0$ 
5:   for  $i \in (1, 2, \dots, N)$  do:
6:      $(\widetilde{b, is}), \tilde{a} \leftarrow$ 
       
$$\text{argmax}_{(b, is) \in B_{curr}, a \in A} \sum_{o \in O} \sum_{is' \in IS | ((b, is), a, o, is') \notin \text{cachedHeuristics}} (\hat{O}((b, is), a, o, is') \\ \epsilon_{GER}(\text{UPDATEBELIEF}((b, is), a, o, is')))$$

7:      $\tilde{o}, \tilde{is'} \leftarrow$ 
       
$$\text{argmax}_{o \in O, is' \in IS | ((\widetilde{b, is}), \tilde{a}, o, is') \notin \text{cachedHeuristics}} \hat{O}((\widetilde{b, is}), \tilde{a}, o, is') \\ \epsilon_{GER}(\text{UPDATEBELIEF}((\widetilde{b, is}), \tilde{a}, o, is'))$$

8:      $(b', is') \leftarrow \text{UPDATEBELIEF}((\widetilde{b, is}), \tilde{a}, \tilde{o}, \tilde{is'})$ 
9:      $\text{cachedHeuristics} \leftarrow \text{cachedHeuristics} \cup ((\widetilde{b, is}), \tilde{a}, \tilde{o}, \tilde{is'})$  # cache this combo
10:     $B_{new} \leftarrow B_{new} \cup ((b', is'))$ 
11:  return  $B_{new}$ 

```

Additional procedures for the $GEREXPANDBELIEFS()$ procedure:

- The logic to calculate the **GER error bound**, i.e., $\epsilon_{GER}(b', is')$ is given by:

If $((b', is') \notin B_{curr})$ **do**:

$B \leftarrow \{b | (b, is') \in B_{curr}\}$

If B not equal to ϕ **do**:

$\epsilon_{GER}(b', is') \leftarrow \min_{\forall b \in \{(b) | (b, is) \in B_{curr} \text{ and } is = is'\}} \sum_{es \in ES} (k - \alpha_{is}(es)) * (b'(es) - b(es))$;

where $k = \begin{cases} \frac{R_{max}}{1-\gamma} & \text{if } b'(es) \geq b(es) \\ \frac{R_{min}}{1-\gamma} & \text{if } b'(es) < b(es) \end{cases}$ and $\alpha_{is} \in V_{curr}$ that maximizes value for respective (b, is)

Else do: # during initial iterations where all $is \in IS$ not yet added to B_{curr}

$\epsilon_{GER}(b', is') \leftarrow \frac{n(R_{max}-R_{min})}{1-\gamma}$ # $n \geq 1$ such that $\frac{n(R_{max}-R_{min})}{1-\gamma}$ becomes very large.

Else $\epsilon_{GER}(b', is') \leftarrow 0$
- The function that calculates the reachability, $\hat{O}((b, is), a, o, is')$, is given by

$$\hat{O}((b, is), a, o, is') = \sum_{C \in \mathcal{C}} P(C) \sum_{es \in ES} \sum_{es' \in ES} \Omega(o, es', is', a, C) T(es, is, a, C, es', is') b(es)$$

3.1.2. Level-1 Planning with PBVI for Level-0

We adopt the following I-POMCP-PF algorithm from [Kakralapudi 2021] to discard agent communication and replace the POMCP-PF based level-0 reasoning with PBVI. For simplicity, we also discard the frame-action anonymity from the same paper and consider all the agents (neighbors) in the system when evaluating a joint action. Unlike what we do with MOMDP-PBVI, re-implementing the I-POMCP-PF algorithm is not strictly needed even for an environment with both partial and fully observable components because the particle filter of the I-POMCP-PF algorithm already represents beliefs over the external (partially observable) states, while the internal (fully observable) state is tracked separately and updated deterministically. This separation is naturally handled in the implementation, so no structural changes to the algorithm are required.

Algorithm I-POMCP-PF (Level-1)

Variables and Definitions for I-POMCP-PF (Level-1):

- Agent i refers to the subject agent or the planning agent, and $Ag/\{i\}$ refers to the set of its neighbors.
- l is the current level of hierarchy in the planning process.
- τ refers to the number of trajectories used in the planning.
- t refers to the current horizon (initialized as 0) and H refers to the planning horizon.
- h (analogous to observation-action history) refers to the current belief node of the tree (initialized as ε).
 - The tree itself is denoted using B .
- $n_{i,l}(h)$ is the number of visits to the node h and $n_{i,l}(ha)$ is the number of times action a has been chosen when in node h .
- $\beta_{i,l}$ refers to the nested particle filter associated with the current node h , for agent i at level l .
 - β_o represents an observation particle filter that is used to estimate an observation during the planning stage.
 - $\omega(o)$ is the weight of the observation particle filter corresponding to observation o
- $(p_{l_k}, w_k) \in \beta_{i,l}$ refers to the k^{th} nested level- l particle of the particle filter, and its respective weight.
- $s \in S$ is a state from the set of possible states.
- $o \in O_i$ is an observation from the set of possible observations.

- o_i is an observation that is specific to agent i .
- $A_i \times A_j \times \dots \times A_z$ is the joint set of actions of the agents. From this:
 - a_i is an action that is specific to agent i .
 - a_{-i} is the set of actions estimated for the neighbors of agent i .
 - $\mathcal{C}$ is the set of all possible configurations C from the point of view of the modeled level-0 neighbors.
- T is the transition function given by $T_i(s, a_i, a_{-i}, s') = P(s' | s, a_i, a_{-i})$.
 - Here, a_{-i} is analogous to the Configuration C from the PBVI notations.
- Ω is the observation function in the POMDP with $\Omega(s', a_i, a_{-i}, o_i) = P(o_i | a_i, a_{-i}, s')$
- R refers to the reward computed by the function $R_i(s, a_i, a_{-i}) \in \mathbb{R}$.
 - r_i refers to the reward computed by agent i .
- $Q_{i,l}(h, a_i)$ is the Q-value associated with the node h upon taking the action a_i .
- ucb_c is the constant that drives the UCB-based exploration.

Algorithm I-POMCP-PF:

I-POMCP-PF (short for Interactive Partially Observable Monte-Carlo Planning – (with) Particle Filter) [Kakralapudi 2021] is a multi-agent, multi-level hierarchical decision-making planning algorithm that accepts an environmental belief of the agent and returns the computed optimal action for that belief. For simplicity, we limit this algorithm's scope to level-1 where the subject agent models each of its neighbors to be a level-0 agent. A level-0 agent is one that chooses its own action without modeling its neighbors (meaning it assumes a random or heuristic-based policy to compute the joint action for its neighbors which in turn is used along with its own planning action to simulate the environment). The flow of the algorithm is outlined below.

We start with the main *IPOMCPO* procedure which accepts the current belief of the simulation and returns the optimal action. This function initializes the planning tree's root node using the passed-in belief and associates a particle filter to this belief to represent its distributed state space. For trajectory τ number of times (to facilitate a thorough exploration of the search tree), this belief node and the associated particle filter are passed into the *UPDATETREE()* procedure where the core functionality of the algorithm lies (*IPOMCPO* Line 3).

UPDATETREE() updates the Q-value associated with the root node and in-turn updates the respective Q-values for all the future beliefs it encounters while it recursively plans down the given horizon. This algorithm, as mentioned in [Kakralapudi 2021], passes the entire nested particle filter down the tree for a better efficiency in belief estimation and value function computation.

- This *UPDATETREE()* procedure first checks if the tree has reached the specified depth as per the planning horizon and returns 0.0 as the future reward in that case (*UPDATETREE()* Line 3).
- Otherwise, if the current belief is a new leaf, it first initializes a belief node for the leaf (using *EXPAND(h, i, l)* invoked in *UPDATETREE()* Line 3), with 0 visits for the belief node and each of the children action nodes, and calls the *ROLLOUTALL()* procedure which in-turn calls the *ROLLOUT()* procedure for each particle (*UPDATETREE()* Line 6). These rollout functions essentially do a random rollout of the rewards by simulating each particle filter down the horizon using randomly sampled joint actions, since a new belief has not yet accumulated any particles. In other words, we get a rough value estimate for this belief node for a given horizon which in turn aids the UCB algorithm in the future trajectories.

- For the other belief nodes (that have been visited at least once in the past), the *UPDATETREE()* procedure models an action for each neighbor (for the level-0 neighbors, this action is retrieved from the pre-computed offline policy, i.e., PBVI) as shown in the *PLANNEIGHBORS()* procedure (*UPDATETREE()* Line 15).
 - The *PLANNEIGHBORS()* procedure works by choosing the optimal action for each modeled neighbor. Suppose that the subject agent is a level-1 agent. Then, when the procedure is called, it receives as an argument the level-1 agent's nested particle. Note that such a particle has a set of nested level-0 particle filters, one filter for each neighbor. Given this particle, the procedure then fetches the optimal action for each of the nested level-0 particle filters using the pre-computed PBVI policy (this is where the original algorithm from [Kakrapudi 2021] calls the online level-0 POMCPPF procedure instead of PBVI).
- In addition to planning for each neighbor, this *UPDATETREE()* procedure also chooses the subject agent's planning action using the argmax-UCB function (*UPDATETREE()* Line 8). Using the joint action, the current (root) belief node is updated (along with the estimated beliefs of each modeled neighbor) in the *NESTEDBELIEFUPDATE()* procedure (*UPDATETREE()* Line 18), a new weighted observation is sampled (*UPDATETREE()* Lines 20 to 24), and the nested particle filter is propagated to correspond to the updated belief.
 - To facilitate this *NESTEDBELIEFUPDATE()* procedure, the algorithm uses a nested particle filter where each particle of the subject agent's particle filter has n particle filters nested to it, each representing the estimated belief of one of the n modeled neighbors, the same as those mentioned in the

PLANNEIGHBORS() procedure above. In other words, the subject agent maintains a belief over the beliefs of each of its neighbors. The update happens in two steps for each nested particle of the level-1 subject agent's particle filter. First, we sample the next state of the subject agent's nested particle using the joint action (this next state is later used by the subject agent to get its own observation). The subject agent also uses this next state to sample an observation for each modeled neighbor in the partially observable environment (*NESTEDBELIEFUPDATE()* Line 6). Second, using the sampled observation for each neighbor, the subject agent uses the modeled optimal (PBVI) action chosen for the neighbor plus a random joint action of this level-0 neighbor's neighbors to sample the modeled neighbor's next state that corresponds to the earlier sampled observation (*NESTEDBELIEFUPDATE()* Line 8). The update of the particle filter is completed after each of the level-1 nested particles completes this procedure.

- *UPDATETREE()* procedure recurses (*UPDATETREE()* Line 27) until the updated belief node is either a leaf node (in which case there is a rollout) or until the planning reaches the specified horizon. The cumulative rewards (*UPDATETREE()* Line 27), the Q-values (*UPDATETREE()* Line 30), the counts of visits to each belief node and respective action nodes (*UPDATETREE()* Lines 28, 29), are also recursively updated in the process, and backpropagated up the tree to update the root node.
- Finally, the root node selects an action with these updated values using the argmax Q-function shown in (*IPOMCPO()* Line 5).

The following procedure *IPOMCPO* updates the I-POMCP-PF tree over a given number of trajectories τ (Line 2) and returns the optimal action a_i^* for the given belief represented using the level- l particle filter $\beta_{i,l}$.

```

1: procedure IPOMCPO( $\beta_{i,l}, l$ ):
2:   for  $traj \in 1, 2, \dots, \tau$  do
3:      $UPDATETREE(\beta_{i,l}, 0, \varepsilon, l)$  # 0 is the current horizon,  $\varepsilon$  is the root node
4:      $a_i^* \leftarrow \operatorname{argmax}_{a_i \in A_i} Q_{i,l}(\beta_{i,l}, a_i)$ 
5:   return  $a_i^*$ 

```

The following procedure *EXPAND* creates a new leaf node when we encounter a belief node with 0 previous visits (*UPDATETREE* Line 4) and updates its initial visit count as 0.

```

1: procedure EXPAND( $h, i, l$ ):
2:    $B_{i,l}(h) \leftarrow \emptyset$  # expanding the tree from node  $h$ 
3:    $n_{i,l}(h) \leftarrow 0$ 
4:   for  $a \in A_i$  do
5:      $n_{i,l}(ha) \leftarrow 0$ 
6:      $Q_{i,l}(h, a) \leftarrow 0$ 

```

The following procedure *ROLLOUTALL* calls the rollout function for each particle of the particle filter (Line 3), and returns the final reward computed for the whole particle filter over the remaining horizon from t (Line 6).

```

1: procedure ROLLOUTALL( $\beta_{i,l}, t$ ):
2:    $R \leftarrow 0$ 
3:   for  $(p, w) \in \beta_{i,l}$  do
4:      $s \leftarrow p.state$ 
5:      $R \leftarrow R + w.ROLLOUT(s, t)$ 
6:   return  $R$ 

```

The following procedure *ROLLOUT()* simulates a rollout for each respective state of each particle, and returns the reward computed for this state over the remaining horizon t (Line 10).

```

1: procedure ROLLOUT( $s, t$ ):
2:    $R \leftarrow 0, t' \leftarrow t$ 
3:   while  $t < H$  do
4:      $C \leftarrow \text{SampleRandomConfiguration}()$ 
5:      $a_i \leftarrow \text{SampleAction}(A_i)$ 
6:      $s', r_i \leftarrow \text{Simulate}(s, a_i, C)$ 
7:      $R \leftarrow R + \gamma^{t-t'} \cdot r_i$ 
8:      $s \leftarrow s'$ 
9:      $t \leftarrow t + 1$ 
10:  return  $R$ 

```

The following procedure *UPDATETREE()* simulates the planning tree thereby updating the explored belief nodes and their Q-values with respect to various actions. The Q-value for the root node with respect to each possible action is updated too, letting the algorithm decide the optimal action for the current world belief.

- Lines 4 to 6 check if the explored node is a leaf, add it to the tree, and call the rollout.
- Line 8 chooses the optimal exploratory action for the current belief based on the given UCB-C value.
- Line 15 gets the modeled action for each neighbor.
- Line 18 updates the nested particle filter's beliefs, i.e., the belief of the subject agent and its modeled neighbors.
- Lines 20 to 23 build the observation particle filters based on the sampled next state obtained from each particle of the subject agent's particle filter. The weights of each observation particle filter are also updated which will later be used for sampling an actual observation for the subject agent.

- Line 24 samples an observation based on the weights of the constructed observation particle filters.
- Line 26 propagates the current belief node to the next belief node.
- Line 27 recursively updates the reward, simulating the next step of the planner until a horizon H .
- Lines 28, and 29 update the number of visits to the current belief, and the number of times the current action is chosen for the current belief, respectively.
- Line 30 updates the Q-value for the given belief-action pair.

```

1: procedure UPDATETREE( $\beta_{i,l}, t, h, l$ ): #  $t$  is the current depth, we start with 0
2:   if  $t \geq H$  then                                     # planning horizon- $H$  is the limit
3:     return 0
4:   if  $h$  is a leaf then #  $h$  is the current belief node
5:     EXPAND( $h, i, l$ )
6:     return ROLLOUTALL( $\beta_{i,l}, t$ )
7:   else
8:      $a_i \leftarrow \operatorname{argmax}_{a \in A_i} Q_{i,l}(h, a) + \operatorname{ucb}_c \cdot \sqrt{\frac{\log(n_{i,l}(h))}{n_{i,l}(ha)}}$ 
9:      $r \leftarrow 0$ 
10:    for  $o \in Z$  do
11:       $\beta_o \leftarrow \emptyset$                                 # observation particle filter
12:       $\omega(o) \leftarrow 0$                                 # weight of the observation particle filter
13:      for  $(p_{l_k}, w_k) \in \beta_{i,l}$  do
14:         $s \leftarrow p_{l_k} \cdot \text{state}$ 
15:         $a_{-i} \leftarrow \text{PLANNEIGHBORS}(p_{l_k}, l)$         # neighbors' actions
16:         $a \leftarrow a_i \cup a_{-i}$  # joint action of agent + neighbors
17:         $s', r_k \leftarrow \text{Simulate}(s, a)$ 
18:         $p'_{l_k} \leftarrow \text{NESTEDBELIEFUPDATE}(s', p_{l_k}, a)$ 
19:         $r \leftarrow r + w_k \cdot r_k$ 
20:        for  $o \in Z$  do
21:           $w_o \leftarrow w_k \cdot O(s', a_i, o)$ 
22:           $\beta_o \leftarrow \beta_o \cup (p'_{l_k}, w_o)$ 
23:           $\omega(o) \leftarrow \omega(o) + w_o$ 
24:         $o_i \sim \text{Norm}(\omega)$  #  $o_i$  is the sampled observation for agent  $i$ 
25:         $\beta' \leftarrow \text{RESAMPLE}(\beta_{o_i}, \text{size}_{pf})$  # resampling by weights;  $\text{size}_{pf}$  is global
26:         $h' \leftarrow ha_i o_i$ 
27:         $R \leftarrow r + \gamma \cdot \text{UPDATETREE}(\beta', t+1, h', l)$ 
28:         $n_{i,l}(h) \leftarrow n_{i,l}(h) + |\beta_{i,l}|$             #  $|\beta_{i,l}|$ , the weight of the PF, = 1
29:         $n_{i,l}(ha_i) \leftarrow n_{i,l}(ha_i) + |\beta_{i,l}|$ 
30:         $Q_{i,l}(h, a_i) \leftarrow Q_{i,l}(h, a_i) + \frac{R - Q_{i,l}(h, a_i)}{n_{i,l}(ha_i)}$ 
31:    return  $R$ 

```

The following procedure *PLANNEIGHBORS()* gets the estimated optimal action for each neighbor.

- In Line 8, the optimal action for the given belief is given by performing a dot product of the given belief with the pre-computed set of PBVI alpha-vectors to get the argmax action.

```

1: procedure PLANNEIGHBORS( $p_l, l$ ):
2:    $a_{-i}, \bar{\beta}_{-i} \leftarrow \emptyset$ 
3:   for  $j \in -i$  do
4:      $\beta_{j,l-1} \leftarrow \text{GetBelief}(j, p_l, l - 1)$ 
5:     if  $l - 1 \geq 1$  then
6:        $a_j \leftarrow \text{IPOMCP}(\beta_{j,l-1}, l - 1)$ 
7:     else
8:        $a_j \leftarrow \text{GETPBVI ACTION}(\beta_{j,l-1})$  # PBVI offline policy (level-0)
9:      $a_{-i} \leftarrow a_{-i} \cup a_j$ 
10:  return  $a_{-i}$ 

```

The following procedure *NESTEDBELIEFUPDATE()* updates the nested particle of the subject agent. Note that the nested particle holds a belief over the beliefs (i.e., the particle filters) of each modeled neighbor.

- Line 6 samples an observation based on the subject agent's next state.
- This observation is used to update the particle filter of each neighbor in Line 8 using the procedure *MODEL AWARE MONTE CARLO BELIEF UPDATE()*. This "model-aware" belief update procedure assumes that it is aware of the domain's transition and observation functions so that it can pre-compute and cache the possible next states and respective transition probabilities. Such caching is critical since the procedure will be called numerous times (millions) during the simulation, and it would otherwise be inefficient to compute these possible next states and probabilities on the go.


```

1: procedure NESTEDBELIEFUPDATE( $s', p_l, a$ ):
2:    $p'_l(i) \leftarrow \text{belief}(s')$            # creating a particle
3:   for  $j \in -i$  do
4:      $p'_{l-1}(j) \leftarrow \emptyset$ 
5:   for  $j \in -i$  do
6:      $o_j \sim \text{SampleObservation}(s', a)$    # sampling based on observation error
7:      $\beta_{j,l-1} \leftarrow p_{l-1}(j)$          #  $p_{l-1}(j)$  is obtained from  $p_l$ 
8:      $\beta'_{j,l-1} \leftarrow \text{MODEL-AWARE-MONTE-CARLO-BELIEF-UPDATE}(\beta_{j,l-1}, a_j, o_j)$  # level 0
9:      $p'_{l-1}(j) \leftarrow \beta'_{j,l-1}$ 
10:  return  $p'$                                # nested particle

```

The following procedure *RESAMPLE()* returns the resampled particle filter assigning a uniform weight to each newly sampled particle.

- In Line 3 a new weight is computed based on the standard size of the particle filter
- Lines 6 to 13 sample particles from the existing particle filter based on their weights
- Line 10 adds the k^{th} particle whose weight w_k is greater than the random weight.

Line 11 assigns a uniform weight to this sampled particle (so that the number of particles sampled overall will decide the final weight for each state)

```

1: procedure RESAMPLE( $\beta_o, \text{size}_{pf}$ ):           #  $\text{size}_{pf}$  is a global constant
2:    $\beta'_o \leftarrow \emptyset$                      # initializing a new particle filter
3:    $w_{\text{new}} = \frac{1.0}{\text{size}_{pf}}$ 
4:    $\text{particles}_{\text{sampled}} \leftarrow \emptyset$ 
5:    $\text{weights}_{\text{sampled}} \leftarrow \emptyset$ 
6:   for  $i \in \text{size}_{pf}$  do                           # default number of particles
7:      $u \sim \text{Uniform}[0, 1]$ 
8:     for  $(p_k, w_k) \in \beta_o$  do                     # standard roulette wheel selection
9:       if  $w_k \geq u$  then
10:         $\text{particles}_{\text{sampled}} \leftarrow \text{particles}_{\text{sampled}} \cup p_k$ 
11:         $\text{weights}_{\text{sampled}} \leftarrow \text{weights}_{\text{sampled}} \cup w_{\text{new}}$ 
12:        break
13:       else
14:         $u \leftarrow u - w_k$ 
15:    $\beta'_o.\text{particles} \leftarrow \text{particles}_{\text{sampled}}$ 
16:    $\beta'_o.\text{weights} \leftarrow \text{weights}_{\text{sampled}}$ 
17:  return  $\beta'_o$ 

```

The following procedure *CACHEPROBSNEXTSTATE()* is a helper algorithm that caches (i) the possible next states (Line 14), and (ii) their respective probabilities (Line 13), and normalizes them (Line 17). These values are used by the *MONTECARLOBELIEFUPDATE()* algorithm to perform the Bayesian belief update on the particle filter.

```

1: procedure CACHEPROBSNEXTSTATE ( $S, A, \Omega, T, O$ ): # returns global values
2:    $probsNextState \leftarrow [[\Omega_j]][[A_j]][[S]][[S]]$  # default value is 0.0
3:    $possibleNextStates \leftarrow \emptyset$  # default value is an empty set
4:   for  $o_j \in O_j$  do
5:     for  $a_j \in A_j$  do
6:       for  $s \in S$  do
7:          $sum \leftarrow 0.0$ 
8:         for  $s' \in S$  do
9:           for  $C \in \mathcal{C}$  do # marginalizing over the neighbor Configurations
9:              $obsProb \leftarrow obsProb + \Omega(s', a_j, C, o_j)$ 
10:             $transProb \leftarrow transProb + T_i(s, a_j, C, s')$ 
11:             $nextProb \leftarrow obsProb * transProb$ 
12:            if  $nextProb > 0$  then
13:               $probsNextState[o_j][a_j][s][s'] \leftarrow nextProb$ 
14:               $possibleNextStates[o_j][a_j][s] \leftarrow possibleNextStates[o_j][a_j][s] \cup \{s'\}$ 
15:               $sum \leftarrow sum + nextProb$ 
16:            for  $s' \in possibleNextStates[o_j][a_j][s]$  do # normalize to 1.0
17:               $probsNextState[o_j][a_j][s][s'] \leftarrow probsNextState[o_j][a_j][s][s'] / sum$ 
19:   return  $probsNextState, possibleNextStates$  # GLOBAL VALUES

```

The following procedure *MODELWAREMONTECARLOBELIEFUPDATE()* assumes that it is aware of the transition and observation model of the domain, and therefore uses the values cached from *CACHEPROBSNEXTSTATE()* that is shown above.

- This algorithm samples next states for the updated particle filter using the states from the current particle filter until the total probability of the updated particle filter becomes 1.0 (Line 4).

- In Line 5, we sample a random particle, weight from the current particle filter
- In Line 11, we check if the above weight is greater than the randomly sampled weight, and then this particle is added to the updated particle filter.
- Line 16 Normalizes the new particle filter to ensure the weights sum to 1.0

```

1: procedure MODEL AWARE MONTE CARLO BELIEF UPDATE( $\beta_j, a_j, o_j$ ):
2:    $\beta'_j \leftarrow \emptyset$ 
3:    $cumulative \leftarrow 0.0$ 
4:   while  $cumulative < 1.0$  do    # sample until the new PF's total weight = 1.0
5:      $(p, w) \leftarrow Uniform(\beta_j)$   # random uniform sampling
6:      $s \leftarrow p.state$ 
7:      $nextStates \leftarrow possibleNextStates[o_j][a_j][s]$  # global
8:      $u \sim Uniform[0, 1]$ 
9:     for  $s' \in nextStates$  do
10:       $nextProb \leftarrow probsNextState[o_j][a_j][s][s']$  # global
11:      if  $nextProb < u$  then
12:         $w' \leftarrow prob * nextProb$ 
13:         $\beta'_j \leftarrow \beta'_j \cup \{(s', w')\}$ 
14:         $cumulative \leftarrow cumulative + w'$ 
15:      continue
16:    $\beta'_j \leftarrow Normalize(\beta'_j)$  # ensure the probs in  $\beta'_j$  sum to 1.0
17:   return  $\beta'_j$ 

```

The following procedure *GET PBVI ACTION()* retrieves the best action after performing a dot product between the belief (constructed from the particle filter) and the set of alpha-vectors that is the offline solution generated and cached by PBVI.

- Line 2 constructs the belief vector using the particle filter to be used for the dot product computation.
- Line 3 fetches the argmax action after performing the dot product between the belief vector and the set of PBVI alpha-vectors.

```

1: procedure GETPBVIACTION( $\beta_{j,l-1}$ ):
2:    $b(s) = \sum_{k=1}^{|S|} w_k \mid [p_k.state = s]$  where  $(p_k, w_k) \in \beta_{j,l-1}; \forall s \in S$  # belief build
3:    $a^* \leftarrow \operatorname{argmax}_a (b \cdot \alpha_a); \alpha_a \in V_{curr}$  #  $V_{curr}$  is global
4:   return  $a^*$ 

```

Chapter 4 - Implementation Overview

4.1 Introduction

This chapter discusses our implementation of the I-POMCP-PF algorithm for the Cybersecurity Defense domain [Eck, Unpublished]. It gives our readers an overview of the structure of our code base that we adapted from [Eck, Unpublished] and the various technologies that we carefully chose to efficiently implement it and run our experiments. *This walkthrough is intended to help readers who wish to adapt or extend our research by making it easier to understand how different modules fit together and how the reasoning algorithms are integrated into the domain.* For additional context, this chapter also gives the readers a detailed overview of the Cybersecurity Defense domain, and all its essential properties that make up the multi-agent system.

Our codebase inherits the core structure used to simulate the Wildfire Fighting domain that [Kakarlapudi 2021] also uses for their experiments. It also implements the Cybersecurity Defense algorithm [Eck, Unpublished]. This is a thoroughly organized codebase with different modules dedicated to different parts that make up the end-to-end simulation algorithm. These modules can be broadly classified as agent-related, domain-related, planning-related, and simulation-related. Additionally, this codebase also provides with scripts to run and analyze the experiments on a high-level. Building on this, we introduced domain-specific updates to key

functions, incorporated PBVI-based reasoning into the level-0 models of the I-POMDP framework, improved solver efficiency through optimized belief updates, GPU-based acceleration, and Cython integration, and added scripts to support deeper agent behavior analysis in the Cybersecurity Defense domain.

Briefly, the agent-related modules, in Section 4.2, define the properties of each agent, i.e., the frame of the agent, the actions of the agent, etc. Here, the frame of the agent refers to the agent's type or role in the environment, e.g., defender, attacker, ground firefighter, helicopter, etc. The frame module encapsulates the specific functionality and properties tied to the agent type or role—such as the actions it can choose from, the observations it can make, its strength, and any openness parameters, making the frame a self-contained module that governs agent behavior with a given domain (see Figure 4.2). The domain-related modules, in Section 4.3, define the domain-specific properties that shape the two implemented domains in the original codebase, i.e., Wildfire Fighting, and Cybersecurity Defense. These domain-specific properties include the environment's state space, observation space, the reward structure, the state transition probabilities, etc. In addition to these, each domain-related module also extends the respective agent, planning, and simulation-related modules so that it can define specific properties, and settings, associated with that specific domain. For example, there could be different frames associated to a Cybersecurity Defense domain agent (e.g., attackers and defenders) compared to a Wildfire Fighting domain agent (e.g., ground-based firefighters and helicopters), and the Cybersecurity Defense domain might want to initialize its reasoning models with a set of parameters (e.g. the reward structure) that could be different from what the Wildfire Fighting domain could use. The reasoning module, in Section 4.4, essentially discusses the implementation of the planning algorithms like I-POMCP-PF, POMCP-PF, and PBVI, and other naïve reasoning algorithms that are initialized using the domain-related modules. The

simulation-related modules, in Section 4.5, define properties of the simulation, like the number of time-steps, transitions between the time-steps, generating the output files, etc.

Batches of runs are started using the run-scripts, described in Section 4.6, that define various parameters for each run, for example, the number of runs, and the run-time efficiency's profiling parameters. Each run of the simulation generates output files which print the states, observations, rewards, and actions of each agent, for each time step. These output files for all the different runs with different seeds are processed to compute an accurate average. These results are further analyzed using more comprehensive scripts that look for patterns in agents' actions, explored states, etc.

In a nutshell, all these modules work together to execute the simulation. The domain module sets up the reasoning methods (solvers) with the state, action, and observation spaces, and the reward structure, and agent configurations. The reasoning module then uses the agent module to access frame-specific properties, such as the available actions, observations, and strengths of each agent. At each time step, and for each agent, the simulation module calls the reasoning module to fetch the agent's chosen action and uses these actions to update environment state. The run-scripts initiate this process by setting the parameters for the simulation, defining the number of runs, number of time-steps, and the reasoning methods and their respective configurations for each agent. This overall interaction between the different modules is shown in Figure 4.5 below.

4.2 Agents

The Agents module has two submodules, i.e., Agent, and Frame, whose functionality is described below. The Frame submodule extends the Agent submodule and helps each agent implement their functionalities in a frame-specific manner. On a high-level, these submodules

are implemented such that they are independent of the domain. This makes it easier for us to extend them and adapt to a specific domain like the Cybersecurity Defense where the functionalities can be tailored accordingly.

The Agent submodule has two key functions, i.e., choose-action, and make-observation. The choose-action function helps the agent choose an available action from the domain given the frame under which this agent is operating. For instance, a Cybersecurity Defense defender agent (operating under the Defender frame) can only choose from a set of actions that are specific to its domain (i.e., Cybersecurity Defense) and frame (Defender), such as {Patch, Move to Subnetwork 1, Move to Subnetwork 2, Move to Home, Monitor}, while a Cybersecurity Defense attacker agent belonging to a different frame (i.e., Attacker) can only choose from {Attack Subnetwork 1, Attack Subnetwork 2, Attack Subnetwork 3, NOOP}, which is the set of actions that are available to that frame, and so on. The make-observation function enables each agent to receive an observation and in-turn helps them to maintain a belief of the current state of the environment. Just like the set of actions, the set of observations that an agent might receive also depends on their frame. For example, a ground-based firefighting agent of the Wildfire Fighting domain could receive a different kind of observation of the environment compared to an airborne wildfire fighting agent, that belongs to a different frame and has a better estimate of the environment's state given their aerial top-down view.

While the Frame submodule works with the Agent submodule to customize its functionality as discussed above, it also defines and holds various properties of its own for each agent that correspond to a specific frame. Since these submodules are defined on a high-level, independent of the domain, this allows a specific domain to extend these submodules and implement its own functionality. An example of this would be the properties that define the strength of the attacker and the defender agents in the Cybersecurity Defense domain. This

“strength” parameter of the Cybersecurity Defender frame agent determines the likelihood of a successful “Patch” action, i.e., higher strength corresponds to a higher transition probability for the subnetwork to transition to a healthier security state when the agent takes this action. These strengths are an example of frame-specific properties of these agents and are defined in their respective Frame submodule. Similarly, the level of agent openness is also a frame-based property and is defined in the same Frame submodule. The following Figure 4.2 represents the Agents module, and how its functionality and properties are organized.

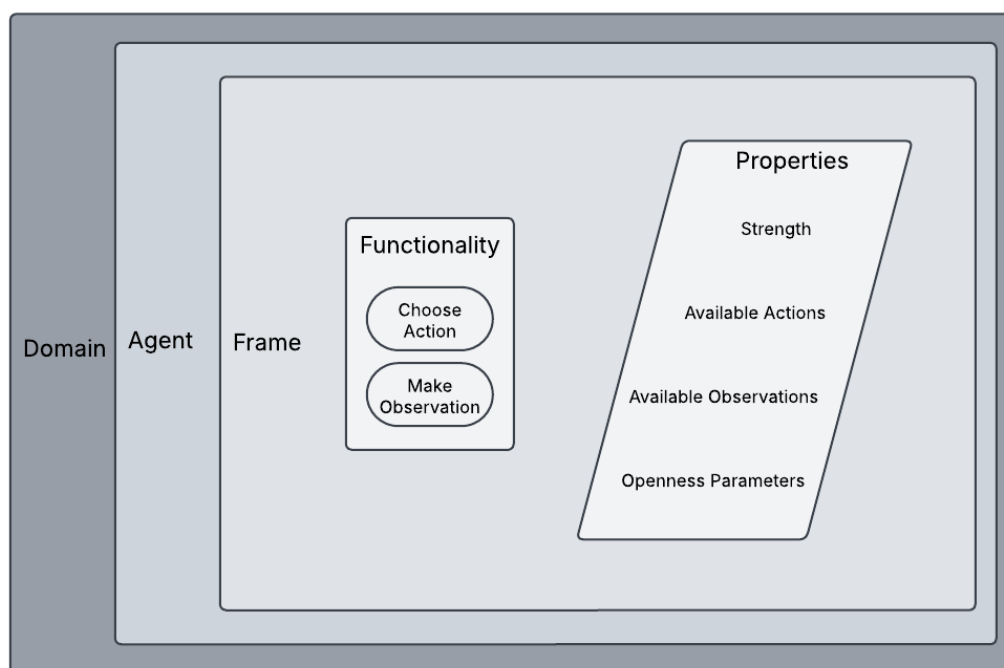


Figure 4.2 Agents module depicts the hierarchy between Domain, Agent, Frame, and specified frame-specific properties and functionalities

4.3 Domains

The domain module is responsible for implementing several key functionalities. It sets up both the framework for a specific domain as well as the properties that help simulate it. Additionally, it also helps in initializing the reasoning planners that fetch optimal actions for each agent during the simulation.

To setup the domain's framework, the domain module defines the state, action, and the observation spaces, the transition, and reward functions. Additionally, it is also responsible for generating the agents and the frame space. We have implemented our code such that any specific domain, like the Cybersecurity Defense, or the Wildfire Fighting, can extend this domain module and override these functionalities to suit the respective domain's needs. Also, for a clean implementation, for all our domains, the state, action, and the observation spaces are all defined using lists of integers where each state, action, and observation has a unique identifying index that correlates with the said integer. More information on how these indices are defined for the Cybersecurity Defense domain is given later in Section 4.7

In addition to setting up each domain's framework, the domain module also implements the functionalities that come in handy to simulate the environment. These functionalities help us generate start states, sample next states, sample observations, sample multi-agent configurations, cache the maximum and minimum possible rewards for the domain, and so on.

Finally, the domain module also helps us setup and initialize our reasoning algorithms (or solvers), both intelligent, and naïve. The intelligent reasoning algorithms include the level-0 PBVI, POMCP-PF, and the level-1 I-POMCP-PF planners. For the intelligent reasoning algorithms since they are more complex to implement, the domain module just initializes these planners for the specific domain that we are interested in and passes-in the respective attributes of the domain, like the state, action, and observation spaces, the reward functions, the frame and number of respective agents per frame, and so on. These initializations help us organize the codebase better and reduce the boilerplate code required to implement these algorithms. For the relatively simpler naïve reasoning algorithms, the domain module completely sets up and maintains their logic for the specific domain they are designed for. For the Cybersecurity Defense domain, for example, we have as the naïve reasoning algorithms: (1) the Random

Reasoning where each agent chooses only random actions from the set of possible actions; (2) the Heuristic Fixed Defender Reasoning, and (3) the Heuristic Spread Defender Reasoning where the defender agents naively prioritize patching over the other possible actions. The following Figure 4.3 shows us structure and functionality within a domain module.

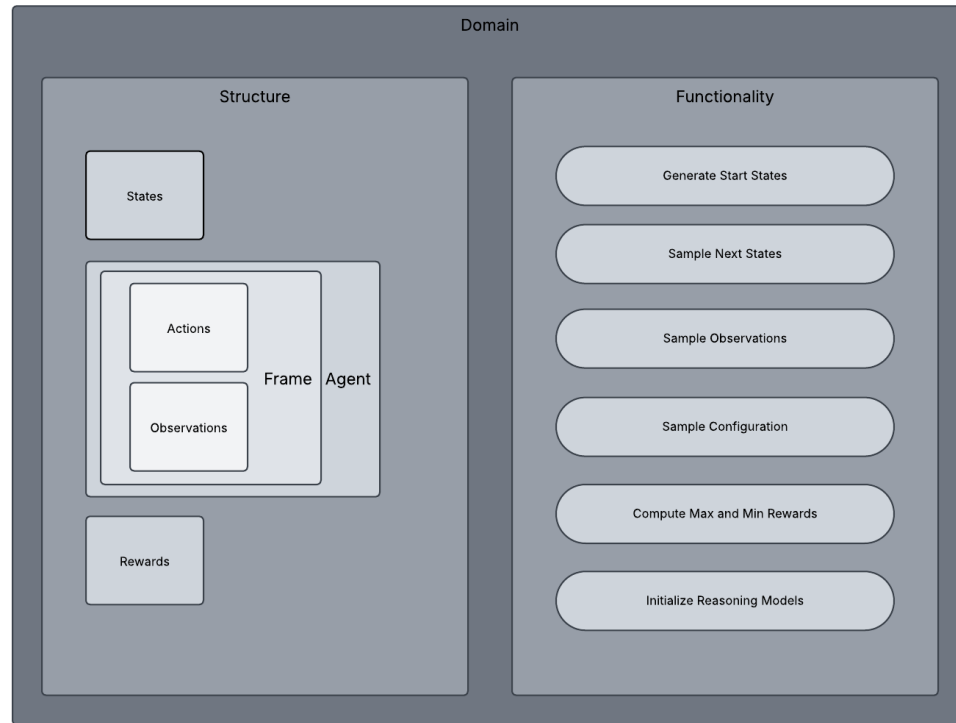


Figure 4.3 Domain module depicts the structure and functionality within a specific domain

4.4 Reasoning Module

As we just discussed above, we have two kinds of reasoning algorithms, i.e., naïve, and intelligent. The naïve reasoning algorithms include Random reasoning, Heuristic Fixed Defender, and Heuristic Spread Defender where the last two are unique to the Cybersecurity domain. These are relatively simple to implement are already implemented in the domain module discussed above. The intelligent reasoning algorithms, i.e., I-POMCP-PF, POMCP-PF, and

MOMDP-PBVI, on the other hand, require data structures of their own to be defined, for example, the particle filters, and are implemented as stand-alone modules that work with the domain-related and other modules to simulate the decision-making of each agent. They are still, however, initialized using the domain-specific parameters in their respective domain-based modules as discussed above.

I-POMCP-PF, and POMCP-PF algorithms are implemented in their standalone modules. Their fundamental functionality is to accept the current state or belief as the input, plan for the optimal action, and return this action to the simulator module. On top of this, we also note that both these modules use custom data structures that are not defined in the domain modules. For POMCP-PF, we require data structures to represent a particle filter, and a particle that forms the particle filter; which are respectively used to represent a belief, and a state of the environment. I-POMCP-PF, unlike POMCP-PF, requires a nested particle which has the ability to hold a particle filter separately for each of the modeled I-POMCP-PF level-1 neighbors. In addition to these data structures, these modules also need functionalities to process them. For example, throughout the planning process, we constantly need to update the particle filter to propagate the belief and then resample the updated particle filter. Additionally, we also need to deal with issues like particle deprivation. Our planners are well-equipped to deal with all such functionalities and we implement a ‘structures’ submodule to deal with the data structures and their updates, which are generalized for all our planners. Finally, it is important that the level-1 planners are able to work well and integrate with their level-0 counter parts, and our implementation ensures that our level-1 I-POMCP-PF module integrates well with both the level-0 planners, i.e., MOMDP-PBVI, and POMCP-PF, as required.

MOMDP-PBVI, like the above two, is also implemented as a standalone module but has two independent submodules. While one of the submodules is used to generate the offline raw

policies, the other submodule is used to integrate the policies with the domain modules and simulate experiments which use these offline policies. For example, to integrate offline MOMDP-PBVI policies into the level-1 I-POMCP-PF planning, we first generate the MOMDP-PBVI policies and use the output files to feed into the MOMDP-PBVI simulator submodule which in-turn integrates with the domain, and the level-1 I-POMCP-PF module to deliver the level-0 optimal action. Apart from being used with the level-1 planner, this MOMDP-PBVI submodule can also be used to simulate standalone MOMDP-PBVI based experiments. Coming to the data structures that are required for MOMDP-PBVI, in addition to the particle filters and the particles used to represent the beliefs and states of the environment, MOMDP-PBVI also requires alpha-vectors to cache and re-use the value-function when choosing an optimal action. The data structures for these alpha-vectors are implemented also in the structures-submodule, alongside the particle filters, particles, and their functionalities that we discussed above. The following Figure 4.4 represents a high-level organization of different reasoning algorithms including the naïve baselines as well as the intelligent reasoning algorithms.

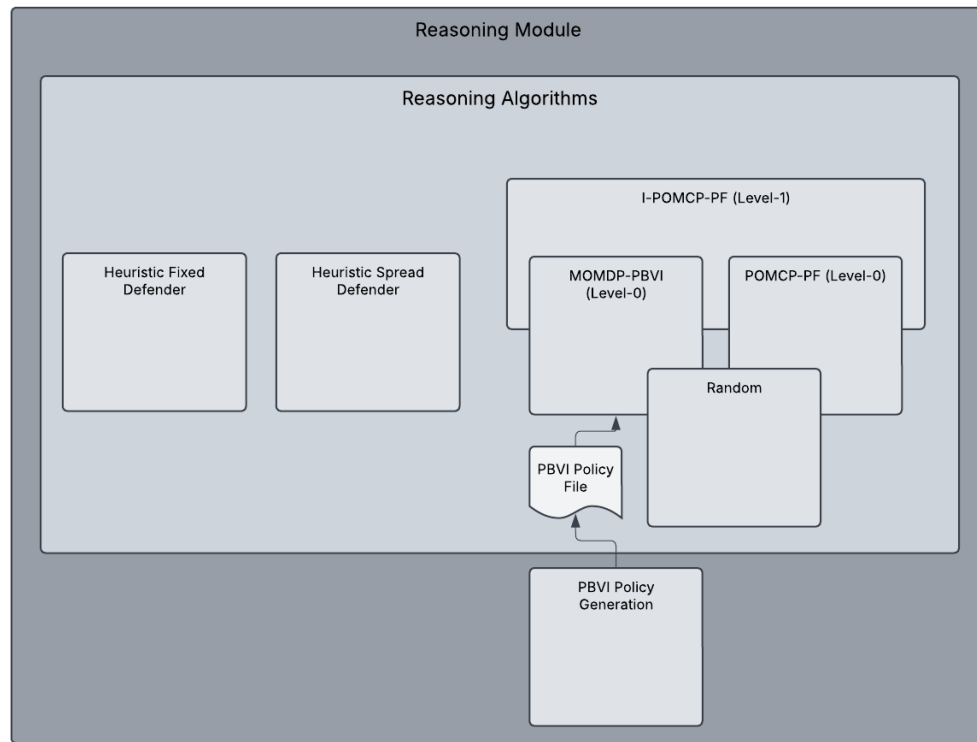


Figure 4.4 Reasoning module depicts the different reasoning algorithms, and the relation between the level-1 and level-0 reasoning algorithms.

4.5 Simulation

The simulation module organizes the end-to-end flow of the simulation. It is triggered using the run scripts which define the necessary parameters for our experiments, and then simulates the domain of our choice, and finally generates domain-specific output files that can be processed to get a summary and analysis of the experiments. Just like the above Agent, Domain, and other modules, the Simulation module is also implemented independent of the domain and can be extended to work for any specific domain.

The simulation module fundamentally handles 1) the number of runs, 2) the number of time steps, 3) the agent reasoning types, and 4) the reasoning parameters (e.g., for PBVI, we

have the number of iterations and the belief point cap per iteration; for POMCP-PF and I-POMCP-PF, we define the number of planning trajectories, number of particles, and general planning parameters like the planning horizon, discount factor, etc.), all of which are defined in the run scripts.

- The number of runs determines how many times the same experiment is repeated, each time with a different seed, to give us a generalized result that represents an accurate average of the rewards, states explored, and so on. In our case, each run of our Cybersecurity Defense experiments is seeded such that the start state is different each time. Our summary-generating scripts automatically detect the number of runs using the number of files and generate relevant output.
- The simulation module also accepts the number of time-steps required, as a parameter, and is designed to update the state, action, and observations based on that. This is a standard implementation for simulation-based multi-agent Markov-Decision Process experiments.
- Next, the simulation module is responsible for setting up the simulation based on the agent reasoning types. It first adapts to the designated domain and then sets up the agents and their planners accordingly, with respect to the reasoning types of each agent. For example, the simulation module that extends the Cybersecurity Defense domain generates two frames of agents, i.e., the attackers and the defenders. Based on the settings in the run script, each of these types of agents can have a different reasoning type (the attacker agents are usually Random-reasoning agents, and the defender agents use either the baselines or the intelligent reasoning models), and the simulation module is responsible to initialize the respective reasoning model for each agent.

- In this process, the simulation module also accepts the reasoning parameters (e.g., number of particles, number of trajectories, type of level-0 reasoning to be used for model neighbors in level-1, and so on) from the run scripts and uses them to initialize these reasoning models.

The output files are also generated within the simulation module. The definitions necessary to generate these output files, such as the names of the files, the path where we store the files, the headers, the information that needs to be stored within each file, are all defined in the simulation module, and this is generalized to work with different domains, and different reasoning models (because different reasoning models can have files with different names, and so on).

The simulation module is represented using Figure 4.5 which shows us the end-to-end flow of a typical simulation.

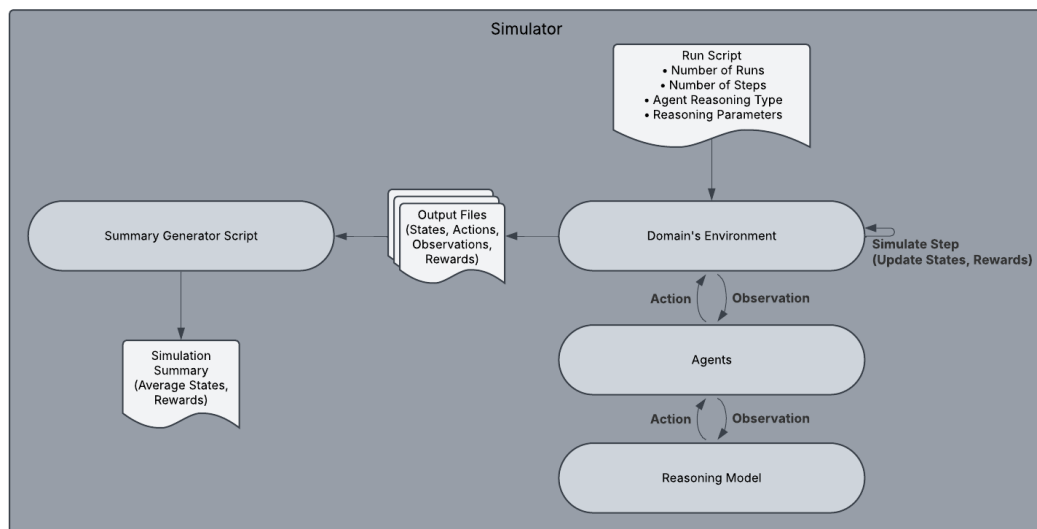


Figure 4.5 Simulation module depicts the overall flow of the simulation and file generation

4.6 Technology Details

This section discusses the technology stack that we have used for our implementation, and the infrastructure that helped us run our experiments. We primarily used Cython to implement our algorithm logic, a Python library called TensorFlow for hardware acceleration, R to analyze our results, and heavy work-load CPU and GPU computing resources from the super computers at University of Nebraska-Lincoln’s Holland Computing Center (UNL HCC), and Oberlin College and Conservatory.

Cython (C + Python) is a powerful language which combines the speed of C, with the ease of programmability and even the machine learning capabilities of Python. Cython lets us introduce C’s static data typing into our code and eliminates Python’s inherent run-time overhead introduced by its dynamic data typing, thereby improving the execution speed.

Our implementation is optimized for best efficiency practices as well as hardware acceleration.

First, to implement best efficiency practices, *we took advantage of Cython to mitigate critical bottlenecks that are bound to arise given the nature of the planning algorithms that we use*. For instance, PBVI value iteration happens at the heart of several nested loops (one for each of the set of states, actions, next states, observations, configurations, and beliefs), and even a simple operation such as a summation might be called millions, if not, billions of times. It is in such places we leverage Cython’s performance capabilities, by introducing C-based static data typed syntax to perform any computations within these nested loops. Furthermore, we carefully identified areas where we can avoid such redundant computations, e.g., in an instance where the transition probability for the next state is 0.0, we can skip the subsequent loops of computations for computing the rewards and observations, thereby saving computation time.

To help with identifying all such areas of code, our codebase is integrated with Cython’s profiling feature which lets us analyze the individual run-time allocated to every callable function of our code and generates HTML files for each of our modules highlighting the lines of code which are computationally intensive. These optimizations helped us improve the run-times by approximately 3x.

Second, for hardware acceleration, *we used the Python library TensorFlow to significantly optimize the code further on top of the above-mentioned improvements.* Since implementing this is non-trivial for all working components of the codebase due to the nature of the algorithm that operates through sequential *for* loops for the most part, we applied hardware acceleration to the most problematic area of the level-1 algorithm. This is where the level-1 subject agent computes the level-0 policy for each neighbor using the PBVI reasoning model. This is essentially computing an argmax action using the dot product between the belief vector of each neighbor against the set of alpha-vectors that represent the value function for each neighbor. TensorFlow effectively lets us make this computation more efficient by leveraging the power of a GPU which lets us parallelize all these dot products and then extract the argmax with $O(1)$ time complexity.

Given the complexity of Cybersecurity Defense domain, our experiments are computationally intensive (requiring tens of gigabytes of memory, and several days of run-time), making it impractical to run them on local machines. *We therefore made use of the supercomputer clusters available at UNL HCC, and at Oberlin College and Conservatory.* Our experiments are submitted as jobs to these clusters using a Slurm script. Slurm is a resource management system that Linux-based High-performance Computing (HPC) clusters like HCC use, and it allows us to submit batch jobs with predefined requirements like the memory needed, number of CPUs, GPUs, estimated run-time, and so on. One other advantage of these HPC apart

from the high performance is that they let us parallelize our jobs, i.e., we can trigger different runs with different seeds all using a single Slurm script to submit a batch job.

Finally, we use an R script to analyze the results and generate a summary. Our set of output files consists of two types of files, 1) the states file, and 2) action-observation-reward file, and the format of these files, as mentioned in the Simulation section, is generalized for different domains. Our R and Python scripts, based on the number of agents, number of time steps, and number of runs, is designed to process these files, and generate a summary that is specific to the domain. Note that although we have generalized output files, each domain still needs a separate R script because our criteria to evaluate the experiments can vary drastically from domain to domain. For example, while for the Cybersecurity Defense domain, we want to look at attributes like Attacker Rewards, Defender Rewards, Subnetwork Status, Attacker Presence, Subnetwork Health, etc., the Wildfire Fighting domain requires us to look at a different set of attributes like Suppressant Level, State of Fires, and so on). Given this, the domain-specific scripts generate a summary that generalizes the rewards, and other custom domain-specific parameters, across all runs, for us to help analyze the experiment results effectively.

4.7 Cybersecurity Defense Domain

The *Cybersecurity Defense* domain [Eck, unpublished] is a multi-agent multi-frame domain with agent openness and an environment with both fully and partially observable components. There are two types of agents: Attacker agents who try to damage a connected network of nodes or subnetworks (analogous to a computer server's nodes, the Internet, intranets, etc.), and Defender agents who try to defend and repair it. The presence of each agent in the system is stochastic. Each agent can only partially observe the external states of the environment (e.g., the security state of a subnetwork), but it has full knowledge of its own internal state (e.g., its

location in the network). Since an agent can only partially observe the external state space, it can only have a belief over what the actual external state is, in each time step.

4.7.1. States

The Cybersecurity Defense domain's state space comprises of (a) the external states that represent the state of the environment, i.e., the health (security state) of each subnetwork, and (b) the internal states that represent information that is internal to each agent, i.e., the location of each agent. In this domain, each subnetwork can have five possible security states (the security states of all the subnetworks together makeup a factored external state), i.e., $\{patched, vulnerable, exploited-1, exploited-2, exploited-3\}$, and each agent can have $n + 2$ internal states, where n is the number of subnetworks. The internal states of the defender agent are given by $\{loc_1, loc_1, \dots, loc_n, loc_{n+1}, loc_{n+2}\}$. Here, loc_1 through loc_n represent the locations of n subnetworks; loc_n is the *home node* that is analogous to a base location where the defender agents start from, or come to when they were outside the environment in the previous timesteps; loc_{n+1} is *out-of-network* which means that the defender agent is out of the environment and is not available to take any actions (taking no action is equivalent to choosing a *noop*, short for no-operation). The attacker agent always has only two internal states, i.e., $\{in-network, out-of-network\}$.

4.7.2. Actions

The sets of actions for the defender agents, and the attacker agents, respectively, with n subnetworks involved, are: $A_{def} = \{move_1, \dots, move_n, patch_1, \dots, patch_n, noop, move_{home}, monitor\}$ and $A_{att} = \{noop, attack_1, \dots, attack_n\}$. Here, the $move_n$, and the $move_{home}$ actions represent the defender agent choosing to move to a specific subnetwork n , or the home node, respectively; $patch_n$ represents the defender agent choosing to patch (repair) a specific

subnetwork n ; *monitor* is chosen by the defender when they simply want to make an observation without targeting a change in the state-space, and the nature of the observation depends on the location from which the defender agent is choosing this action. We note that, in our current implementation, the defender agent receives an observation for choosing any action other than the *noop* action—including actions other than the *monitor* action. However, *monitor* can still be considered special because it allows the defender agent to receive an observation without having to change its current location or without choosing to patch a specific subnetwork which can have a patch cost. Additionally, choosing an action from a subnetwork gives the defender agent an observation of the security state of only that specific subnetwork with a low observation error. Choosing an action from the home node gives the defender agent an observation of the security states of all the subnetworks but this is more erroneous compared to the observation that the defender agent receives when choose an action from a subnetwork. The attacker agents receive an observation of the security state of the subnetwork they chose to attack, by default unless they choose a *noop*—they receive it with the same observation error as the defender agent choosing an action from the home node.

Each agent at a given time has access to only a subset of the actions and this is based on their internal state. Both the defender agents and the attacker agents can only choose *noop* when *out-of-network*. When *in-network*, the actions available for a defender agent are: (1) *patch* the subnetwork where currently located at, (2) *move* to any other subnetwork, or *home*, and (3) *monitor* (a) the subnetwork currently located at, or (b) all the subnetworks when located at *home*. When *in-network*, the actions available for an attacker agent are to attack any of the n subnetworks, and unlike the defender agents, the attacker agents can also choose a *noop* even when *in-network*.

4.7.3. State Transition Function

The transition function for the Cybersecurity Defense domain, $T(es, is, a, C, es', is')$ is defined as $T = T_{SUB} \times T_{Def} \times T_{Att}$. Here T_{SUB} represents the external state transition of all the environment's subnetworks; T_{Def} and T_{Att} represent the internal state transitions of the defender agent, and the attacker agent, respectively. The transition function gives the transition probabilities of the next external state es' , and the next internal state is' , given the current external state es , the current internal state is , the subject agent's action a , and the joint action (or Configuration) C of the neighbors.

4.7.3.1 Internal State Transition Probabilities

The internal state transition probabilities of the Cybersecurity Defense domain are primarily associated with the change of location of the agent. Incorporating agent openness, two additional factors that help define the internal state transitions are *persistence*, and *return-rate*. *Persistence* $[0,1]$ is the probability with which each agent stays inside the environment, i.e., *in-network*. *Return rate* $[0,1]$ is the probability with which each agent returns to the environment when it is outside, i.e., *out-of-network*. Since there are only two possibilities, if the agent is inside the environment with a probability of *persistence*, its likelihood to be outside will be $1 - \text{persistence}$, and similarly for return rate. Also, while the definitions of persistence and return rate remain the same for both the attackers and the defenders, agent openness works slightly differently for them. That is, when the attackers go outside the environment, they directly return to inside the environment and can readily continue attacking the subnetworks as they were before. For the defenders, however, regardless of where the defender agent was when it was pulled out of the environment, it always returns to the home node, and then must move to the subnetworks before they can start patching them.

The following Table 4.1.a, and Table 4.1.b depict the internal state transition probabilities of the defender agent, and the attacker agent, respectively, from the current internal state is to the next internal state is' , given an action a .

is	a	is'	$p(is')$ for defender
<i>out-of-network</i>	<i>any</i>	<i>home-node</i>	<i>return rate</i>
		<i>out-of-network</i>	<i>1 - return rate</i>
<i>in-network</i>	<i>move_home</i>	<i>home-node</i>	<i>persistence</i>
		<i>out-of-network</i>	<i>1 - persistence</i>
<i>in-network</i>	<i>move_n</i>	<i>subnetwork</i>	<i>persistence</i>
		<i>out-of-network</i>	<i>1 - persistence</i>
<i>in-network</i>	Actions other than <i>move</i>	<i>no change</i>	<i>persistence</i>
		<i>out-of-network</i>	<i>1 - persistence</i>

Table 4.1.a Internal state transition probabilities for the defender agent

is	a	is'	$p(is')$ for attacker
<i>out-of-network</i>	<i>any</i>	<i>in-network</i>	<i>return rate</i>
		<i>out-of-network</i>	<i>1 - return rate</i>
<i>in-network</i>	<i>any</i>	<i>in-network</i>	<i>persistence</i>
		<i>out-of-network</i>	<i>1 - persistence</i>

Table 4.1.b Internal state transition probabilities for the attacker agent

4.7.3.2 External (Security) State Transition Probabilities

In the Cybersecurity Defense domain, the external state transitions are the result of transitions of the security (akin to “health status”) states of all the subnetworks. We assume that the security state transitions of the subnetworks are independent of one another. The security state transition function is given by

$$T_{Net} = \prod_{i=1}^n T_{sub_i},$$

where T_{Net} represents the transition function of the whole network, and T_{sub_i} represents the transition function of subnetwork i .

Given that there are five security states for each subnetwork: *{patched, vulnerable, exploited -1, exploited -2, exploited -3}*, the transition probability between two security states of each subnetwork depends on the following: (1) configuration of the actions of the attackers; (2) configuration of the actions of the defenders; (3) current security state of the subnetwork; and (4) the strengths of attacker agents and defender agents. Additionally, there is always a non-zero residual threat (chip damage) as long as at least one attacker agent is attacking a subnetwork.

For a subnetwork i , we compute the transition probabilities as follows. This is nothing but a logistic sigmoid function of the net impact coming from both the attacker agents and the defender agents.

- We first compute the threat from the attacker agents and the mitigation from the defender agents.
 - $attacker_threat = \text{number of attacker agents choosing } attack_i * attacker_strength$
 - $defender_mitigation = \text{number of defender agents choosing } patch_i * defender_strength$
 - $residual_threat = residual_threat_baseline \times (attacker_threat) / (attacker_threat + defender_mitigation)$

- $residual_threat = residual_threat / (residual_threat + 1.0) \#$
normalized
- $net\ impact = defender\ mitigation - attacker\ threat$
- Using the above, we compute $prob_{better}$ and $prob_{worse}$ that determine if the subnetwork's health will transition to a better security state, or a worse security state, respectively (as seen in Figure 4.6).
 - $sigmoid_{def} = exp(net\ impact) / (1 + exp(net\ impact))$
 - If $sigmoid_{def} > 0.5$ then $prob_{better} = 2 * (sigmoid_{def} - 0.5)$ else $prob_{better} = 0.0$
 - $sigmoid_{att} = 1 - sigmoid_{def}$
 - If $sigmoid_{att} > 0.5$ then $prob_{worse} = 2 * (sigmoid_{att} - 0.5)$ else $prob_{worse} = 0.0$
 - $prob_{better} = (1 - residual_{threat}) * prob_{worse}$
 - $prob_{worse} = residual_{threat} + (1 - residual_{threat}) * prob_{worse}$

Given the above, the transition function for the Cybersecurity Defense domain can be represented as a Markov chain as shown in Figure 4.6

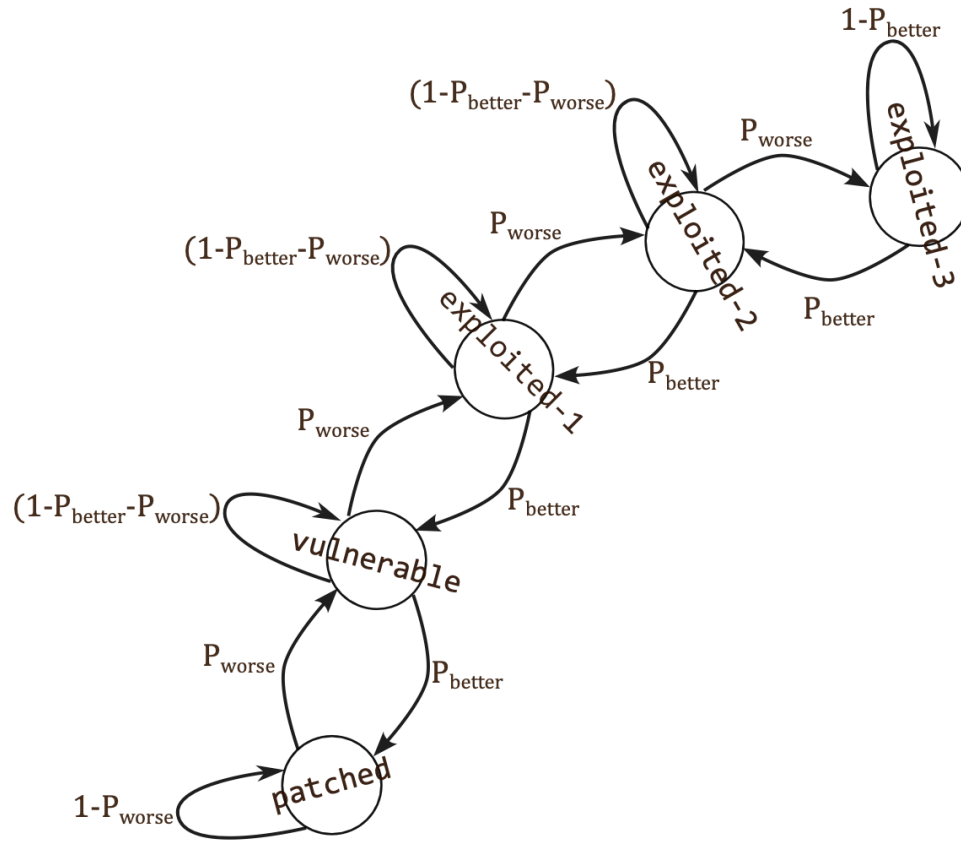


Figure 4.6 Markov chain depicting the transition function of the Cybersecurity Defense domain.

4.7.4. Observation Function

The general observation function for the Cybersecurity Defense domain, $\Omega(o, is, s', a)$, gives the probability of an agent observing o , given the agent had taken an action a , from a subnetwork location is (internal state), and the environment has reached a state s' .

As mentioned in section 4.7.2, the type of the observation received by an agent depends on their action. Additionally, the defender agent receives observations with varying levels of noise depending on their location in the network. The following Table 4.2 depicts the

different types of observations received by the attacker agent and the defender agent that are based on their action and location.

Actions	Observation Received
Defender – NOOP	Does not receive observation
Attacker – NOOP	Does not receive observation
Defender – Other than NOOP	When at home - receive observation with equal error z corresponding to all the subnetworks When at subnetwork - receive observation with error x for local subnetwork and error y for other subnetworks
Attacker – Other than NOOP	Receive observation with error z corresponding to all the subnetworks

Table 4.2 Type of observation received by an agent depending on their action

4.7.5. Reward Function

The reward function $R(s, a)$ in the Cybersecurity Defense domain is a function of the current state, and the action of the agent.

The structure of the rewards is straightforward. Each security state of a subnetwork has a predefined reward associated with it and the total reward each agent receives is the cumulative reward that comes from all the subnetworks, along with any patch cost associated with the defender agent's patch action. The attacker agents do not have a penalty associated with their actions.

For the five security states *{Patched, Vulnerable, Exploited-1, Exploited-2, and Exploited-3}*, the respective rewards are {9.0, 2.0, 0.0, -2.0, -9.0}; and the default patch cost is -0.1 which is accounted for whenever the defender agent chooses a patch action, irrespective of what subnetwork the defender agent chooses to patch.

Chapter 5 – Investigations and Results

5.1. Introduction

In this chapter, we discuss the multi-agent planning-based investigations we conducted in the Cybersecurity Defense domain. We first give an overview of the research questions that this thesis aims to investigate in Section 5.2. We then present our experiment-setup corresponding to each of these scenarios in Section 5.3. Next, we discuss in-depth each of our scenarios and investigate the results in Section 5.4 and finally give a summary of our discussions in Section 5.5.

5.2. Investigations

Our investigations can be broadly classified into three categories: (i) baselines, (ii) agent openness-related (on top of baselines), (iii) and high-observation-error-related (on top of baselines + agent openness). They are discussed in-depth in the below sections 5.2.1, 5.2.2, and 5.2.3, respectively.

Each of the above-mentioned categories corresponds to the real-world-like problems discussed in Chapter 1 and take us closer to modeling the MAS to address those concerns. The baselines exist to give us insights into the default agent behavior in the Cybersecurity Defense domain – a domain that models different agent types (i.e., cooperative, and competitive), in an environment that has both fully and partially observable components. These baselines in-turn

helps us understand more complex conditions involving agent openness and a high observation error. The agent openness scenarios model the complexity of uncertainty in the presence of each agent, challenging the coordination abilities of the level-1 I-POMCP-PF defender agent by forcing it to reason about the presence of its teammates. Introducing high observation error on top of agent openness amplifies these challenges (sort of like a stress test) by significantly increasing uncertainty in observations and testing how effectively the level-1 team of defender agents can adapt their behavior accordingly.

5.2.1. Baselines

The purpose of these baseline-investigations is to understand the agent behavior in the Cybersecurity Defense domain without agent openness or a high observation error. These investigations should not only give us an insight into the intricacies of the Cybersecurity Defense domain's mechanics but also will help us understand better, the more complex scenarios, i.e., the agent-openness, and the high observation error, that we investigate subsequently.

Cybersecurity Defense, even without agent openness and a high observation error, is already a complex domain which mimics many real-world challenges discussed in Chapter 1. That is, the defender agents need to carefully reason against the attacker agents who compete with them for the rewards.

Forming our baselines, we investigate such a complex domain with a total of 6 different defender reasoning algorithms given in Section 5.3.3 – including naïve approaches such as Random Reasoning, Heuristic Fixed Defender Reasoning, and Heuristic Spread Defender Reasoning (all level-0), and intelligent strategies such as PBVI (level-0), POMCPPF (level-0), and I-

POMCP-PF (level-1). All our experiments are conducted using 3 defender agents and 2 attacker agents (please see below Table 5.5).

For the level-1 I-POMCP-PF reasoning, we employ a hybrid defender team structure consisting of one level-1 I-POMCP-PF defender agent and two level-0 PBVI defender agents. motivated by our pilot investigations which showed that such a configuration achieved higher rewards and exhibited more coordinated and interpretable behavior in comparison with a homogenous I-POMCP-PF approach. The homogeneous I-POMCP-PF approach faced coordination challenges as each agent modeled its neighbors as level-0 PBVI defenders, even though those neighbors were themselves reasoning at level-1, ultimately resulting in lack of coordination.

Simultaneously, as a good first step for now, we limit the scope of the attacker agents to just using the heuristic reasoning that attacks the weakest (i.e., most exploited) subnetworks (this is a level-0 reasoning algorithm where the attacker agents don't model any other agent in the MAS).

The following research questions establish how hybrid level-1 reasoning affects the defender team's behavior under our baseline setup. We examine both rewards and coordination behavior, along with how defender strength influences patching decisions in environments with no agent openness.

- **Research Question 1: Does the level-1 hybrid defender team bring better rewards than the homogenous level-0 defender team?**
- **Research Question 2: How does the level-1 I-POMCP-PF defender agent coordinate with its level-0 neighbors during the simulation?** For example, how is their workload

distributed across scenarios, i.e., do the defender agents move towards each other?

How frequently do joint patch actions occur, and how are these behaviors influenced by the exploitation levels of individual subnetworks?

- How is this behavior different compared to the homogenous level-0 reasoning methods where the defender agents do not model their neighbors?

Building on the above, we further investigate to see how the agents of different frames—based on their strengths—behave differently across similar scenarios. In other words, we investigate the above across different setups with *different attacker-defender strengths*, i.e., the weak defender, the neutral defender, and the strong defender scenarios. For context, the strength of an agent here refers to the likelihood of the desired change they can bring to the security state of a subnetwork by taking the respective action, i.e., *patch* action for the defender agent, or *attack* action for the attacker agent. For this setup, the weak defender agents get slightly lesser rewards than the neutral defender agents, and the neutral defender agents get slightly lesser rewards than the strong defender agents. The actual strength values are given in Table 5.2. We note that for the hybrid level-1 hybrid defender team, similar to the other homogenous teams, all the defender agents have the same strength.

- **Research Question 3: How would the level-1 hybrid I-POMCP-PF defender team – with all agents (both I-POMCP-PF and PBVI) operate with the same strength setting – respond across different scenarios as we vary that common defender strength?** For example, in a scenario where all the subnetworks are patched, would the strong defender still care to patch, or choose a random action because it thinks that the subnetwork is unlikely to be exploited in the next time step? Similarly, would a weak

defender agent attempt to patch a fully exploited subnetwork, or would it simply give up and choose a random action after reasoning that it is too weak?

- In turn, would the level-0 defender agent behave any differently compared to the level-1 I-POMCP-PF defender agent?
- Would using level-1 hybrid reasoning perform differently with different defender agent strengths?

5.2.2. Agent Openness

Agent openness brings in a new set of challenges for the defender agents, and therefore the purpose of this investigation is to see how they adapt to different levels of agent openness, i.e., low agent openness and high agent openness, compared to the baseline scenarios where there is no agent openness.

Agent openness in the Cybersecurity Defense domain is defined as the rate at which the agents join or leave the environment. It is defined for each agent, including both the attackers and the defenders, using two attributes, Persistence, and Return Rate. Persistence is the likelihood of each agent staying within the environment in the next time step when it is currently inside, and Return Rate is the likelihood of each agent returning to the environment in the next time step when it is currently outside. For simplicity, we ascribe the same level of openness to all the agents in the system for a given scenario, including both the attackers and the defenders. Low agent openness refers to a setting where each agent in the domain has a high value for Persistence and a low value for Return Rate. In other words, the rate at which each agent leaves or rejoins the environment will be low. High agent openness, on the other hand, refers to a setting where each agent in the domain has a low value for Persistence, and a

high value for the Return Rate. This essentially means that each agent is highly likely to change their presence between being inside and outside the environment. The mathematical model for low and high levels of agent openness is given in Table 5.3 below with actual values for Persistence and Return Rate used in Cybersecurity Defense investigations. In the Cybersecurity Defense domain, agent openness is associated with the location of each agent (or the internal state), as we have seen earlier in Tables 4.1a, and 4.1b for defenders, and attackers, respectively. Each agent can always fully observe their own internal state (i.e., their location or presence in the environment). Additionally, since the level-1 planning agent models its neighbors, it can keep track of each neighbor's internal state in its mental model even though it cannot observe it. In doing so, it correctly assumes that each modeled neighbor can fully observe their own internal state and therefore separately tracks this internal state in such neighbor's (nested) belief update as a fully observable component to the respective neighbor.

The impact of agent openness can differ based on the type of reasoning that the defender agent employs, i.e., level-0, and level-1.

For the intelligent level-0 reasoning methods, PBVI and POMCP-PF, where the defender agent does not directly model its neighbors, agent openness (both high and low) can introduce more uncertainty in terms of how accurately the agent can plan for an optimal action without modeling the agent openness for its neighbors. While each level-0 defender agent assumes a random action for each of its neighbors, some of the neighbors might not even be present inside the environment to be taking any action other than NOOP, thereby misguiding the level-0 defender agent's planning process.

For the level-1 I-POMCP-PF defender agent, evaluating the best strategy with agent openness involved can still be very challenging, even if this defender agent models its neighbors

directly. While the I-POMCP-PF defender agent can model its neighbors and therefore has the capability to coordinate with them, agent openness challenges this ability since the defender agent must now first accurately evaluate if each neighbor is going to be present in the environment in the next time step for it to successfully execute any action. While I-POMCP-PF is at an advantage compared to the level-0 reasoning, this could still require comprehensive planning.

The following set of research questions addresses agent openness on top of the baselines. Specifically, we investigate whether the level-1 hybrid team can still outperform the homogenous level-0 defender team when the presence of each agent in the MAS becomes uncertain. We again check this for different levels of defender strength to see if it brings about any changes compared to when there was no agent openness.

- **Research Question 4:** Will the level-1 hybrid I-POMCP-PF defender team bring in more rewards than the homogenous level-0 defender team when there is agent openness involved?
 - **Will the level-1 I-POMCP-PF defender agent act more greedily, especially when there is high agent openness?** High agent openness means that the presence of the agent, either inside or outside the environment, is relatively unstable. Given this, if the defender agent when inside the environment reasons sensibly, it must be able to predict that both its neighbor defender agent and itself are going to be pulled out of the environment in the coming time-steps. Will the defender agent therefore start acting greedily in the current time step? For example, it could choose not to move to a different subnetwork even if it is exploited, to save time, and continue to coordinate with another defender

agent and patch the current subnetwork because it understands that moving to a different subnetwork could be useless since it is going to be pulled out of the environment anyway.

- **Will the level-1 defender agent start acting randomly, especially when there is high agent openness?** For example, when there is high agent openness and all the subnetworks are patched, does the defender agent resort to choosing a random action since it successfully reasons that no agent will be inside the environment long enough to cause an impact? Or when a subnetwork is highly exploited, does it easily give up on the subnetwork because it understands that the other defender agent will not be inside the environment to help successfully patch it?
- **How will agent openness impact the coordination between the level-1 I-POMCP-PF agent and its level-0 PBVI neighbors?**
- **Research Question 5: How will defender strength play a role here in level-1 hybrid I-POMCP-PF team – where all agents (both I-POMCP-PF and PBVI) operate with the same strength setting – to handle agent openness?**

5.2.3. High Observation Error

The purpose of this investigation is to understand how high observation uncertainty (i.e., where the observation error is amplified by 3x relative to the baselines each time the agent makes an observation) complicates agent openness further, and how the level-1 I-POMCP-PF hybrid reasoning method responds to this challenge compared to its homogenous level-0 counterparts.

As discussed earlier in Section 4.7.4 in Chapter 4, in the Cybersecurity Defense domain, the observation model can elicit diverse behaviors depending on the defender agent's reasoning method. Specifically, since this observation model introduces location-dependent observation noise; when located at the home node, the defender agent receives moderately noisy observations across all the subnetworks; at a local subnetwork, the noise is significantly reduced; and when observing non-local subnetworks, the noise becomes considerably high. With a high observation error setting compared to the baselines, these different location-based noises are all significantly amplified (x3 times).

This high uncertainty directly influences emergent behavior – where level-1 defender agents can leverage their neighbor modelling capabilities (given the defender agent can fully observe their teammates' subnetwork locations) to reason about their likely actions and coordinate with them even under this high uncertainty.

The final set of research questions, presented below, focuses on how high observation uncertainty, where the capability of each agent to make an accurate observation of the current environment state is severely degraded, further complicating the agent behavior in the presence of agent openness. These questions are designed to evaluate whether the level-1 I-POMCP-PF agent can still coordinate effectively with the team, when compounded uncertainty exists, i.e., where the agent is uncertain about both the presence of each agent in the system and the true state of the environment.

- **Research Question 6: How does a high observation error affect the rewards of a level-1 hybrid defender team (1 I-POMCP-PF + 2 PBVI) compared to a homogenous level-0 defender team?**

- **Research Question 7: How will defender strength play a role here in level-1 hybrid I-POMCP-PF team – where all agents (both I-POMCP-PF and PBVI) operate with the same strength setting – to handle high observation error?**

5.3. Cybersecurity Defense Setups for the Investigations

This section presents the various setups (Section 5.3.1), domain’s default values (Section 5.3.2), and reasoning-based parameters (Section 5.3.3) that we used to run our experiments in the Cybersecurity Defense domain.

5.3.1. Setups

We have designed a total of 12 setups, given below in Table 5.1, that help us conduct the above investigations in the Cybersecurity Defense domain. Overall, these setups differ in terms of their (i) attacker-defender strengths—specifically the weak, neutral, and strong defender scenarios where the attacker-defender strengths are complementary, (ii) agent openness, and (iii) observation error. They can also be broadly categorized to correlate with our three investigations discussed previously in Section 5.2: the baselines, the agent openness, and the high observation error cases.

Category	Setup	Level of Agent Openness	Observation Error	Defender Strength	Research Questions Addressed
Baselines	1	None	Low	Weak Defender	1, 2, 3
	2	None	Low	Neutral Defender	
	3	None	Low	Strong Defender	
Baselines + Agent Openness	4	Low	Low	Weak Defender	4, 5
	5	Low	Low	Neutral Defender	
	6	Low	Low	Strong Defender	
	7	High	Low	Weak Defender	
	8	High	Low	Neutral Defender	
	9	High	Low	Strong Defender	
Baselines + Agent Openness + High Observation Error	10	None	High	Weak Defender	6, 7
	11	None	High	Neutral Defender	
	12	None	High	Strong Defender	
	13	Low	High	Weak Defender	
	14	Low	High	Neutral Defender	
	15	Low	High	Strong Defender	
	16	High	High	Weak Defender	
	17	High	High	Neutral Defender	
	18	High	High	Strong Defender	

Table 5.1 Setups used in the Cybersecurity Defense domain. Setups 1-3 Correspond to 5.2.1 Baselines; Setups 4-9 correspond to 5.2.2. Agent Openness; Setups 10-18 correspond to 5.2.3 High Observation Error investigations + Agent Openness. See Table 5.2 for the different attacker-defender frame-related strengths used.

As we can see from the above Table 5.1, we investigate each of the scenarios discussed in Section 5.2 for all three types of defender agents, i.e., weak, neutral, and strong. Under the baseline category, the setups have no agent openness and have low observation error in the environment. Agent openness setups are the same as the baselines—plus the low, and high levels of agent openness. High observation error setups are also the same as the baselines plus the agent openness and a high observation error instead of a low observation error.

The actual values used for each of the above configurations is presented in the following Tables 5.2, 5.3, and 5.4, respectively for Defender Strength, Agent Openness, and Observation Error configurations.

Setting	Strength Value (Attacker-Defender)
Weak Defender	0.10-0.50
Neutral Defender	0.05-1.00
Strong Defender	0.01-1.50

Table 5.2 Different Attacker-Defender frame-related strengths used in our implementation.

Agent Openness	Setting	Value	
		Persistence	Return Rate
	No Agent Openness	1.00	1.00
	Low Agent Openness	0.90	0.10
	High Agent Openness	0.10	0.90

Table 5.3 Different Agent Openness settings used in our implementation. Persistence is the likelihood of an agent remaining inside the environment, and Return Rate is the likelihood of the agent returning when it is outside. Both attackers and defenders have the same setting within a simulation.

Observation Error	Agent Location	Observing	Observation Error Value
Low Observation Error	Home	Any Subnetwork	0.20
	Subnetwork	Current (Local) Subnetwork	0.05
	Local Subnet	Other Subnetworks	0.35
High Observation Error	Home	Any Subnetwork	0.60
	Subnetwork	Current (Local) Subnetwork	0.15
	Local Subnet	Other Subnetworks	0.90

Table 5.4 Observation Error – Low and High, used in our implementation.

5.3.2. Domain Defaults

For each of the above setups, we define some default values that define the domain, given in

Table 5.5 below. These are standard for all our experiments.

Baseline Setup		
Category	Parameter	Value
Simulation Setup	Number of Subnetworks	3 Subnetworks
	Number of Time-Steps	10
	Number of Runs	30
Agents & Reasoning	Number of Attackers	2
	Number of Defenders	3
	Level-1 Agent Composition	1 I-POMCP-PF Defender + 2 PBVI Defenders
	Attacker Reasoning	Heuristics – Attack Weak (Exploited) Subnetworks
Initial Conditions	Agent Openness	No Agent Openness
	Simulation Start State	Random per run, consistent trend across reasoning models
	Attacker Start Location	Inside environment (Subnetwork: N/A)
Observation & Uncertainty	Defender Start Location	Inside environment (Subnetwork: Home Node)
	Observation Function	Separate Location-Based Observation Errors
Rewards & Costs	Observation Error	Refer to Table 5.4
	Rewards (Amplified For Edge States)	Patched: 9.0 ; Vulnerable: 2.0; Exploited-1: 0.0; Exploited-2: -2.0; Exploited-3: -9.0
	Patch Cost	Low (0.01)
Defender Strengths	Attacker Chip Damage	Low (0.1)
	Attacker-Defender Strengths	Weak, Neutral, Strong

Table 5.5 Cybersecurity Defense domain’s default parameters used in our implementation.

5.3.3. Reasoning Algorithms

This section presents in Table 5.6 the list of all the reasoning algorithms we employ in our investigations, along with any applicable run-time parameters. The table also includes a brief overview of each reasoning algorithm’s core logic.

Defender Reasoning	Type	Overview	Special parameters
Random	Naïve	Defender agent chooses a random legal action based on their current location	N/A
Heuristic Fixed Defender	Naïve	Defender agents start at random subnetworks, stay there, and patch the respective subnetworks	N/A
Heuristic Spread Defender	Naïve	Defender agents intermittently spread between the subnetworks based on their observation, moving to the subnetwork that looks most exploited, and patching it	N/A
PBVI	Intelligent (level-0)	Offline level-0 reasoning where the defender agent doesn't model its neighbors	1. Number of PBVI iterations = 15 2. Maximum number of beliefs added per iteration = 500 3. Value iteration is performed once every 3 iterations 4. Horizon = 10 5. Gamma = 0.95
POMCP-PF	Intelligent (level-0)	Online level-0 reasoning where the defender agent doesn't model its neighbors	1. Trajectories = 5000 2. Number of particles = 100 3. Horizon = 5 4. Gamma = 0.95
I-POMCP-PF + PBVI Hybrid	Intelligent (level-1)	Online level-1 reasoning where one of the defender agents models their neighbors as level-0 agents, and the two other defender agents act as PBVI agents. Within the scope of this thesis, the level-1 defender agent models the attackers as Heuristic reasoning agents who target the most exploited subnetworks	5. Number of PBVI particles (only for level-1) = 50 6. UCB = ~6 for POMCPPF and ~18 for I-POMCPPF

Table 5.6 Different Defender Reasoning algorithms used for all our investigations and any applicable reasoning-specific parameters.

A breakdown of rationale for the run-time parameters used in Table 5.6 is given below. While the goal was to reduce the run time, we also empirically checked to see that the sensibility and performance of each of the reasoning methods was not compromised.

For PBVI, we have chosen the following parameters:

- Number of iterations = 15, as this lets the algorithm explore a reasonably large number of belief points (~3000) and through 15 time-steps from the initial belief. Given that the number of time steps in all our simulations is 10, 15 time-steps should be a good stopping point for the PBVI algorithm.
- Maximum number of beliefs added per iteration = 500, to limit the explosion of the explored belief space (PBVI belief expansion, by default, doubles the number of belief points in each iteration), and makes it hard to scale beyond the 10th iteration where the number of beliefs is already in the thousands.
- Interval between the value iterations = 3, so that we can save computation time by iterating over the value function less frequently. We make sure that the last iteration still ends with a value iteration to generate the solution set to all the beliefs explored.

For POMCPPF and I-POMCP-PF, we have chosen the following parameters:

- Trajectories = 5000, number of particles = 100, and number of neighbor particles for each level-1 nested belief = 50, because we had earlier empirically checked to see that the actions of the defender agent converged with this configuration.
- UCB_C = ~6 for level-0 POMCPPF, and UCB = ~18 for I-POMCPPF because we had empirically observed that they result in convergence given the reward structure of the domain, and the number of trajectories = 5000. We observed that, in general, a stronger

defender agent requires a higher UCB_C value, and this value almost triples from level-0 POMCPPF to level-1 I-POMCPPF.

5.4. Results

This section discusses the results for each of our investigation setups corresponding to our research questions 1 through 7 (refer Table 5.1), each of which is respectively presented in below sections 5.4.1 through 5.4.7. That is, sections 5.4.1, 5.4.2, and 5.4.3 correspond to the baseline investigations; sections 5.4.4, and 5.4.5 correspond to the agent openness investigations; and sections 5.4.6, and 5.4.7 correspond to the agent openness + high observation error investigations.

For each investigation, we first present our results (agent rewards, and action choices), make any observations, justify the observations, and finally discuss the implications.

5.4.1. Research Question 1: Does the level-1 hybrid defender team bring better rewards than the homogenous level-0 defender team?

The following Figure 5.1 depicts the rewards for the defender agent for the baseline setups, i.e., Setup 1, Setup 2, and Setup 3 (refer to Table 5.1), where there is no agent openness, and the observation error is low. We compare the results across all our reasoning types. In-turn, within each reasoning type, we compare the results across the different defender agent types, i.e., weak defender, neutral defender, and strong defender.

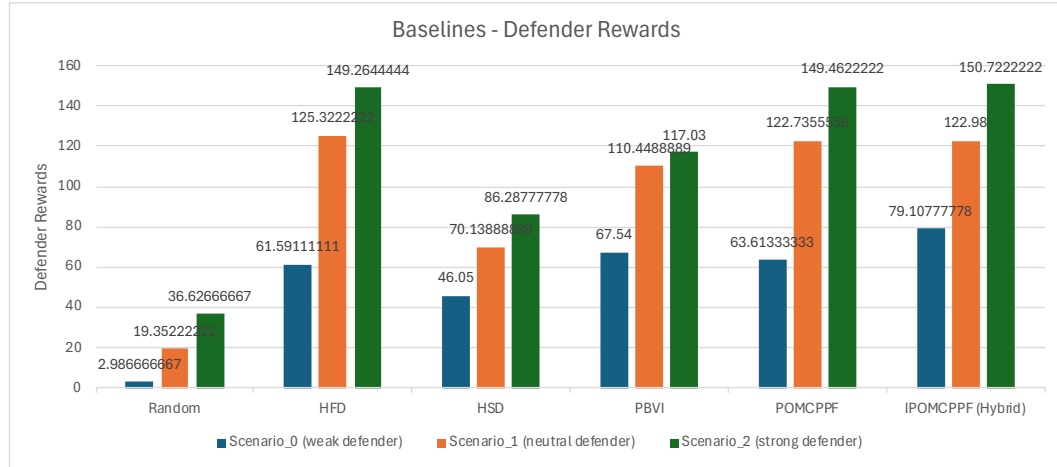


Figure 5.1 Weak, neutral, and strong defender agent’s rewards across each reasoning type. Higher values are better.

From Figure 5.1, we observe in terms of rewards:

- Weak Defender (Scenario 0): The hybrid I-POMCP-PF team ranks first, with PBVI and POMCP-PF in clear second and third places; HFD, HSD, and Random follow in descending order.
- Neutral Defender (Scenario 1): HFD takes the top spot, while the hybrid I-POMCP-PF and POMCP-PF occupy a close second-and-third pair; PBVI comes next, with HSD and Random at the bottom.
- Strong Defender (Scenario 2): Hybrid I-POMCP-PF leads again, POMCP-PF is runner-up, and HFD finishes third; PBVI, HSD, and Random remain in the lower ranks.

Overall, the hybrid I-POMCP-PF team performs best when the defender is weakest, and as defender strength increases, it closely trails HFD in the neutral case and then slightly outperforms all methods when the defender is strong.

The hybrid I-POMCP-PF agent excels in the weak defender scenario because its level-1 reasoning allows it to anticipate and join its PBVI teammates efficiently, enabling coordinated three-agent and two-agent patching where needed (as we will shortly see in Table 5.10). As defender strength increases, the need for such intensive coordination diminishes—there are fewer exploited subnetworks, and simpler level-0 policies like HFD (which prioritize monitoring and split the team naturally) begin to match or exceed the hybrid in terms of reward. POMCP-PF, with its online adaptability, also becomes more effective in stronger defender cases.

We can therefore imply that the hybrid I-POMCP-PF reasoning performs fairly effectively when the defenders are weak, but as the strength of the defender increases (i.e., when it becomes easier to patch subnetworks), a lightweight or specialist planner can be just as effective—making the additional complexity of hybrid reasoning less critical in high-resource settings.

5.4.2. Research Question 2: How does the level-1 I-POMCP-PF defender agent coordinate with its level-0 neighbors during the simulation?

The following Table 5.8 reports, for each defender-strength scenario, the share of joint patch actions (i.e., when multiple defender agents choose to patch the same subnetwork in the same time-step) that involve (a) the level-1 I-POMCP-PF defender agent pairing with one PBVI neighbor, (b) two PBVI defender agents alone, or (c) all three defender agents together. This data, **like all other data from this section**, is from across all the simulation runs and considers all the time-steps within a run and corresponds to the respective reasoning methods.

Scenario (Defender Strength)	1 I-POMCPPF + 1 PBVI Pair (%)	1 PBVI + 1 PBVI Pair (%)	1 I-POMCPPF + 1 PBVI + 1 PBVI Triad (%)
0 (Weak)	2.95	45.15	51.90
1 (Neutral)	8.72	40.37	50.92
2 (Strong)	15.17	34.12	50.71

Table 5.8 Joint patch actions in the Hybrid I-POMCP-PF + PBVI defender team. All the joint-patches (from 2 or 3 defender agents) are considered and split as a % based on the reasoning types of the defender agents involved.

From the above Table 5.8, we can see that

- For the **weak** defender scenario, most coordination is a full three-agent effort, with I-POMCP-PF defender agent only rarely patching alongside a single PBVI defender agent.

With many subnetworks simultaneously exploited, both PBVI defender agents independently converge on the most critical subnetworks. The level-1 I-POMCP-PF agent, detecting this clustering, joins in, leading to a higher percentage of full three-agent working together to patch a subnetwork.

- In the **neutral** defender scenario, full-team patches remain dominant, but I-POMCP-PF defender agent's direct pairings rise (~3×), while PBVI pairs drop.

As the network heals and fewer exploits remain (due to increased defender strength), PBVI agents spread their efforts across different targets, causing PBVI–PBVI pairings to naively dip. Meanwhile, I-POMCP-PF's ability to anticipate each PBVI partner lets it pick the right collaborator, driving up hybrid pairwise patches.

- When the defender agents are strong, three-agent patches remain just above half; I-POMCP-PF + PBVI pairings climb further, and PBVI + PBVI pairings continue to decline.

This pattern suggests that as subnetworks become healthier and fewer attacks remain, the hybrid I-POMCP-PF agent increasingly steps in to guide efficient two-agent patching—compensating for the lack of coordination among PBVI agents. At the same time, it continues to support full three-agent patches when necessary, ensuring that critical subnetworks still receive adequate coverage. This balance reflects I-POMCP-PF’s ability to tailor coordination based on evolving network conditions—avoiding unnecessary over-patching while still reinforcing collective responses where most impactful.

Continuing from the above, this shifting patch-team behavior is mirrored in how often agents traverse the network (as shown in Table 5.9):

Scenario (Defense Strength)	IPOMCPPF Defender Moves	PBVI Defender Moves	Hybrid Average Moves
0 (Weak)	93.00	54.00	67.00
1 (Neutral)	90.00	71.50	77.67
2 (Strong)	92.00	83.00	86.00

Table 5.9 Moves-to-subnetwork actions by the hybrid reasoning (I-POMCP-PF + PBVI). Since the Hybrid team consists of 1 I-POMCP-PF defender but 2 PBVI defenders, the data is averaged across the 2 PBVI defenders of the team.

- Weak Defender:** The I-POMCP-PF agent moves about **93** times, its PBVI partner **54** times, and the combined team **67** times. With many simultaneously exploited subnetworks, defender agents spread unevenly across targets, and full-team patches dominate even though PBVI–PBVI pairing remains substantial.
- Neutral Defender:** Movement stays relatively the same (**90**) for I-POMCP-PF, increases to **71.5** for PBVI, and **77.67** overall. Fewer exploits keep agents somewhat dispersed—maintaining high three-agent coordination—but dyadic IPOMCPPF–PBVI patches increase as pair overlaps grow.

- **Strong Defender:** Nearly the entire network is traversed—**92** moves by I-POMCP-PF, **83** by PBVI, and **86** combined—forcing all agents into the same location to clear the last breach. Despite fewer actively exploited subnetworks, full-team patching remains high, while IPOMCPPF–PBVI pairings rise and PBVI–PBVI pairings fall (see Table 5.8 above), reflecting the level-1 agent’s increasing role in selectively coordinating with individual partners.

This implies that as the subnetworks become less exploited, effective coordination hinges on agents converging to the same subnetwork deliberately, and not by chance. As we can see from Table 5.8, and Table 5.9, the level-1 I-POMCP-PF defender agent increases its pairwise coordination with PBVI while maintaining high movement, signaling strategic teaming. In contrast, while PBVI defender agents also move more, their pairings decline and show no clear selectivity—suggesting incidental, not intentional, coordination.

We have so far seen how the hybrid I-POMCP-PF + PBVI defender agents coordinate within themselves. But ***how is this behavior different compared to the homogenous level-0 methods where the defender agents do not model their neighbors?***

Below Table 5.10 shows the data from our experiments for the share of joint-patch actions of the defender agents across two- vs. three-defender teams.

Defender Reasoning	Scenario	Two-Agent (%)	Three-Agent (%)
PBVI	Weak	6.25	93.75
	Neutral	15.38	84.62
	Strong	17.97	82.03
POMCPPF	Weak	25.10	74.90
	Neutral	26.84	73.16
	Strong	33.63	66.37
HFD	Weak	59.19	40.81
	Neutral	55.26	44.74
	Strong	58.99	41.01
Hybrid (IPOMCPPF + PBVI)	Weak	48.10	51.90
	Neutral	49.08	50.92
	Strong	49.29	50.71

Table 5.10 Joint-patch shares, split between two-agent patches and three-agent patches, for level-0 vs. I-POMCP-PF. For the Hybrid reasoning, Two-agent (%) includes both 1 I-POMCPPF + 1 PBVI, and 1 PBVI + 1 PBVI agent shares.

Table 5.10 is shown below in Figure 5.2 as a bar chart for easier comprehension.

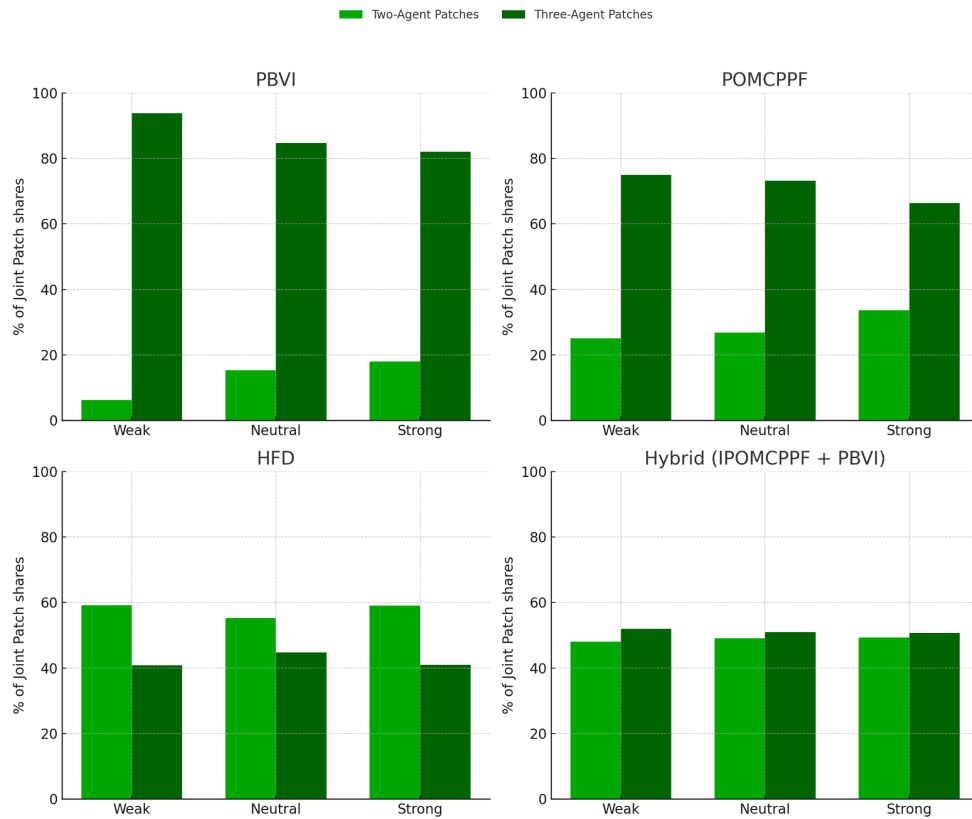


Figure 5.2 Joint-patch shares, split between two-agent patches and three-agent patches, for level-0 vs. I-POMCP-PF (same as Table 5.10)

From the above Table 5.10, we can see that

- Level-0 **PBVI** defender agents almost always attempt three-defender patches, yielding very low two-agent shares, and only shift slightly toward pairs when the strength of the defender agent increases.

This is because PBVI relies on offline value iteration with fixed beliefs, which tends to lock agents into converging on the same high-value subnetwork – especially when the defender agent is weak – since they cannot adapt to real-time teammate

behavior. This results in persistent over-patching, even as the defender agent gets stronger.

- Level-0 **POMCP-PF** likewise favors full-team patches but shows a moderate growth in pair-wise actions as the defender strength increases.

POMCP-PF, unlike PBVI, is an online planner and has the capability to adjust its action choices based on current belief states, even without explicitly modeling neighbors. As the subnetworks stabilize in stronger defender scenarios, this adaptivity lets POMCP-PF defender agents spread out slightly, reducing full-team patching more than PBVI does.

- **HFD** consistently achieves the highest two-agent share and seems to naturally divide the team into pairs when the defender agents first move to random subnetworks from the home node.
- **Hybrid I-POMCP-PF** team: Maintains a relatively stable mix of two-agent (~49%) and three-agent (~51%) coordination across all scenarios. Even as defender strength increases and exploited subnetworks become sparser, the team adapts without overcorrecting—balancing full-team and pairwise patching efficiently.

Overall, although level-0 models PBVI and POMCP-PF show some adjustment in team coordination as defender strength increases, they begin with extreme reliance on full-team patches and only shift modestly as the threat level drops. In contrast, the hybrid I-POMCP-PF + PBVI team maintains a relatively stable mix of two- and three-agent coordination across all scenarios—even though the neighboring PBVI defender agents show a consistent decline in *PBVI–PBVI* pairings. This indicates that the level-1 I-POMCP-PF agent is actively compensating

for reduced coordination among the neighboring PBVI agents by selectively forming effective two-agent teams, thereby preserving overall team cohesion and patch coverage efficiency regardless of shifting threat levels.

5.4.3. Research Question 3: How would the level-1 hybrid I-POMCP-PF defender team – with all agents (both I-POMCP-PF and PBVI) operate with the same strength setting – respond across different scenarios as we vary that common defender strength?

To understand how the level-1 Hybrid I-POMCP-PF agent responds to increasing defender strength when compared to level-0 planners (PBVI and POMCP-PF), we examine both the average of patch actions (Table 5.11 below) and the associated team rewards (Figure 5.1 above).

Reasoning	Weak	Neutral	Strong	Trend
PBVI	238.00	215.33	211.00	Decreases
POMCPPF	236.00	219.33	211.33	Decreases
Hybrid PBVI	240.00	219.50	207.00	Decreases
Hybrid IPOMCPPF	143.00	149.00	166.00	Increases
Hybrid Team	191.50	184.25	186.50	Stable

Table 5.11 Average per-Agent Patch Counts across Reasoning Methods. Hybrid IPOMCPPF corresponds to the level-1 defender agent from the Hybrid team, and Hybrid PBVI corresponds to the average of the two level-0 PBVI defender agents from the Hybrid team. PBVI and POMCPPF correspond to the averages across the three defender agents from the homogenous teams.

The above Table 5.11 is show below as a line chart in Figure 5.3

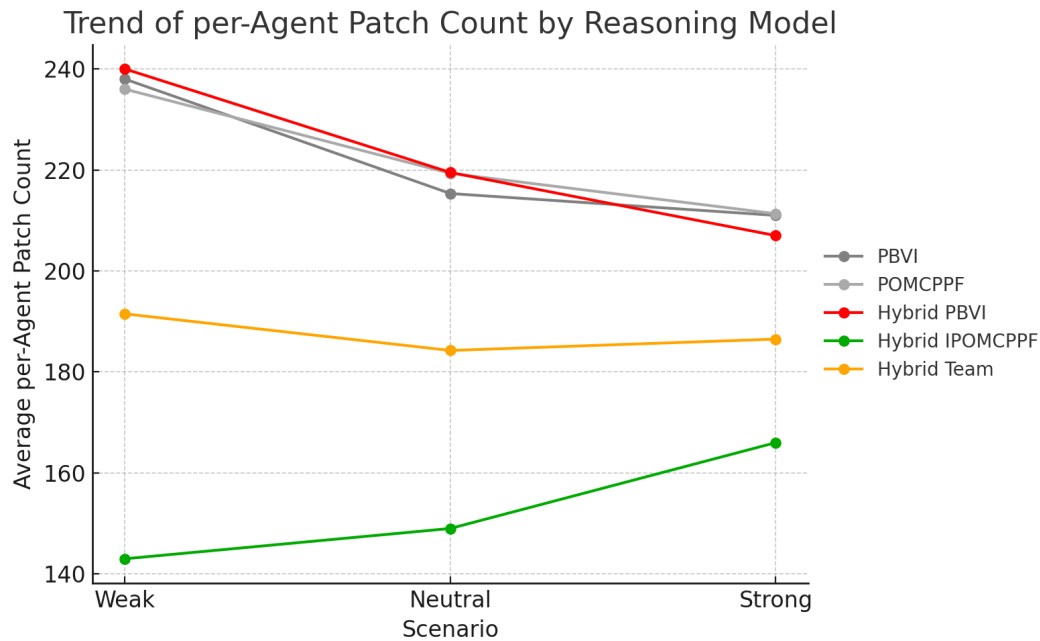


Figure 5.3 Average per-Agent Patch Counts across Reasoning Methods (same as Table 5.11)

We observe from the above data that as defender strength increases, PBVI and POMCPPF steadily reduce their patch counts, while the Hybrid I-POMCP-PF defender agent uniquely *increases* its patching effort thereby compensating for the declining activity of PBVI. Furthermore, despite using significantly fewer patches on average than both level-0 reasoning types, the Hybrid I-POMCP-PF agent manages to achieve the highest reward across all scenarios, especially outperforming homogenous PBVI by a great margin.

The contrasting patching trends between Hybrid I-POMCP-PF and the level-0 reasoning methods arise from fundamental differences in their reasoning capabilities. PBVI depends on fixed, offline value policies that ignore teammate behavior; when defender strength rises and the environment appears safer, these static policies simply translate stronger defenses into fewer patch actions, without considering whether other defenders are making the same

deduction. POMCP-PF, although it replans online, likewise lacks an explicit model of neighbors and therefore follows a similar heuristic—reducing its patch rate as conditions improve on the assumption that less work is required. By contrast, the Hybrid I-POMCP-PF agent performs level-1 reasoning: it maintains a belief over its PBVI neighbors’ future actions, detects their collective drop in patching, and deliberately increases its own patch effort to keep overall coverage steady. Thus, the hybrid’s rising patch count is not an indiscriminate response to defender strength but a targeted adjustment that compensates for the coordination blind spots of its level-0 teammates.

These findings demonstrate that as defender strength increases, coordination cannot be assumed to improve uniformly across reasoning types. While level-0 agents scale back patching under the assumption that stronger defenses require fewer patch actions, the level-1 Hybrid I-POMCP-PF agent actively monitors this shift and adjusts its behavior to fill emerging gaps. This behavior highlights that strategic adaptability—not raw strength alone—is critical to maintaining coordinated responses as threat conditions evolve and contributes to the hybrid team’s consistently strong performance across all scenarios—especially compared to pure PBVI, where introducing just one level-1 agent into an otherwise homogenous team not only mitigates over-reliance on static policies but also drives notable gains in overall team reward without requiring full-scale mutual modeling.

5.4.4. Research Question 4: Will the level-1 hybrid I-POMCP-PF defender team bring in more rewards than the homogenous level-0 defender team when there is agent openness involved?

- Will the level-1 I-POMCP-PF defender agent act more greedily, especially when there is high agent openness?

- Will the level-1 defender agent start acting randomly, especially when there is high agent openness?
- How will agent openness impact the coordination between the level-1 I-POMCP-PF agent and its level-0 PBVI neighbors?

The following Figure 5.4 depicts the rewards for the defender agent for the setups for baselines (Scenarios 0, 1, 2), low agent openness (Scenarios 3, 4, 5), and high agent openness (6, 7, 8); where the observation error is low. We compare the results across all our reasoning types. In turn, within each reasoning type, we compare the results across the different defender agent types, i.e., weak defender, neutral defender, and strong defender.

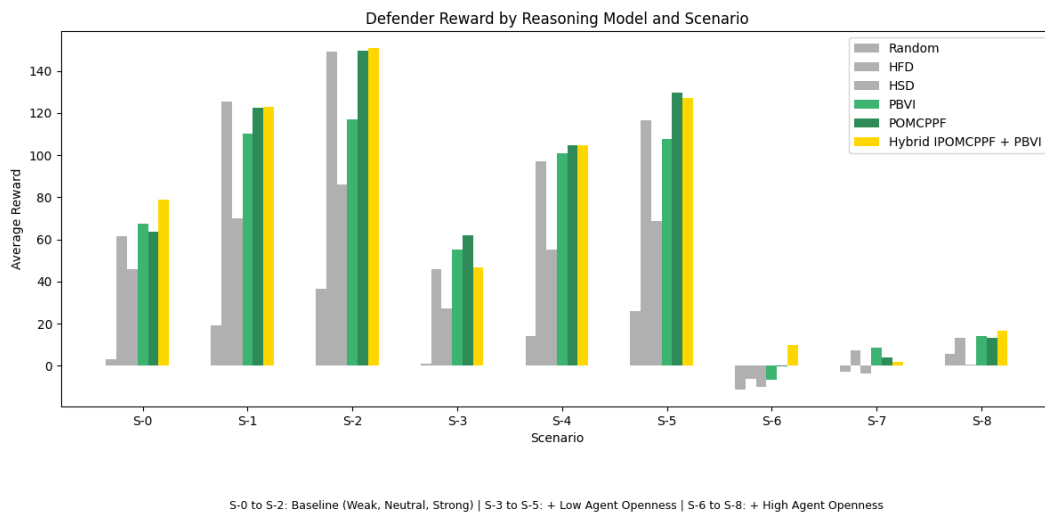


Figure 5.4 Weak, neutral, and strong defender agent’s rewards across each reasoning type, and across low, and high agent openness. Higher values are better.

From the reward outcomes under agent openness as shown above, we observe the following patterns:

- **Weak Defender (Scenarios 3 & 6):** In the low agent openness case (Scenario 3), the hybrid (I-POMCP-PF + PBVI) team averages about **46.77** in rewards, which is lower than

the homogeneous PBVI team (~**55.22**) and the POMCP-PF team (~**61.89**). Under high agent openness (Scenario 6), however, the hybrid team outperforms all others – achieving a positive reward around **10.06** while all homogeneous level-0 teams suffer negative rewards (e.g., PBVI $\approx$ **-6.49**, HFD $\approx$ **-6.27**) or near zero (POMCP-PF $\approx$ **-0.28**).

This indicates that when defenders are very weak and there is high agent openness, the hybrid’s reward stays positive whereas simpler teams falter.

- **Neutral Defender (Scenarios 4 & 7):** With low agent openness (Scenario 4), the hybrid achieves roughly **104.74** in rewards, on par with the best level-0 strategy (POMCP-PF **104.95**) and slightly above the PBVI team (**101.02**). In the high-openness neutral case (Scenario 7), the hybrid team’s reward drops dramatically (**1.65**). Here, a homogeneous PBVI team (**8.68**) and even the simple heuristic (HFD **7.27**) outperform the hybrid. Notably, the hybrid is one of the worst performers when agent openness is high and defenders have medium strength.
- **Strong Defender (Scenarios 5 & 8):** For low agent openness (Scenario 5), the hybrid team scores about **127.33**, just a bit below the (top) POMCP-PF team (**129.95**). Under high agent openness (Scenario 8), the hybrid again ranks highest with approximately **16.86** in rewards, edging out the PBVI (**14.02**) and POMCP-PF (**13.10**) teams. All defender teams manage to get positive rewards in the strong-defender, high agent openness setting, but the hybrid maintains a slight lead.

Overall, the hybrid I-POMCP-PF + PBVI team does not consistently dominate the homogeneous level-0 teams once agent openness is introduced. In low agent openness conditions, the hybrid’s reward is comparable to the best level-0 strategy (and sometimes slightly lower). In high-openness conditions, the hybrid shows a mixed outcome: it excels in the most challenging scenario (weak defenders) but lags simpler teams in a moderate scenario

(neutral defenders) and slightly dominates again in the scenario where the defender agents are stronger.

To make sense of the above trend, we look at how agent openness disrupts coordination dynamics within the environment.

Low agent openness in the Cybersecurity Defense domain eases two-agent coordination and diminishes three-agent coordination, thereby taking away I-POMCP-PF's edge. In the baseline scenarios (no agent openness), we saw how the level-1 hybrid reasoning gave a clear benefit by coordinating defender agents, which often translated to higher rewards. However, compared to no agent openness, when the system has low agent openness, it is more unlikely that all three defender agents are simultaneously present in the system. This is supported by the consistent drop in three-agent patch events across all reasoning types (**see Figure 5.5 below**). Given the design of the environment—where rejoining defender agents always return to the home node—they tend to independently move toward the most exploited subnetworks upon re-entry. As a result, even level-0 agents naturally converge on the same high-risk subnetworks, increasing the frequency of two-agent coordination (Such convergence is otherwise less likely when agents are already stationed at subnetworks, since they tend to prioritize patching locally rather than relocating—resulting in fewer spontaneous pairings). In such a setting, the level-1 I-POMCP-PF agent's coordination advantage diminishes, as partial team-ups become sufficient and even level-0 policies achieve effective convergence without explicit modeling. Consequently, the hybrid agent's cautious coordination behavior leads to outcomes that are merely on par with, or in some cases slightly worse than, aggressive level-0 planners like POMCP-PF.

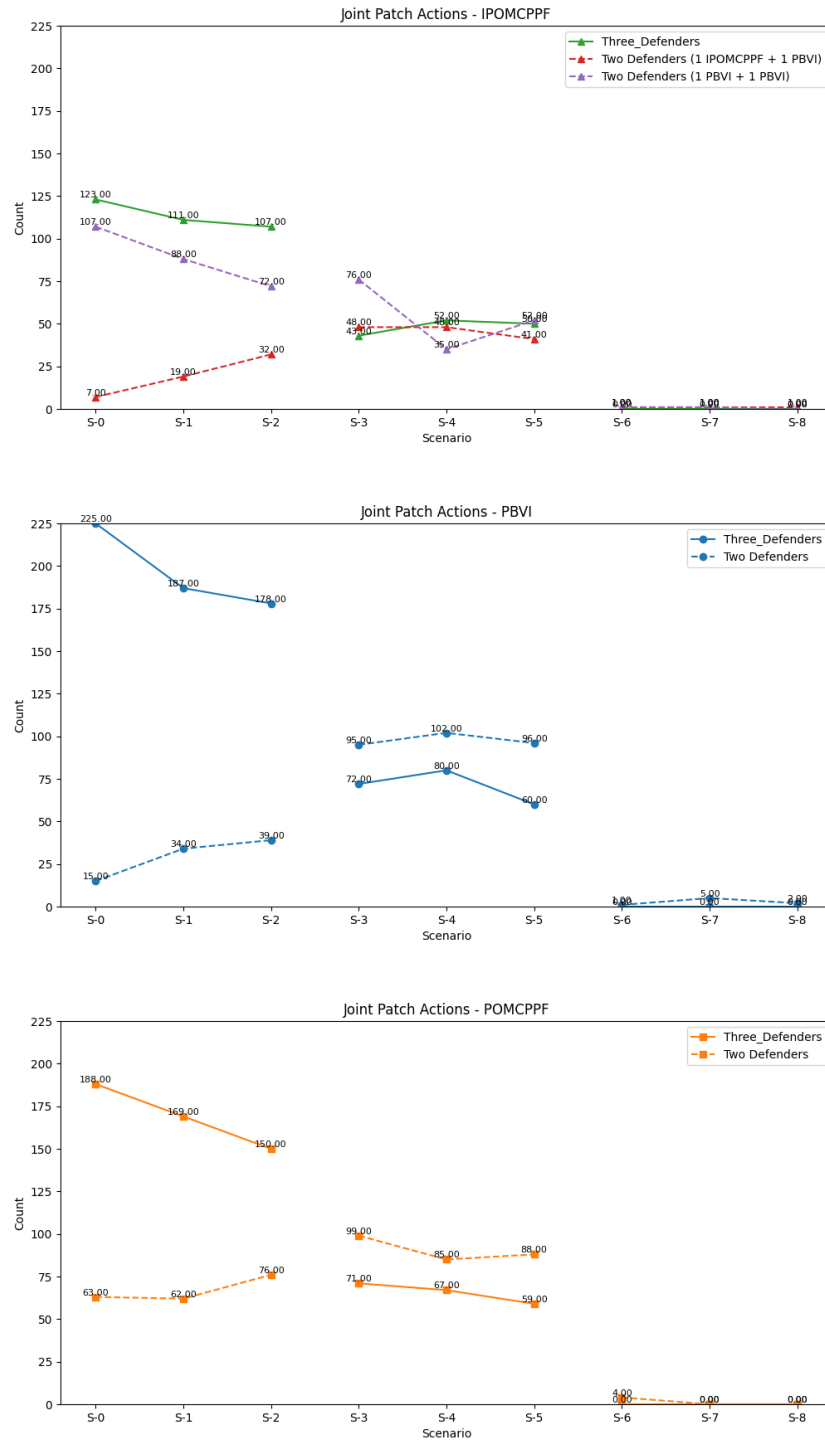


Figure 5.5 Two vs. Three-Agent Joint Patch Actions in the Hybrid I-POMCP-PF, PBVI, and POMCP-PF reasoning methods. Scenarios 0-2 are baselines, 3-5 are baselines + low agent openness, and 6-8 are baselines + high agent openness.

High agent openness in the Cybersecurity Defense domain forces the level-1 I-POMCP-PF agent to operate selectively depending on the conditions, amplifying its benefits in extreme defender setups (i.e., when the defender strength is low or high) while undermining it in more balanced ones (i.e., when the defender strength is neutral).

- When the defender agents are weak (Scenario 6), attacker agents exploit the weak subnetworks rapidly, while coordination becomes infeasible at the same time due to defender agents frequently exiting the environment. In such cases, the I-POMCP-PF agent abandons complex joint strategies and instead patches immediately upon reaching an exploited subnetwork, often acting alone. This can be seen from the following Figure 5.6 where we show the percentage number of times a defender agent was stationed at a subnetwork and chose to patch. Compared to its PBVI teammate in the weak defender scenario (Scenario 0), the I-POMCP-PF agent patches 35.1% less frequently, *highlighting its tendency to wait for joint coordination before acting*. However, under high agent openness (Scenario 6), this relative gap shrinks dramatically to just 7.7%, *indicating that the I-POMCP-PF agent increasingly abandons its cautious, model-based behavior in favor of solo, reactive patching*. This sharp convergence in behavior—driven entirely by the I-POMCP-PF agent’s 54.2% relative increase in patching frequency—*shows how high openness compels the level-1 defender agent to shift its strategy toward greedy and independent patching, effectively allowing the hybrid team to retain positive rewards even when all the level-0 teams collapse*.
 - I-POMCP-PF defender agent’s behavior in general shifts towards greedy patching as agent openness increases.

As seen from below Figure 5.6, across all defender strengths, the I-POMCP-PF agent's patch frequency (when they are stationed at a subnetwork) **increases steadily** with agent openness. For weak defenders (Scenarios 0→3→6), the patch rate rises from 57.66 % to 65.81 %, and finally to 92.31 %. Similar trends hold for neutral (61.57 % → 65.08 % → 84.62 %) and strong defenders (64.59 % → 65.57 % → 91.67 %), indicating a consistent shift toward immediate patching.

These results suggest that as the environment becomes more volatile, the I-POMCP-PF agent abandons its cautious, coordination-seeking behavior in favor of solo, greedy responses. This adaptation is most visible under high agent openness, where waiting for teammates is less rewarding and acting immediately becomes the only viable option.

The increasing greediness of the I-POMCP-PF agent under high openness **implies a shift in its role**—from a coordination-driven planner to a fast-acting solo responder. This behavioral pivot suggests that the value of level-1 reasoning is highly conditional: it excels when coordination is possible, but under high openness, its complexity merely enables a fallback to opportunistic behavior that simpler policies could replicate.

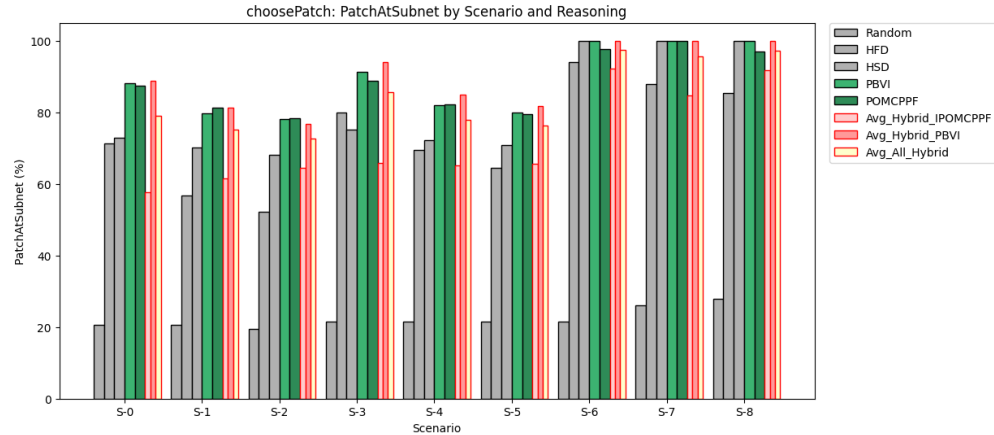


Figure 5.6 % number of times defender agent chooses Patch action when stationed at a subnetwork. Averaged for three agents for the homogenous PBVI and POMCPPF teams, and across (1 I-POMCP-PF + 2 PBVI) defender agents for the Hybrid team.

- In contrast, when the defender agents are neutral (Scenario 7), the need for coordination is still present but harder to fulfill. The hybrid agent often spends time anticipating help from teammates—through monitoring or detours—that rarely arrives due to high agent openness. This results in missed patching opportunities, allowing simpler level-0 strategies such as PBVI and HFD, which patch immediately without waiting, to outperform the hybrid. The same can be seen from the above Figure 5.6 where the % number of times the I-POMCP-PF defender agent chooses to patch shrinks compared to the weak defender scenario from 92.31 % (Scenario 6) down to 84.62 % (Scenario 7), a drop of **7.69 percentage points**.
- Finally, in the strong defender case (Scenario 8), individual agents are capable enough to patch threats on their own and I-POMCP-PF adapts to this by increasing its patch rate from 84.62 % in the neutral defender scenario (Scenario 7) up to 91.67 % in Scenario 8—a **gain of 7.05 percentage points**.

Overall, the hybrid I-POMCP-PF + PBVI team's performance under agent openness reveals that level-1 reasoning is not universally advantageous—it is highly sensitive to context.

In challenging settings where defender agents are weak and the environment has high agent openness, the level-1 I-POMCP-PF agent’s ability to act decisively in isolation allows the team to bring positive rewards, offering a clear advantage over level-0 strategies. However, in moderate settings—particularly when defenders are neutral—the same level-1 planning becomes a burden. Here, the I-POMCP-PF agent’s attempts to anticipate coordination under high agent openness leading to delayed patching and ultimately weaker team performance compared to the level-0 reasoning methods. Conversely, when defenders are strong, level-1 reasoning again proves useful—not for coordination, but for opportunistically patching gaps left by teammates.

- This indicates that the agent’s behavior is far from random, thereby addressing our research question *“Will the level-1 defender agent start acting randomly, especially when there is high agent openness?”*; rather, it is adaptive and context-aware—greedy when teammate presence is uncertain, opportunistic when solo action is sufficient, and cautious only when coordination is feasible. The level-1 I-POMCP-PF agent does not follow a fixed response pattern but adjusts its strategy based on both the presence of teammates and the dynamics introduced by agent openness.
- This also speaks to the third research question on coordination: *“How will agent openness impact the coordination between the level-1 I-POMCP-PF agent and its level-0 PBVI neighbors?”*. While the I-POMCP-PF agent is explicitly designed to model and anticipate teammate actions, high agent openness degrades the reliability of such models. As teammates frequently exit and re-enter the environment, their presence at key subnetworks becomes uncertain, making coordination impractical. This is particularly evident in the neutral defender case, where coordination would be valuable but fails to materialize—leading to suboptimal delays from the hybrid agent. In contrast, in weak or

strong defender settings, the I-POMCP-PF agent adapts by either acting alone or filling residual gaps, showing that its coordination attempts scale with feasibility, not randomness.

Thus, agent openness turns the hybrid agent’s modeling capability into a conditional asset—highly effective when coordination is both required and more importantly, feasible, or when independent action becomes the only viable means of maintaining coverage. This reinforces the notion that level-1 hybrid reasoning should be deployed selectively, with careful consideration of both agent strength and expected agent openness.

5.4.5. Research Question 5: How will defender strength play a role here in level-1 hybrid I-POMCP-PF team – where all agents (both I-POMCP-PF and PBVI) operate with the same strength setting – to handle agent openness?

Here, we turn to a more granular question: how does defender strength influence the workload-sharing dynamics between the Hybrid PBVI and Hybrid I-POMCP-PF agents as agent openness varies? Note that we have already discussed in Research Question 4 how defender strength interacts with agent openness to shape both the coordination behavior and the reward outcomes of hybrid teams—showing, for instance, that level-1 reasoning proves most beneficial when defenders are either very weak (where decisive solo action is needed) or very strong (where opportunistic patching fills residual gaps), but falters in moderate-strength settings where coordination fails to materialize. Aspects like joint patch actions (**see Figure 5.5 above**) and patch-when-at-subnetwork rates (**see Figure 5.6 above**) showed that as defender strength increases, *the hybrid I-POMCP-PF agent gradually shifts from cautious coordination toward more reactive or opportunistic strategies*. In particular, we examine whether the level-1 I-POMCP-PF

agent continues to compensate for its PBVI teammate's patching behavior as defender strength rises—or whether this balancing mechanism deteriorates under high openness.

To compare teammate behavior, we place side-by-side:

- Average patch counts for each agent (i.e., Hybrid I-POMCP-PF, and Hybrid PBVI)
- % of times each defender agent chooses to patch when at subnetwork (see Figure 5.6 above)

Defender strength	Agent Openness	Hybrid PBVI patch count	Hybrid I-POMCP-PF patch count	PBVI % patched-when-at-subnetwork	I-POMCP-PF % patched-when-at-subnetwork
Weak	none (S-0)	240.00	143.00	88.90%	57.70%
	low (S-3)	171.50	102.00	93.90%	65.80%
	high (S-6)	13.00	12.00	100.00%	92.30%
Neutral	none (S-1)	219.50	149.00	81.30%	61.60%
	low (S-4)	145.00	123.00	85.00%	65.10%
	high (S-7)	16.50	11.00	100.00%	84.60%
Strong	none (S-2)	207.00	166.00	76.70%	64.60%
	low (S-5)	149.50	120.00	89.20%	65.60%
	high (S-8)	12.00	11.00	100.00%	91.70%

Table 5.12 Hybrid agents patch counts and % patches when at subnetwork. Hybrid I-POMCP-PF corresponds to the level-1 agent from the Hybrid team, Hybrid PBVI corresponds to the average of the 2 PBVI agents from the Hybrid team, and PBVI, POMCPPF correspond to the averages of the respective 3-agent homogenous teams.

We make the following observations from the above data with respect to agent openness across different agent strengths.

- No agent openness: As defender strength rises, Hybrid PBVI's patch count steadily falls (weak → strong: 240 → 207), while Hybrid I-POMCP-PF's count rises (143 → 166). The gap shrinks from 97 patches (weak) to 41 patches (strong) as the level-1 agent compensates for Hybrid PBVI's retreat.
- Low agent openness: Hybrid PBVI again retreats (weak 171 → strong 150). Hybrid I-POMCP-PF still lifts its effort (102 → 120), but by a smaller margin, narrowing the gap only at neutral strength; compensation is present but muted.

- High agent openness: Both agents collapse to near-identical, single-digit workloads (13 vs 12, 17 vs 11, 12 vs 11). At this scale differences are indistinguishable from noise; the compensatory pattern disappears.
- Across all strength settings, Hybrid PBVI's rate when located at subnetwork climbs to 100 %, and Hybrid I-POMCP-PF's rate climbs past 90 % under high openness, showing that both the agents increasingly utilize their presence inside the environment (i.e., a subnetwork) when the level of agent openness increases.

With no or low agent openness, teammates remain in the environment long enough for Hybrid I-POMCP-PF to model Hybrid PBVI's shrinking workload and raise its own patch count, even while holding a lower patch rate (i.e., waiting when solo action would waste coordination potential). High openness slashes the frequency of subnetwork visits; both agents must patch instantly, driving patch rates to saturation. With no slack left to redistribute, I-POMCP-PF's modelling advantage cannot translate into extra patches—the workload converges to PBVI's.

These results suggest that *level-1 reasoning enables adaptive workload balancing between hybrid teammates*, particularly when defender strength varies under settings when there is no or low agent openness. However, when agent openness becomes high, opportunities for such compensation diminish. In these conditions, Hybrid I-POMCP-PF's behavior converges with Hybrid PBVI's, not due to diminished level-1 reasoning, but because both agents must act immediately when present—leaving little room for anticipatory adjustment.

5.4.6. Research Question 6: How does a high observation error affect the rewards of a level-1 hybrid defender team (1 I-POMCP-PF + 2 PBVI) compared to a homogenous level-0 defender team?

To address this research question, we present the rewards data in **Tables 5.13, 5.14, and 5.15** where observation error is systematically varied across three agent openness settings—none, low, and high—to examine how reward outcomes shift for the Hybrid I-POMCP-PF team relative to PBVI, POMCP-PF, and other baselines, under weak, neutral, and strong defender configurations.

Defender Rewards	LOW OBSERVATION ERROR			HIGH OBSERVATION ERROR		
	NO AGENT OPENNESS					
Defender Reasoning	Scenario_0 (weak)	Scenario_1 (neutral)	Scenario_2 (strong)	Scenario_9 (weak)	Scenario_10 (neutral)	Scenario_11 (strong)
Random	2.99	19.35	36.63	6.04	22.46	39.15
HFD	61.59	125.32	149.26	68.20	127.94	150.51
HSD	46.05	70.14	86.29	42.85	67.66	90.58
PBVI	67.54	110.45	117.03	64.67	103.84	118.37
POMCPPF	63.61	122.74	149.46	75.08	115.49	133.23
IPOMCPPF (Hybrid)	79.11	122.98	150.72	66.52	127.07	153.64

Table 5.13 Rewards–Baseline Scenarios (No Agent Openness) with Low and High Observation Errors, for Weak, Neutral, and Strong Defender settings – All Reasoning Methods.

5.4.6.1. Under No Agent Openness (Scenarios 0–2 vs 9–11)

- **Low** observation error: Hybrid I-POMCP-PF tops the weak (S-0, 79.1) and strong (S-2, 150.7) cases and sits a close second in neutral strength (S-1, 123.0) behind HFD (125.3).
- **High** observation error: With weak defenders, Hybrid falls to third (S-9, 66.5) behind POMCP-PF (75.1) and HFD (68.2). Once defender strength increases, the level-1 Hybrid stabilizes—regaining joint-lead in neutral (S-10, 127.1 vs HFD 127.9) and first in strong (S-11, 153.6).

In stable environments with no agent openness, high observation error pushes the Hybrid I-POMCP-PF agent toward conservative local patching—penalizing reward under weak

defenders but having minimal impact when defender strength is neutral or high. The I-POMCP-PF defender’s level-1 planning now discounts moves toward remotely “suspect” subnetworks (off-subnet error ~ 0.9) and over-values patching the local subnetwork it can still read with relatively higher accuracy (error ~ 0.15). In the weak-defender case, where many subnetworks are simultaneously exploited, this shift produces redundant effort: the Hybrid defender agent lifts its patch rate from $\sim 58\%$ to $\sim 64\%$ (see Figure 5.7 below; S0 vs. S9), and three-defender patches rise from 123 to 136 (see Figure 5.8 below) —a pattern unique to the Hybrid team, as no other reasoning shows a similar increase in triple-patch coordination under high observation error. Because each additional patch carries a cost and leaves other subnetworks uncovered, reward falls ($79.1 \rightarrow 66.5$). In neutral and strong defender scenarios, however, only a few viable options exist: staying together clears them quickly, so the same conservatism becomes an advantage and the Hybrid’s reward rebounds. Level-0 PBVI and POMCP-PF “get away” with similar aggressive patching because their baseline policies were already myopic—noise does not meaningfully alter their decision rule, so their efficiency changes only marginally.

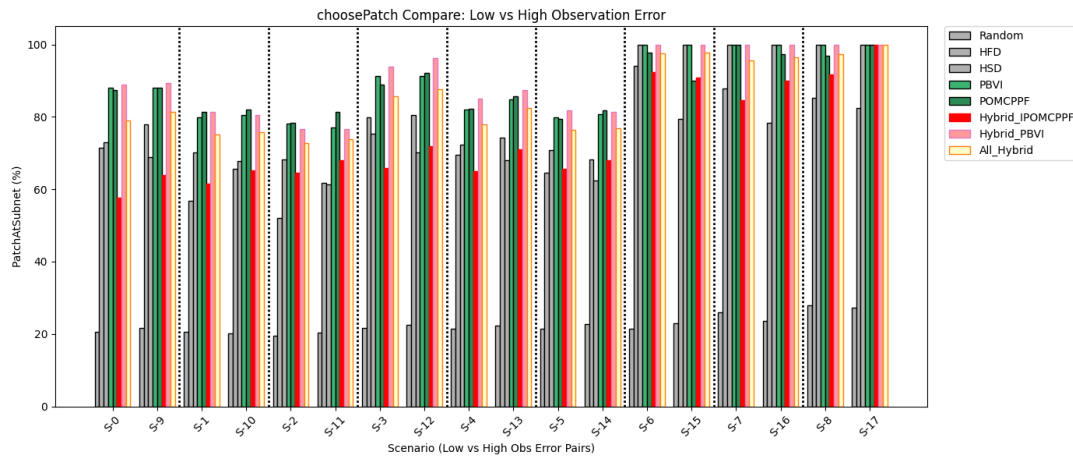


Figure 5.7 Percent of the number of times defender agent chooses Patch action when stationed at a subnetwork – Comparing Low vs. High Observation Settings. Scenarios 0-2 are baselines (no agent openness), 3-5 are baselines + low agent openness, and 6-8 are baselines + high agent openness (and 9 to 17 are counterparts with high observation error)

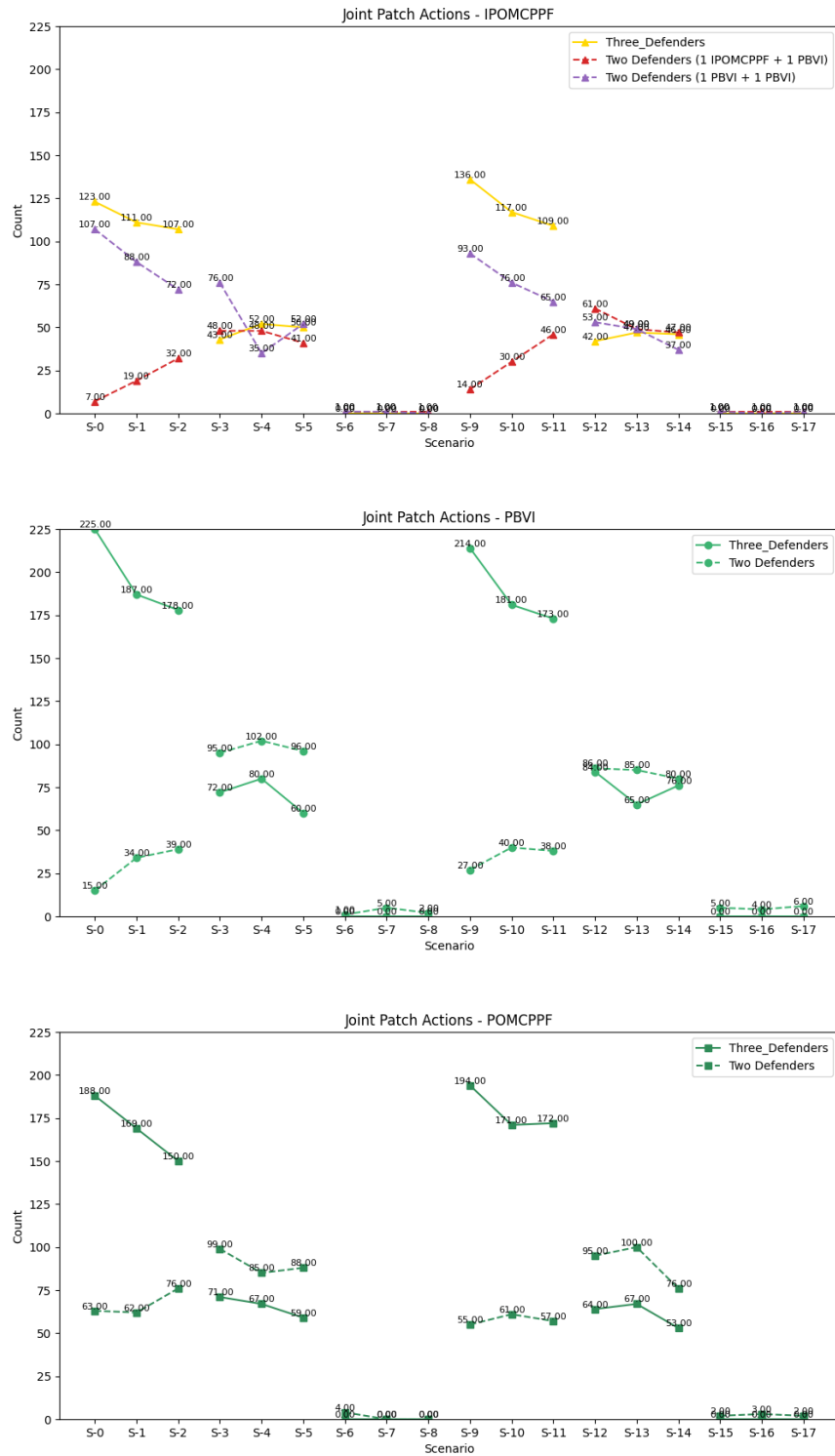


Figure 5.8 Two vs. Three-Agent Joint Patch Actions in I-POMCP-PF, PBVI, and POMCP-PF reasoning methods. Scenarios 0-2 are baselines (no agent openness), 3-5 are baselines + low agent openness, and 6-8 are baselines + high agent openness (and 9 to 17 are counterparts with high observation error)

Defender Rewards	LOW OBSERVATION ERROR			HIGH OBSERVATION ERROR		
	LOW AGENT OPENNESS					
Defender Reasoning	Scenario_3 (weak)	Scenario_4 (neutral)	Scenario_5 (strong)	Scenario_12 (weak)	Scenario_13 (neutral)	Scenario_14 (strong)
Random	0.94	14.11	25.94	5.47	14.77	33.76
HFD	45.95	97.24	116.73	49.42	97.40	130.23
HSD	27.25	55.21	68.81	36.44	53.42	72.93
PBVI	55.22	101.02	107.75	54.34	86.58	108.00
POMCPPF	61.89	104.95	129.95	57.06	102.18	125.34
IPOMCPPF (Hybrid)	46.77	104.74	127.33	52.34	93.72	112.19

Table 5.14 Rewards—Low Agent Openness Scenarios with Low and High Observation Errors, for Weak, Neutral, and Strong Defender settings – All Reasoning Methods.

5.4.6.2. Under Low Agent Openness (Scenarios 3–5 vs 12–14)

- **Low** observation error: POMCP-PF edges Hybrid at every strength setting: e.g., weak S-3, 61.9 > 46.8; neutral S-4, 105.0 > 104.7; strong S-5, 129.9 > 127.3.
- **High** observation error: The gap widens. Hybrid settles mid-pack—third in weak (S-12, 52.3), fourth in neutral (S-13, 93.7), and third in strong (S-14, 112.2)—while POMCP-PF or HFD claim the top spots.

With low agent openness, defenders exit infrequently but do spend short spells at the home node before re-entering, naturally dispersing across subnetworks (unlike when there is no agent openness, and agents tend to start with a specific high-reward target). Under high observation error, the I-POMCP-PF defender agent again raises its patch rate—from ~ 65.8% → 72.0% in the weak case (S-3→S-12), ~ 65.1% → 71.1% in neutral (S-4→S-13), and ~ 65.2% → 67.4% in strong (S-5→S-14) (see **Figure 5.7 above**)—because unreliable remote estimates of the neighbors make local patching comparatively safer. However, this caution does not produce redundancy: three-defender patch events *decline* in all cases—weak: 43 → 42, neutral: 52 → 45, strong: 50 → 42—as agents remain spatially dispersed. Consequently, in the weak scenario the extra patches fill genuine gaps (unlike no-agent-openness scenario where overconcentration produced overlap), and reward rises from 46.8 → 52.3. In neutral and strong settings, however, fewer viable options exist: added patching creates unnecessary effort, dragging reward down

(104.7 → 93.7; 127.3 → 112.2). PBVI and POMCP-PF avoid this drop as their myopic policies remain stable and naturally align with local patching.

Defender Rewards	LOW OBSERVATION ERROR			HIGH OBSERVATION ERROR		
	HIGH AGENT OPENNESS					
Defender Reasoning	Scenario_6 (weak)	Scenario_7 (neutral)	Scenario_8 (strong)	Scenario_15 (weak)	Scenario_16 (neutral)	Scenario_17 (strong)
Random	-11.24	-3.01	5.53	-13.58	-8.44	2.22
HFD	-6.27	7.27	13.23	-2.10	4.60	15.87
HSD	-9.97	-3.70	0.57	-6.40	-2.93	3.33
PBVI	-6.49	8.68	14.02	-8.56	1.28	12.21
POMCPPF	-0.28	3.93	13.10	-5.66	-1.04	17.79
IPOMCPPF (Hybrid)	10.06	1.65	16.86	-2.88	1.37	5.73

Table 5.15 Rewards—High Agent Openness Scenarios with Low and High Observation Errors, for Weak, Neutral, and Strong Defender settings – All Reasoning Methods.

5.4.6.3. Under High Agent Openness (Scenarios 6–8 vs 15–17)

- **Low** observation error: Hybrid still capitalizes on high agent openness when defenders are weak or strong (S-6, 10.1 and S-8, 16.9), but PBVI overtakes in neutral strength (S-7, 7.3 > 8.68).
- **High** observation error: Combined agent openness + high observation error erodes the level-1 edge. Hybrid slides to second in weak (S-15, -2.9) behind HFD (-2.1), clings to third in neutral (S-16, 1.4), and drops to fourth in strong (S-17, 5.7) as POMCP-PF surges to 17.8.

With high agent openness, defender agents may vanish immediately after acting.

Knowing this, every agent—Hybrid or level-0—prioritizes immediate local payoff. For I-POMCP-PF, the compounded uncertainty (unreliable remote subnetwork health and unstable teammate presence) collapses its modeling advantage; planning treats coordination as improbable and prescribes a greedy patch policy (as also noted in above cases with no and low agent openness). Patch rates saturate across all defender strengths—rising from ~ 97.4% to 97.8% in weak defender scenario (S-6→S-15), ~ 95.7% to 96.6% in neutral (S-7→S-16), and ~ 97.4% to 100.0% in strong (S-8→S-17) (see **Figure 5.7 above**). Yet three-defender patch events remain at zero (see **Figure 5.8 above**), as agents are rarely co-present long enough to coordinate.

Under weak defenders, where many subnetworks are simultaneously exploited, and therefore this individualistic behavior leads multiple agents to patch the same subnetwork and neglect others. Although each agent acts promptly, the one-shot nature of highly open agents causes coverage to be uneven—some threats are redundantly handled while others are missed. Consequently, reward falls sharply ($10.1 \rightarrow -2.9$). In neutral and strong defender cases, fewer exploits exist, so this same lack of coordination causes milder—but still avoidable—reward drops ($1.7 \rightarrow 1.4$; $16.9 \rightarrow 5.7$). PBVI and POMCP-PF experience smaller behavioral shifts, since their default policies already favor immediate patching; their efficiency therefore erodes less, keeping their rewards competitive.

Overall, high observation error consistently undermines the Hybrid team’s coordination advantage, but its impact varies with agent openness and defender strength. In stable, closed environments (no agent openness), the I-POMCP-PF agent adapts conservatively, over-patches under weak defenders (hurting reward), but remains efficient when threats are sparse (i.e., when the defender agent is neutral or strong). With low agent openness, this increased patching is less harmful—agents stay spatially dispersed and fill real gaps. However, under high agent openness, coordination becomes nearly impossible: defender agents act reactively and redundantly before vanishing, producing uneven subnetwork coverage. In this setting, the Hybrid’s level-1 planner can no longer differentiate itself behaviorally or strategically, reverting to level-0-like action patterns without achieving comparable efficiency. Thus, when both agent presence and observations are unstable, modeling-based reasoning offers little practical benefit.

5.4.7. Research Question 7: How will defender strength play a role here in level-1 hybrid I-POMCP-PF team – where all agents (both I-POMCP-PF and PBVI) operate with the same strength setting – to handle high observation error?

We have earlier discussed a similar research question (i.e., Research Question 5, Section 5.4.5) on how the strength of the defender agent impacts workload sharing between Hybrid I-POMCP-PF and Hybrid PBVI agents across different agent openness levels (and a baseline low observation error). The current question extends that analysis to a setting where observation error is high. Similar to that question, we examine whether the level-1 I-POMCP-PF agent continues to compensate for its Hybrid PBVI teammate's patching behavior as defender strength rises—or whether this balancing mechanism deteriorates when the observability is noisier.

To compare teammate behavior, we place side-by-side:

- Average patch counts for each agent (i.e., Hybrid I-POMCP-PF, and Hybrid PBVI)
- % of times each defender agent chooses to patch when at subnetwork

Defender strength	Agent openness	Hybrid PBVI patch count	Hybrid I-POMCP-PF patch count	PBVI % patched-when-at-subnetwork	I-POMCP-PF % patched-when-at-subnetwork
Weak	none (S-9)	241.50	158.00	89.40%	64.00%
	low (S-12)	156.00	131.00	96.30%	72.00%
	high (S-15)	17.00	10.00	100.00%	90.90%
Neutral	none (S-10)	217.50	165.00	80.60%	65.20%
	low (S-13)	161.00	113.00	87.50%	71.10%
	high (S-16)	9.50	9.00	100.00%	90.00%
Strong	none (S-11)	207.00	177.00	76.70%	68.10%
	low (S-14)	141.00	119.00	81.30%	68.00%
	high (S-17)	16.00	4.00	100.00%	100.00%

Table 5.16 Hybrid agents patch counts and % patches when at subnetwork (High observation error). Hybrid I-POMCP-PF corresponds to the level-1 agent from the Hybrid team, Hybrid PBVI corresponds to the average of the 2 PBVI agents from the Hybrid team, and PBVI, POMCPPF correspond to the averages of the respective 3-agent homogenous teams.

We make the following observations from the above data with respect to agent openness across different defender strengths under high observation error:

- No agent openness: Just like how we saw with the earlier Research Question 5 (low observation error), as defender strength rises, Hybrid PBVI's patch count steadily falls (weak $\rightarrow$ strong: 241.5 $\rightarrow$ 207.0), while Hybrid I-POMCP-PF's patch count increases (158.0 $\rightarrow$ 177.0). The difference in this patch count narrows notably from 83.5 patches (between the weak Hybrid I-POMCP-PF and Hybrid PBVI defenders) to just 30 patches (between the strong Hybrid I-POMCP-PF and Hybrid PBVI defenders) as the level-1 agent compensates effectively for Hybrid PBVI's retreat.
- Low agent openness: Consistent with our earlier observation, Hybrid PBVI again changes gradually (weak 156.0 $\rightarrow$ neutral 161.0 $\rightarrow$ strong 141.0). Hybrid I-POMCP-PF correspondingly adjusts its own patch effort (131.0 $\rightarrow$ 113.0 $\rightarrow$ 119.0), i.e., by decreasing first and then increasing. Compensation is present but noticeably less pronounced compared to no openness.
- High agent openness: Both agents' patch counts collapse to very low levels (weak: 17.0 vs. 10.0, neutral: 9.5 vs. 9.0, strong: 16.0 vs. 4.0). At these minimal patch counts, differences become indistinguishable from random fluctuations; the compensatory pattern disappears entirely.
- Across all strength settings under high openness, Hybrid PBVI's patch-when-at-subnetwork rate saturates at 100%, and Hybrid I-POMCP-PF's rate similarly approaches saturation (90.9% to 100%), clearly indicating both agents maximize their limited opportunities when agent openness becomes high.

With no or low agent openness, teammates remain in the environment long enough for Hybrid I-POMCP-PF to recognize Hybrid PBVI's decreasing workload and respond by increasing its own patch count, even while maintaining a lower overall patch rate. This pattern mirrors our observations from Research Question 5 (low observation error), confirming the robustness of

level-1 modeling-driven compensation despite noisy observations. However, high openness drastically reduces agents' presence in subnetworks: both must act instantly when present, pushing patch rates toward saturation. With no opportunities left to adjust behavior based on modeling predictions, the Hybrid I-POMCP-PF agent's workload converges closely to that of its Hybrid PBVI teammate.

These findings suggest that, just as we previously observed under low observation error, level-1 reasoning under high observation error also supports adaptive workload balancing between Hybrid teammates, especially when defender strength varies within stable (no or low openness) conditions. Yet under high agent openness, the opportunity for workload redistribution disappears entirely—not because level-1 modeling fails, but because agents must act immediately upon arrival. In such fast-turnover settings, I-POMCP-PF no longer has the temporal flexibility to adjust its behavior based on teammate effort, and its patch count converges with PBVI's by necessity rather than design.

5.4.8. Summary

5.4.8.1. No Agent Openness + Low Observation Error (Research Questions 1, 2, 3)

Taken together, these findings reinforce the idea that coordination in multi-agent systems cannot be achieved through strength (i.e., ability of the agent to manifest the desired change) alone—it must be reasoned for.

Across all three questions, we observe that level-0 methods like PBVI and POMCP-PF exhibit limited adaptability. Their patching activity uniformly declines as defender strength increases, and while this trend superficially aligns with reduced threat levels, it masks a deeper

issue: the lack of awareness about teammate behavior. PBVI’s fixed offline policies and POMCP-PF’s neighbor-agnostic planning both lead to blind spots in coordination—especially in scenarios where team composition or task complexity calls for flexible collaboration.

In contrast, the level-1 Hybrid I-POMCP-PF agent consistently demonstrates targeted adaptability. It achieves high rewards when operating in a fully hybrid team (Question 1), dynamically shifts its pairing behavior to complement its PBVI teammates (Question 2), and strategically increases its patching effort to fill coordination gaps left by level-0 agents (Question 3). Rather than over-committing or under-reacting, the Hybrid I-POMCP-PF agent tailors its actions based on real-time inferences about its teammates’ likely intentions—balancing precision with responsiveness.

Crucially, the hybrid setup does not require all agents to be sophisticated. The simple act of introducing a single level-1 agent into an otherwise level-0 team yields measurable gains in both coordination and reward. This suggests that even modest reasoning heterogeneity can significantly improve performance in decentralized environments—especially when full mutual modeling may be computationally infeasible. As such, level-1 reasoning emerges not as a luxury, but as a necessary ingredient for robust team behavior under dynamic, partially observable conditions.

- Notably, a key driver of the I-POMCP-PF agent’s performance is the alignment between its level-1 predictions and the structured, predictable behavior of its PBVI teammates and the heuristic attacker agents it models. This consistency—on both ally and adversary fronts—provides a stable behavioral scaffold, enabling the level-1 agent to anticipate and adapt effectively. Had its partners been random—or themselves level-1 agents with similarly fluid strategies—whether this advantage would persist remains an

open question. Still, these findings highlight that reasoning heterogeneity—when structured asymmetrically—can unlock coordination benefits that homogeneous teams may miss.

Overall, in stable settings with no agent openness and low observation error, coordination advantages emerge not from brute strength, but from the ability to reason about others. Across Research Questions 1–3, we find that level-0 agents like PBVI and POMCP-PF follow rigid, self-centered strategies—leading to declining coverage as defender strength increases and no clear mechanism to redistribute workload. In contrast, *the level-1 Hybrid I-POMCP-PF agent exhibits context-aware adaptability: pairing effectively with teammates, ramping up effort when coordination gaps appear, and delivering high rewards in mixed teams.* These gains stem not from raw capacity, but from the agent’s ability to model its structured PBVI partners and act in complement. Reasoning heterogeneity—when asymmetric and deliberate—proves to be a computationally efficient lever for better team performance.

5.4.8.2. Agent Openness + Low Observation Error (Research Questions 4, 5)

Our findings demonstrate that the interplay between defender strength and agent openness critically shapes the hybrid team’s coordination behavior and workload distribution. The level-1 I-POMCP-PF defender agent shows highly context-sensitive adaptation: it strategically shifts from cautious, coordination-focused patching to immediate, opportunistic patching as agent openness rises, particularly when defender strength is either very weak or very strong. Conversely, when defender strength is neutral, high agent openness creates conditions in which the hybrid agent’s anticipatory coordination becomes difficult and often counterproductive, causing delayed responses and inefficient patching.

At a more granular level, we observe that defender strength directly influences the I-POMCP-PF agent's ability to compensate for fluctuations in its PBVI teammate's patching effort. Under conditions of no or low openness, as defender strength increases and Hybrid PBVI's patch activity decreases, the I-POMCP-PF agent effectively modulates its own patch count upward to preserve team coverage. However, under high agent openness, the scope for such compensatory balancing disappears. Both defender agents are forced into near-immediate action, converging to similar patch behaviors and rendering the adaptive modeling capabilities of the level-1 agent ineffective—not due to diminished reasoning, but because environmental volatility precludes meaningful coordination opportunities.

All considered, these findings reaffirm that the value of level-1 hybrid reasoning lies not in raw complexity but in its ability to match strategy to context. *When agent openness is low or absent, and across defender agent strength levels, the level-1 I-POMCP-PF agent leverages its modeling capacity to adapt fluidly—filling coverage gaps and smoothing coordination inefficiencies left by simpler teammates. Yet as openness intensifies, agents are denied enough presence together to reason about one another's behavior, and modeling collapses into reactive patching.* In such volatile conditions, strategic asymmetry offers no real edge—underscoring that hybrid architectures succeed not by default, but when their operational conditions practically allow reasonable predictions to take effect.

5.4.8.3. Agent Openness + High Observation Error (Research Questions 6, 7)

Taken together, the findings from Research Questions 6 and 7 reinforce how high observation error significantly influences the Hybrid I-POMCP-PF team's coordination, workload distribution, and reward outcomes across varying levels of agent openness and defender strengths.

Specifically, high observation error consistently diminishes the strategic modeling advantage of the Hybrid defender team, compelling the I-POMCP-PF agent towards increasingly conservative and immediate local patching actions.

Under stable conditions with no agent openness, the Hybrid team's performance deteriorates most notably when defenders are weak, as conservative local patching leads to redundant actions that reduce overall efficiency and reward. Conversely, in neutral and strong defender scenarios, the I-POMCP-PF agent's cautious local strategy becomes an advantage due to fewer viable targets, effectively stabilizing or enhancing reward outcomes. This behavior contrasts sharply with level-0 methods (PBVI, POMCP-PF), whose inherently myopic and reactive policies experience minimal disruption from increased observation noise.

When agent openness is low, the increased caution of the Hybrid I-POMCP-PF agent generally aids in filling genuine coverage gaps without introducing excessive redundancy, thereby preserving or slightly boosting rewards, especially in weak defender settings. However, in neutral and strong settings—where fewer threats exist—the same cautious local behavior becomes less beneficial, leading to unnecessary patches and diminished rewards. Again, level-0 methods retain consistent performance due to their stable, immediate-response strategies.

With high agent openness, the environment's volatility severely limits coordination potential and drastically restricts the time agents remain present in subnetworks. In these scenarios, the Hybrid agent's modeling capability effectively collapses, prompting a wholesale shift toward immediate, opportunistic patching. This shift results in highly uneven coverage and redundancy, dramatically reducing rewards, particularly in weak defender scenarios. Though the Hybrid team's level-1 agent still attempts compensatory behavior in no or low openness

conditions, under high openness, this adaptive compensation entirely disappears—not from diminished reasoning capability, but due to the necessity for immediate action upon arrival.

Collectively, these insights highlight the conditional utility of advanced level-1 modeling and adaptive workload balancing in multi-agent defense scenarios. High observation error introduces complexities that can alter—but not diminish—the value proposition of sophisticated modeling. Under no agent openness, I-POMCP-PF’s performance dips under weak defenders but stabilizes once defender strength is neutral or strong—regaining parity or slight reward advantages over level-0 baselines. However, *as agent openness increases, these benefits erode—revealing that the utility of level-1 reasoning and modeling-driven compensation is contingent on environments with at least moderate stability, where strategic balancing is still possible*. Yet, when agent openness is high, both strategic flexibility and coordination advantages dissolve, underscoring a practical limitation: effective modeling still relies on at least a minimal window of teammate presence. In highly unstable or noisy environments, simpler reactive strategies employed by level-0 methods become comparatively robust, challenging the notion that increased sophistication inherently leads to better outcomes. Nonetheless, when conditions support coordination, level-1 reasoning continues to offer a measured and strategic advantage.

5.4.8.4. Why Level-1 Reasoning Shines Overall

The I-POMCP-PF agent stands out for its adaptable reasoning: it dynamically tailors its actions to teammate behavior, threat landscape, and environmental volatility. When coordination is feasible, it anticipates gaps in team coverage and adjusts effort accordingly—often compensating for less adaptive peers. As conditions grow unstable, it gracefully scales back to

more localized decision-making without collapsing into randomness. This ability to flex between inference-driven planning and context-appropriate responsiveness defines its strength as a level-1 reasoning method and underscores its potential in complex, partially observable domains where intelligent tradeoffs between autonomy and collaboration are key to sustained team performance.

Chapter 6 – Conclusion and Future Work

6.1. Summary

In pursuit of our goal to analyze multi-agent, multi-level decision-theoretic planning in a complex domain such as Cybersecurity Defense—characterized by agent openness, mixed observability, and cooperative-competitive agent dynamics—we first integrated offline level-0 PBVI policies into the I-POMCP-PF algorithm. This was implemented efficiently such that it is computationally feasible to conduct extensive experiments with a complex domain like Cybersecurity Defense. In this process, we also extended the PBVI algorithm to work with mixed observability setting. We then investigated the effectiveness of this algorithm against various naïve and intelligent baselines, i.e., Random reasoning, Heuristic Fixed Defender (HFD), Heuristic Spread Defender (HSD), PBVI, and POMCP-PF. We conducted these investigations across various scenarios featuring different levels of agent openness, agent capabilities (i.e., attacker-defender strengths), and with different levels of observation errors.

We have reported the following key findings from our investigations:

1. The introduction of a single I-POMCP-PF agent improves team-level coordination and reward outcomes across a wide range of **stable** environments (no agent openness or

high observation errors), indicating that even modest heterogeneity in reasoning can yield measurable reward gains.

2. Defender strength directly shapes I-POMCP-PF behavior: its modeling advantage is felt most when team capability is limited but is reduced when no such compensation is needed or possible, thereby demonstrating context-aware engagement.
3. I-POMCP-PF modulates both its teaming and overall patching behaviors based on context: coordinating indirectly with other agents when the other agents' intent is predictable but acting independently when agent openness demands localized responses. In particular, agent openness serves as a key modifier of I-POMCP-PF strategy: with no agent openness, the agent invests more in joint actions; under high agent openness, it defaults to opportunistic local behaviors, revealing its capacity to smoothly shift between anticipatory coordination and context-driven autonomy, depending on the predictability and availability of its teammates.
4. In general, high observation error weakens I-POMCP-PF's ability to model the environment well, leading to worse performance. When the teammate behaviors remain predictable (no and low agent openness), it still modulates its workload effectively, with comparable performances with other baselines. However, under high agent openness with high observation error, I-POMCP-PF defaults to more myopic strategies instead of coordinating.

6.2. Future Work

6.2.1. Investigating Defender Performance Against Intelligent Attackers

We are interested in investigating the extent to which intelligent attacker agents could introduce novel challenges to defender agents employing the I-POMCP-PF framework. In the current

thesis, we limit our scope by assuming attacker agents utilize only heuristic reasoning, targeting primarily the most vulnerable subnetworks. However, because such attacker agents consistently follow predictable patterns, defender agents may quickly learn to recognize and effectively counteract their strategies. In contrast, attacker agents equipped with intelligent reasoning capabilities—potentially modeling defender behavior or adapting dynamically to their patterns—may exhibit more sophisticated and less predictable behavior. This increases the burden on defender agents to maintain accurate belief updates and engage in deeper planning to estimate attacker intent and respond optimally. Such complexity could provide insights into potential scopes for improvement for the I-POMCP-PF algorithm when dealing with cooperative-competitive settings.

6.2.2. Investigate Effectiveness of Using Information-Based Exploration in the I-POMCP Framework

While both the level-0 POMCP-PF and level-1 I-POMCP-PF algorithms rely heavily on the UCB exploration constant [Silver & Veness, 2010] [Sunberg & Kochenderfer, 2018] to balance exploration and exploitation, selecting an optimal value remains a challenge due to domain-specific factors like the transition function and the reward structure. An overly high UCB value leads to excessive exploration, while a low setting risks premature convergence to suboptimal policies. Though we employed manually tuned UCB values in our implementation that were empirically evaluated specifically for the Cybersecurity Defense domain, this approach has a potential downside as the right balance between exploration and exploitation could vary depending on the current belief state (and associated rewards), the level of agent openness, and the level of observability (that could vary with the agent’s internal state).

Information-Based POMCP (IB-POMCP) [Alves et al., 2023] addresses this challenge by embedding an information-theoretic component—specifically, an I-UCB term that rewards reduction in belief uncertainty—into the action-selection criterion, thereby guiding exploration toward states that yield both informative and high-reward trajectories. Empirical evaluations across benchmark domains with sparse rewards have demonstrated that IB-POMCP significantly outperforms the standard POMCP and random-policy baselines while maintaining convergence guarantees. However, these results mostly only address the partially observable and cooperative-agent benchmarks. However, more complex domains, e.g., Cybersecurity Defense with features like cooperative-competitive agents acting under mixed observability could pose unique challenges that have not been addressed yet.

We are therefore curious to see if IB-POMCP’s dynamic, information-guided exploration will offer substantial gains in this setting by enabling agents to strategically reduce uncertainty about both the subnetwork states and the competing neighbor models, potentially outperforming current benchmarks that employ static UCB-variants of this algorithm.

References

- [Alves, et al. 2023] do Carmo Alves, M. A., Varma, A., Elkhatab, Y., & Soriano Marcolino, L. (2023). Information-guided planning: an online approach for partially observable problems. *Advances in Neural Information Processing Systems*, 36, 69157–69177.
- [Aberdeen et al. 2004] Aberdeen, Douglas, Sylvie Thiébaux, and Lin Zhang. Decision-Theoretic Military Operations Planning. *ICAPS*, 2004.
- [Balaji and Srinivasan 2010] Balaji, Parasumanna Gokulan, and Dipti Srinivasan. “An introduction to multi-agent systems.” *Innovations in Multi-Agent Systems and Applications-I* (2010): 1–27.
- [Camerer, et al. 2004] Camerer, C., Ho, T. H., & Chong, J. K. (2004). A Cognitive Hierarchy Theory of Games. *Quarterly Journal of Economics*, 119(3), 861–898.
- [Chandrasekaran, et al. 2016] Chandrasekaran, M., Eck, A., Doshi, P., & Soh, L. (2016, June). Individual planning in open and typed agent systems. In *Proceedings of the Thirty-Second Conference on Uncertainty in Artificial Intelligence* (pp. 82–91).
- [Dorri, Kanhere, and Jurdak 2018] Dorri, Ali, Salil S. Kanhere, and Raja Jurdak. “Multi-agent systems: A survey.” *IEEE Access* 6 (2018): 28573–28593.
- [Eck, et al. 2020] Eck, A., Shah, M., Doshi, P., & Soh, L.-K. (2020). Scalable Decision-Theoretic Planning in Open and Typed Multiagent Systems. *Proceedings of the Thirty-Fourth AAAI Conference on Artificial Intelligence (AAAI’2020)*, New York City, NY, February 8–12, 2020.
- [Gmytrasiewicz and Doshi 2004] Gmytrasiewicz, Piotr J., and Prashant Doshi. “Interactive POMDPs: Properties and preliminary results.” *International Conference on Autonomous Agents: Proceedings of the Third International Joint Conference on Autonomous Agents and Multiagent Systems*. Vol. 3. 2004.
- [Gmytrasiewicz and Doshi 2005] Gmytrasiewicz, P. J., & Doshi, P. (2005). A framework for sequential planning in multi-agent settings. *Journal of Artificial Intelligence Research*, 24, 49–79.
- [Gmytrasiewicz and Doshi 2009] Doshi, P., & Gmytrasiewicz, P. J. (2009). Monte Carlo sampling methods for approximating interactive POMDPs. *Journal of Artificial Intelligence Research*, 34, 297–337.
- [Hattab and Chaari 2021] Hattab, Sondes, and Wided Lejouad Chaari. “A generic model for representing openness in multi-agent systems.” *The Knowledge Engineering Review* 36 (2021): e3.
- [Haksar and Schwager 2018] Haksar, R. N., & Schwager, M. (2018). “Distributed deep reinforcement learning for fighting forest fires.” In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* (pp. 6855–6862). IEEE, 2018.

[Hoang and Low 2013] Hoang, T. N., & Low, K. H. (2013). Interactive POMDP Lite: Towards practical planning to predict and exploit intentions for interacting with self-interested agents. *arXiv preprint arXiv:1304.5159*.

[Kaelbling, Littman, and Cassandra 1998] Kaelbling, Leslie Pack, Michael L. Littman, and Anthony R. Cassandra. “Planning and acting in partially observable stochastic domains.” *Artificial Intelligence* 101.1–2 (1998): 99–134.

[Kakrapudi 2021] Kakrapudi, Anirudh Kumar Maurya. *Decision-Theoretic Planning with Communication in Open and Typed Multiagent Systems*. Diss. University of Georgia, 2021.

[Kakrapudi, et al. 2022] Kakrapudi, A., Anil, G., Eck, A., Doshi, P., & Soh, L.-K. (2022). Decision-Theoretic Planning with Communication in Open Multiagent Systems. *Proceedings of the 2022 Conference on Uncertainty in Artificial Intelligence (UAI’22)*, Eindhoven, Netherlands, August 1–5, 2022.

[Ng, et al. 2012] Ng, B., Boakye, K., Meyers, C., & Wang, A. (2012). Bayes-adaptive interactive POMDPs. In *Proceedings of the AAAI Conference on Artificial Intelligence* (Vol. 26, No. 1, pp. 1408–1414).

[Nguyen, et al. 2022] Nguyen, H., Baisero, A., Wang, D., Amato, C., & Platt, R. (2022). Leveraging fully observable policies for learning under partial observability. *arXiv preprint arXiv:2211.01991*, 2022.

[Ong et al. 2010] Ong, S. C., Png, S. W., Hsu, D., & Lee, W. S. (2005). *Planning under uncertainty for robotic tasks with mixed observability*. The International Journal of Robotics Research, 29(8), 1053–1068, 2005.

[Omidshafiei, Shayegan, et al. 2017] Omidshafiei, S., Agha-Mohammadi, A. A., Amato, C., Liu, S. Y., How, J. P., & Vian, J. (2017). Decentralized control of multi-robot partially observable Markov decision processes using belief space macro-actions. *The International Journal of Robotics Research*, 36(2), 231-258, 2017.

[Pashenkova, Rish, and Dechter 1996] Pashenkova, Elena, Irina Rish, and Rina Dechter. “Value iteration and policy iteration algorithms for Markov decision problem.” *AAAI’96: Workshop on Structural Issues in Planning and Temporal Reasoning*. Vol. 39. Citeseer, 1996.

[Pineau, Gordon, and Thrun 2003] Pineau, Joelle, Geoff Gordon, and Sebastian Thrun. “Point-based value iteration: An anytime algorithm for POMDPs.” *IJCAI*. Vol. 3. 2003.

[Pineau, Gordon, and Thrun 2006] Pineau, Joelle, Geoffrey Gordon, and Sebastian Thrun. “Anytime point-based approximations for large POMDPs.” *Journal of Artificial Intelligence Research* 27 (2006): 335–380.

[Synolakis and Karagiannis 2024] Synolakis, Costas E., and George M. Karagiannis. “Wildfire Risk Management in the Era of Climate Change.” *PNAS Nexus* 3, no. 5 (2024): pgae151. <https://doi.org/10.1093/pnasnexus/pgae151>, 2024.

[Puterman 1990] Puterman, M. L. (1990). Markov decision processes. *Handbooks in Operations Research and Management Science*, 2, 331–434.

[Schwartz, et al. 2022] Schwartz, Jonathon, Ruijia Zhou, and Hanna Kurniawati. “Online Planning for Interactive-POMDPs Using Nested Monte Carlo Tree Search.” *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2022.

[Seraj and Gombolay 2020] Seraj, E., & Gombolay, M. (2020). Coordinated control of UAVs for human-centered active sensing of wildfires. *arXiv preprint arXiv:2006.07969*, 2020.

[Silver and Veness 2010] Silver, David, and Joel Veness. “Monte-Carlo planning in large POMDPs.” *Advances in Neural Information Processing Systems* 23 (2010).

[Sondik 1971] Sondik, E. J. *The Optimal Control of Partially Observable Markov Processes*. PhD thesis, Stanford University, 1971.

[Sunberg and Kochenderfer 2018] Sunberg, Z. N., & Kochenderfer, M. J. (2018). Online algorithms for POMDPs with continuous state, action, and observation spaces. *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*, 28(1), 259–267.

[Wang, et al. 2022] Wang, J., Hong, Y., Wang, J., Xu, J., Tang, Y., Han, Q. L., & Kurths, J. (2022). Cooperative and competitive multi-agent systems: From optimization to games. *IEEE/CAA Journal of Automatica Sinica*, 9(5), 763–783.

[Xie and Liu 2017] Xie, Jing, and Chen-Ching Liu. “Multi-agent systems and their applications.” *Journal of International Council on Electrical Engineering* 7.1 (2017): 188–197.

[Zhang, et al. 2021] Zhang, Kaiqing, Zhuoran Yang, and Tamer Başar. “Multi-agent reinforcement learning: A selective overview of theories and algorithms.” *Handbook of Reinforcement Learning and Control* (2021): 321–384.